Bauhaus-Universität Weimar
Faculty of Media
Degree Programme Digital Engineering

Bauhaus-Universität Weimar

Model checking for Self-Replicating Robotic Systems

Master's Thesis

Dmitry Babaytsev (124065)

1. Referee: Prof. Dr. Jan Oliver Ringert
2. Referee: PD Dr. Andreas Jakoby

Submission date: October 17, 2025

Declaration

I hereby certify that I am the independent author of the submitted work and have not used any sources or resources other than those indicated. In the case of group work, my contribution has been clearly indicated. I am responsible for the quality of the text and the selection of all content. I have ensured that information and arguments are substantiated or supported by suitable academic sources. Any passages, ideas, concepts, graphics, etc. that have been taken verbatim or paraphrased from external sources or my own previous work have been clearly labelled as such and properly cited. All unreferenced content in this work is my own according to copyright law. I am aware that this declaration of originality also refers to the use of non-citable, generative AI (referred to henceforth as »generative AI«).

I am aware that the use of generative AI is not permitted without the express consent of the examiner (release declaration). If I have been permitted to use generative AI, I certify that I have only used it as an aid and that the work is clearly predominated by my own creativity. I assume full responsibility for any machine-generated passages in my work. If I have been permitted to use generative AI, I certify that its use is indicated in a separate appendix included in my work. This appendix contains a statement or detailed documentation on the use of generative AI in accordance with the specifications outlined in the release declaration issued by the examiner. I have included detail on how generative AI was used in my work, including the form, aim, and extent to which it was used. These details conform to the documentation requirements outlined in the release declaration.

I certify that this work has not previously been submitted in the same or similar format to any other examining authority in Germany or abroad. I certify that it has not previously been published in German or any other language.

I am aware that violating any of the aforementioned points is punishable in accordance with examination regulations and may result in my examination performance being assessed as fraudulent and assessed as »failed«. In the event of repeated or serious attempts to cheat, I may be banned from submitting work or taking examinations in my degree programme for a limited time or permanently.

Weimar, October 17, 2025

Dmitry Babaytsev (124065)

Abstract

Self-replicating robotic systems (SRRS) promise great potential for space exploration, in-situ construction, and resilient infrastructure, yet their critical importance and complex interactions make traditional testing insufficient to guarantee safety or correctness of such systems.

This initial research investigates how formal verification and model checking – can be integrated into the design of SRRS to provide provable guarantees of liveness, safety, and functional correctness.

We first survey existing verification techniques and classify state-of-the-art model checking algorithms with respect to their scalability and suitability for robotics. Building on this analysis, we propose a tool-supported workflow that automatically generates NuSMV models corresponding to specific stages of self-replication, based on the initial description of the system. The resulting models are checked against temporal logic specifications that encode key replication properties.

In order to validate the proposed approach, we develop a simulation of the system using Gazebo and ROS. An experimental testbed based on a modular robot simulation serves as the reference platform; we evaluate model-checking variants on representative replication scenarios, and try to use counterexample for control and decision-making in the system.

By linking robot self-replication simulation to rigorous formal analysis, the work provides a reproducible pathway for checking of next-generation self -replicating robots.

Contents

1 Introduction

A concept of self-replication, has steadily gained interest within the field of robotics as a potential paradigm for enabling autonomous construction, scalability, adaptability, and resilience. Self-replicating robotic systems (SRRS) possess the ability to autonomously build replicas of themselves using available components and resources. These systems hold significant promise for applications in remote environments such as planetary exploration, in-situ resource utilization, and disaster recovery, where manual intervention is limited or infeasible [SZC02] [San80].

However, the complexity and autonomy involved in such systems pose considerable challenges to ensuring their correctness, safety, and reliability.

As SRRS evolve from theoretical constructs to practical experiments, guaranteeing their functional integrity becomes crucial—particularly given their recursive nature and potential to affect their environments in unanticipated ways. Traditional testing approaches are not enough when dealing with critical applications, vast state spaces and non-deterministic behavior inherent in self-replicating systems. In response to these challenges, formal verification methods, and in particular model checking, have emerged as powerful tools for rigorously analyzing system behaviors against formal specifications.

Formal methods offer a mathematically grounded approach to verifying that a system adheres to desired properties such as liveness, safety, and correctness.

This research explores the integration of formal verification into the design and analysis of self-replicating robotic systems. The goal is to bridge the gap between the physical nature of replication in robotics and the abstract frameworks used in model checking.

The structure of this work is as follows. Section 2 provides background on formal verification techniques, model checking strategies, and existing self-replicating robotic frameworks. Section 3 outlines the core research questions for the thesis. Section 4 describes the methodology and tools planed to use for robotic simulations, and presents an approach for experimental validation.

Together, these sections aim to contribute a formal framework for analyzing self-replicating robotic systems, supporting their future deployment in safety-critical and autonomous environments.

2 Background

2.1 Formal verification overview

Formal Methods refers to mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems.
Expanding this definition Formal methods can be used for formal description of the system (specification) on some level of abstraction, program synthesis (design) according to specification and verification of a system to prove that model of system under consideration satisfies some properties.

Formal verification techniques can be thought of as comprising three parts [HR04]:

- a framework for modelling systems, typically a description language;
- a specification language for describing the properties to be verified;
- a verification method to establish whether the description of a system satisfies the specification.

These methods and techniques of verification include:

1. SAT solving - operate on propositional logic
2. SMT solving (and tools like Z3) extend this to first-order logic, integrating theories such as arithmetic or arrays.
3. Relational model finding (e.g., Alloy)
4. Model checking (e.g., NuSMV) exhaustively explore state-space against CTL/LTL specifications.
5. Interactive theorem provers (e.g., Coq/Rocq, Isabelle/HOL)

Approaches to verification can be classified according to the following criteria [HR04] pages 172-173:
Proof-based vs. model-based
In a proof-based approach, the system description is a set of formulas Γ (in a suitable logic) and the specification is another formula ϕ. The verification method consists of

trying to find a proof that $\Gamma \vdash \phi$. In a model-based approach, the system is represented by a model M for an appropriate logic. The specification is again represented by a formula ϕ and the verification method consists of computing whether a model M satisfies ϕ ($M \models \phi$). This computation is usually automatic for finite models. Logical proof systems are often sound and complete, meaning that $\Gamma \vdash \phi$ (provability) holds if, and only if, $\Gamma \models \phi$(semantic entailment) holds, where the latter is defined as follows: for all models M, if for all $\psi \in \Gamma$ we have $M \models \psi$, then $M \models \phi$. Thus, we see that the model-based approach is potentially simpler than the proof-based approach, for it is based on a single model M rather than a possibly infinite class of them.

Degree of automation.
Approaches differ on how automatic the method is; the extremes are fully automatic and fully manual. Many of the computer-assisted techniques are somewhere in the middle.

Full- vs. property-verification.
The specification may describe a single property of the system, or it may describe its full behavior. The latter is typically expensive to verify. Intended domain of application, which may be hardware or software; sequential or concurrent; reactive or terminating; etc. A reactive system is one which reacts to its environment and is not meant to terminate (e.g., operating systems, embedded systems and computer hardware).

Pre- vs. post-development.
Verification is of greater advantage if introduced early in the course of system development, because errors caught earlier in the production cycle are less costly to rectify. (It is alleged that Intel lost millions of dollars by releasing their Pentium chip with the FDIV error.

2.1.1 SAT solvers

Propositional logic - consider propositions and relations between them and constructions of argument based on them.

Propositions - facts or statements about the world that can be true or false . Also defined as atomic propositions or Atoms. Some propositions can be described in natural language as atomic or indecomposable declarative sentences) [HR04].

Atomic propositions can be connected to each other with logical connectives and compose propositional formulas. The most common connective relations are:

- Negation $\neg$
- Conjunction $\wedge$

- Disunction $\vee$
- Implication $\rightarrow$

The Backus Naur form (BNF) for propositional logic well-formed formula:

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

where p - is any atomic proposition.

Based on these we can construct a system of reasoning: from some set of formulas we can infer (prove) another formula that can be true or false:

$$\phi_1, \phi_2, \phi_3, ...\phi_n \vdash \psi$$

Such formulas called sequents, where on the left side there is a set of formulas called premises and on the right side - a formula called conclusion

Another relation between logical formulas is semantic entailment:

$$\phi_1, \phi_2, \phi_3, ...\phi_n \models \psi$$

it means that if propositional logic formulas $\phi_1..\phi_n$ evaluate to True, then formula ψ also evaluates to True.

The formula ϕ is valid if for all valuations of atomic propositions in formula it valuates to True:

$$\models \phi$$

SAT solver is a program which aims to solve the Boolean satisfiability problem (SAT), where the Boolean satisfiability problem sometimes called propositional satisfiability asks whether there exists an interpretation that satisfies a given Boolean formula.

Given formula ϕ is **satisfiable** if it has a valuation in which it valuates to True.:

In other words, it asks whether the formula's variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable.[HR04].

SAT problem is NP-complete, as a result, only algorithms with exponential worst-case complexity are known. In spite of this, efficient and scalable algorithms for SAT were developed during the 2000s, which have contributed to dramatic advances in the ability to automatically solve problem instances involving tens of thousands of variables and millions of constraints.

SAT solvers usually begin by converting a problem to conjunctive normal form (CNF). **Conjunctive normal form (CNF)** - as a conjunction of clauses, where each clause is a

disjunction of literals. Literal is an atomic proposition or a negated atomic proposition. Conjunctive normal form allows easy check of validity of a formula which otherwise take times exponential to number of atoms.

2.2 Model checking

Model checking is based on temporal logic. The model in temporal logic described as a system with few states that can change with time. The formula is not statically true or false in a model, as it is in propositional and predicate logic, it can be true in some states and false in others, the static notion is replaced by a dynamic one [HR04].

In model checking, the models M are transition systems and the properties ϕ are formulas in temporal logic. To verify that a system satisfies a property, we must do three things:

1. model the system using the description language of a model checker, arriving at a model M;
2. code the property using the specification language of the model checker, resulting in a temporal logic formula ϕ;
3. Run the model checker with inputs M and ϕ.

Model checking is a core approach in Formal verification that involves an algorithmic search of the state space of a system model to check if a given property (specification) holds, where:
Model- is an abstract representation of the system (often a state-transition graph or a program) capturing its possible states and transitions. The model can be described in a modeling language (like Promela for SPIN, or as a transition system in a tool like NuSMV).
Specification- is a formal property that the model should satisfy, commonly expressed in a temporal logic such as LTL or CTL. For example, an LTL specification might state that "whenever request happens, eventually response occurs" or a CTL property might require that "in all execution paths, a certain error state is never reached."

2.2.1 Classification of Model checking methods

Model checking can be classified based on time concept into: **linear-time temporal logic** and **branching time logic.** [HR04]

Linear-time temporal logic

Linear-time temporal logic (LTL) considers time as a set of paths, where each path is a sequence of time instances [HR04]. LTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$
$$\mid (X\phi) \mid (F\phi) \mid (G\phi) \mid (\phi\ U\ \phi) \mid (\phi\ W\ \phi) \mid (\phi\ R\ \phi)$$

The connectives X, F, G, U, W, R called temporal connectives, because they describe relations between states in different moments of time. The meaning of these connectives described in Table 2.2.1

X	Next	ϕ must hold in the very first successor state on the chosen path.
F	Finally ($F\phi$)	ϕ must hold at some future state on the path (i.e., "eventually ϕ").
G	Globally ($G\phi$)	ϕ must hold at every state on the entire path ("always ϕ").
U	Until ($\phi U \psi$)	ψ must be true at a future state, ϕ must stay true at every state before that one.
W	Weak until ($\phi W \psi$)	either ($\phi U \psi$) holds or ϕ stays true forever if ψ never becomes true.
R	Release ($\phi R \psi$)	ψ must hold up to and including the first state where ϕ becomes true; if ϕ never becomes true, ψ must hold forever (the dual of Until).

Table 2.1: LTL temporal connectives

In LTL, model can be represented as a transition system [DGL16].
Transition system T:

$$T = (S, \overset{a}{\rightarrow}, L, s)$$

where:
S - set of states, called the space states of T,
$\overset{a}{\rightarrow}$ - transitions (binary relation of states), associated with some action $a \in A$
L - labelling function that reflects states to propositional atoms
s - initial state of T, can be omitted if we want to describe only global structure.
Sometimes if we consider only one action label we can omit labeling transition: $\rightarrow$.
A path π in a transition system T is a finite or infinite sequence of states and (optionally) actions which transform each state into its successor: $s_0 \overset{a_0}{\rightarrow} s_1 \overset{a_1}{\rightarrow} ...$

Trace or computation in a transition system is a finite or infinite sequence of state descriptions along the path: $L(s_0) \xrightarrow{a_0} L(s_1) \xrightarrow{a_1} ...$

Branching temporal logic (CTL)

Branching temporal logic represents time as a tree rooted in present and branching into the future, the other name is Computation Tree logic (CTL). LTL implicitly quantifies universally over paths [DGL16]. Therefore, properties which assert the existence of a path cannot be expressed in LTL. We can solve this problem by considering the negation of the property in question. For example to check whether there exists a path from s satisfying the LTL formula ϕ, we check whether all paths satisfy $\neg\phi$; a positive answer to this is a negative answer to our original question, and vice versa. We used this approach when analysing the ferryman puzzle in the previous section. However, properties which mix universal and existential path quantifies cannot in general be model checked using LTL approach, because the complement formula still has a mix. Branching-time logics solve this problem by allowing us to quantify explicitly over paths.

CTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \bot \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

$$\mid (AX\phi) \mid (EX\phi) \mid (AF\phi) \mid (EF\phi) \mid (AG\phi) \mid (EG\phi) \mid A[\phi\ U\ \phi] \mid E[\phi\ U\ \phi]$$

New introduced quantifiers A and E mean:
$A\phi$ - formula ϕ for all paths
$E\phi$ - exists a path that formula ϕ
Connectives X,F,G cannot occure without being preceded by A or E.
Weak until W and release R are usually not included in CTL, but can be derived.

Temporal logic (CTL*)

Statistical Model Checking (SMC)

Some systems have huge or continuous state spaces (e.g. complicated analog components, or software with random timing), or they exhibit probabilistic behavior (randomized protocols, hardware failures) where one is interested in quantitative properties (like "the request will be served within 1s with probability at least 0.99"). Statistical

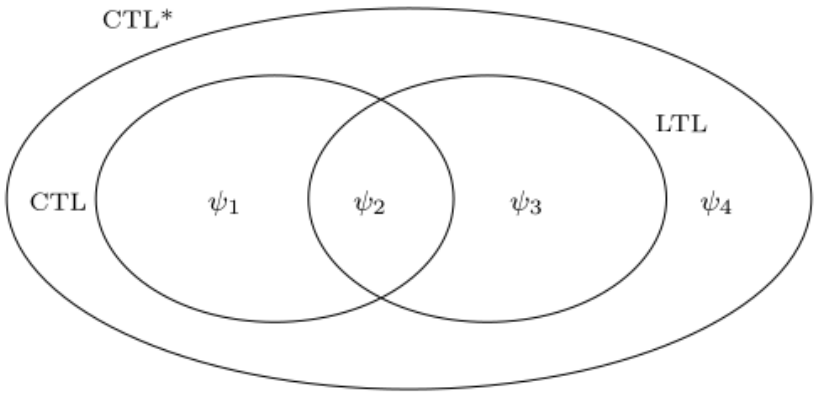

Figure 2.1: LTL, CTL and CTL* logic relation [HR04]

Model Checking (SMC) is an approximate verification technique that uses simulation and statistical methods instead of exhaustive exploration [YS02].

SMC treats the model checking problem as a statistical inference problem [You06]. It executes a number of simulations (random or guided) of the system. Checks whether each simulation trace satisfies the property ϕ (which often is a bounded temporal logic property like “within T time units, something happens”). Based on the outcomes, it uses statistics (like hypothesis testing or estimation with confidence intervals) to infer with high confidence whether the property holds with required probability.

SMC was pioneered by Younes in the 2000s [YS02]. There are two main modes: hypothesis testing (e.g. "check if the probability $P(property) > p$ with confidence 95%") and estimation (estimate the probability itself within some error bound).

They adopt the continuous stochastic logic (CSL) as our formalism for expressing probabilistic real-time properties of discrete event systems. CSL has previously been proposed as a formalism for expressing temporal and probabilistic properties of continuous-time Markov chains (CTMCs) by Aziz et al. [Azi+96].

A CSL formula is either a state formula or a path formula.

The Backus Naur form (BNF) for CSL logic well-formed formula:

State formulas:

$$\phi ::= \text{true} \mid a \mid \neg\phi \mid \phi \wedge \phi \mid P_{\leq\theta}(\rho)(\phi)$$

Path formulas:

$$\rho ::= X\phi \mid \phi U_{\leq t}\phi$$

Where ϕ is a state formula and ρ is a path formula.

Tools like PRISM and its SMC extension, described in [KNP11], STORM [Deh+17], or PLASMA Lab [LST16] implement statistical model checking for various kinds of stochastic systems.

The obvious advantage is that SMC sidesteps the state explosion by not exploring the full state space – instead, it samples paths. This makes it scalable to systems with enormous or even infinite state spaces, as long as we can simulate the system. It is also applicable to systems with continuous dynamics or very large numbers of states where traditional model checking is infeasible . Another benefit is ease of implementation: if there is a simulator for your system, adding a statistical checker on top is easier than building a full model checker. SMC naturally supports probabilistic properties – e.g., verifying properties of a Markov Chain or Markov Decision Process by sampling behaviors and using results like Chernoff or Hoeffding bounds to determine if the property likely holds. Because it uses actual simulations, SMC can handle very complex nonlinear or black-box system aspects (treating them as black boxes that just produce simulation traces).

The trade-off is that SMC provides a confidence result, not a guarantee. It is probabilistic verification: it might say "with 99% confidence the property holds with probability ≤ 0.97". There's a small chance of error in the verification result (false positives/negatives bounded by the confidence level). Also, SMC may require a large number of simulations to reach high confidence, especially for rare events (if the property failing is a rare event, one might need many samples to observe it even once). Rare property verification is a known challenge – advanced importance sampling or rare event simulation techniques might be needed in such cases.

SMC can't truly prove a property in the mathematical sense – it can only give statistical evidence. If a property almost always holds but has a corner-case failure, SMC might miss it if that scenario has very low probability and isn't sampled. Thus, it sacrifices completeness. Another limitation is that SMC typically handles quantitative properties (often expressed in probabilistic temporal logics like PCTL or BLTL) – it's not usually used for pure logical properties that are either true or false; it's more for properties like "the probability of an event is at least p".

Runtime Verification and Monitoring

Runtime verification (RV) is a technique that involves monitoring a running system (or execution traces produced by it) against a formal specification [HSZ14]. Unlike the other methods discussed, which are largely offline (analyzing a model mathematically), runtime verification is applied during execution. It can be seen as a lightweight formal method, sitting somewhere between testing and model checking: you instrument the

system with monitors, and these monitors check whether the execution is satisfying the desired properties. If a violation is detected, it can be logged, or in some cases, a recovery action can be taken.

Key components of runtime verification include:
- A set of monitors or runtime assertions derived from the formal specification (often temporal logic formulae or automata on traces).
- An instrumentation of the system to report events to the monitors.
- The monitors run in parallel with the system (or on logged traces)
and detect property satisfaction or violation on the fly.

For example, if you have a property "whenever interrupt is raised, service routine runs within 100ms," a runtime monitor can watch for interrupts and check timing until service routine is called, flagging an error if the 100ms deadline is missed.

Runtime verification avoids the state explosion problem entirely, because it doesn't explore all behaviors — it only checks the actual behaviors that occur. Runtime verification avoids the complexity of traditional formal verification techniques (like model checking and theorem proving) by analyzing only one or a few execution traces and by working directly with the actual system. This means it scales well to very complex systems, since you're just running the system (or a high-fidelity simulation of it) and the overhead is only in checking the property on that run. It also has the advantage of checking the real system (no abstraction or modeling error, since the code that runs is what's verified against the property , at least for that run). It can catch errors in scenarios that were not anticipated by designers as soon as they occur in the field or during testing, thus serving as a powerful debugging and validation aid. Runtime verification can be used continuously in deployed systems for critical properties (e.g., a monitor that ensures no unauthorized file access operations occur, and triggers an alarm if they do, providing a layer of security monitoring). Another plus is that runtime verification can sometimes detect issues that static model checking might miss due to environment assumptions. At runtime, the real environment provides inputs and the monitor can see violations under real conditions, including those hard-to-model aspects.

The fundamental limitation is that runtime verification is not exhaustive. It can only inspect the executions that actually happen (or a sample of them). If a particular bug does not manifest in the observed runs, RV cannot detect it. It provides no guarantee that "no bug exists" — only that "no bug was observed in the runs we monitored". In other words, it lacks completeness/coverage , unlike model checking which (for a model) covers all behaviors by design. Another challenge is performance overhead: monitors consume resources, and if properties are complex (say, specified in LTL, which might require monitoring a suffix of the trace), the runtime checking might slow the system or require buffering events. However, in many cases monitors can be optimized or run

on a separate core, and the overhead is manageable (reports of a few percent overhead for simple monitors are common). Additionally, writing good monitors (formal properties) and instrumenting the system requires effort. It's easier than full verification, but still requires formal specification skills. If one instrument too much or monitors too many properties, the overhead can become significant, or the system 's real-time behavior might be perturbed.

Runtime verification is often used in safety-critical systems as an additional runtime safety net. For instance, in aerospace or automotive software, one might include monitors for conditions like "no two actuators turn on simultaneously" or "if sensor reading exceeds threshold , system enters safe mode within 1 second," etc. NASA has explored RV for spacecraft software monitoring during missions [Sla+24](since you cannot model check the entire flight code with all analog intricacies, but you can monitor key properties as the mission progresses). Runtime Verification has also become popular in security (e.g., monitoring program execution for compliance with security policies). For example, a monitor can watch system calls to detect a sequence that matches a known attack pattern or a violation of a security automaton. Another domain is testing: during testing, using runtime monitors can greatly increase the chance of noticing an issue. Instead of manually inspecting outputs , formal monitors can rigorously check that each test execution satisfies the expected temporal properties. This is often called "monitoring oracles" in testing – formal specifications serve as test oracles at runtime. In summary, runtime verification does not replace model checking but complements it. Where model checking cannot be applied (due to scale or undecidability), or in deployed scenarios where you want continual assurance, RV provides a way to leverage formal specifications dynamically. It trades completeness for scalability and immediacy.

2.3 Self-replicating robotic systems

2.3.1 Self-replication introduction

Self-replication is the process by which a dynamical system constructs an identical—or very similar—copy of itself. According to Sipper, replication is an ontogenetic, purely developmental phenomenon: it involves no genetic- or code-modifying operators and yields an exact duplicate of the parent organism.

Replication must therefore be distinguished from reproduction, a phylogenetic, evolutionary process that employs genetic operators (e.g., crossover and mutation in biology) and ultimately generates diversity and evolution [Sip98].

This study focuses exclusively on replication and self-replication, code-altering and evolutionary processes are beyond its scope.

Self-assembly refers to the autonomous organization of components into patterns or structures without human intervention. In robotics, this term may describe either (i) a kinematic machine that manipulates discrete parts to build an assembled copy of itself (kinematic self-replication) or (ii) a collective of mobile robots that can autonomously connect, disconnect, and reconfigure themselves (swarm robotics and robotic self-reconfiguration [Tan+20]).

When the replication process is controlled solely by the system being reproduced, it is called a self-replicating system [LC07]. In contrast, if the replication process requires external elements that actively control and manipulate resources and direct the process, it is simply a replicating system. A biological analogue is a virus, which replicates by exploiting a host cell.

A self-replicating machine or self-replicating robotic system (SRRS) is thus an autonomous robot capable of manufacturing a copy of itself from raw materials, or from minimal possible units related to considered environment, thereby exhibiting self-replication in a manner analogous to that observed in nature.

As shown by Chirikjian in [Chi22] "the basic difficulty is that in order to close the self-replicating loop, there must be a balance between the overall system complexity and the ability of the system to handle the simplest inputs possible. Generally speaking, the more complex the system is, the more capable it is to assemble the simplest parts, or even harvest raw materials. But when the system is very complex, it then requires more to reproduce. For example, if a robot needs microprocessors and a 3D printing system needs a high-energy laser, then producing those from in situ resources becomes an enormous challenge." And "One way around that problem is to focus on production

of mechanical parts in situ such as structural members for robots, factories, and habitats, and to send the relatively light-weight “vitamin” components such as computers, lasers, and some chemical reagents to be considered as inputs to the system rather than as items to be replicated. In this way, a practical version of in situ resource utilization (ISRU) can be achieved by reclassifying inputs. Moreover, developing systems that are depen- dent on some inputs that the systems cannot manufacture from scratch is both more akin to living systems that require specific nutrients, and is also safer than developing self-replicating systems that could continue to replicate without the imposition of resource bottlnecks. Without such intentional bottlenecks there could be fears of things running amuck for generations."

2.3.2 Classification of Self-replication systems

Type-related replication

Building upon the foundational work of Jones [Jon21] and works of Lee and Chirikjian , self-replicating systems can be systematically classified according to their type-related replication architecture into three primary categories:

Centralized replication system where robots can produce only robots without replication functionality as is shown on Figure 2.2a.
The centralized approach has several distinct advantages: it provides precise control over population growth, ensures quality consistency through standardized manufacturing processes, and reduces resource competition among robots. However it also introduces significant vulnerability through single points of failure - if the replicating robots are damaged or destroyed, the entire system’s reproductive capacity is compromised.

Additionally, the concentration of replication resources that have to be located or transported near the replicating robots may create bottlenecks that limit the system’s expansion rate and adaptive responsiveness.

Decentralized replication represents a system architecture where every robot within the system possesses full replication capabilities, see Figure 2.2b. This distributed approach is inspired by cellular division processes, where each entity contains the complete blueprint and machinery necessary for creating identical copies of itself.

The decentralized model excels in resilience, scalability and speed of self-replication. The system can continue expanding even if individual robots are eliminated, as each surviving unit has complete reproductive autonomy. This architecture facilitates rapid colonization of new environments and provides redundancy against catastrophic failures. However, decentralized systems face challenges in coordination and control for

some systems, possible uncontrolled exponential growth, and consistent quality across independently operating replication processes.

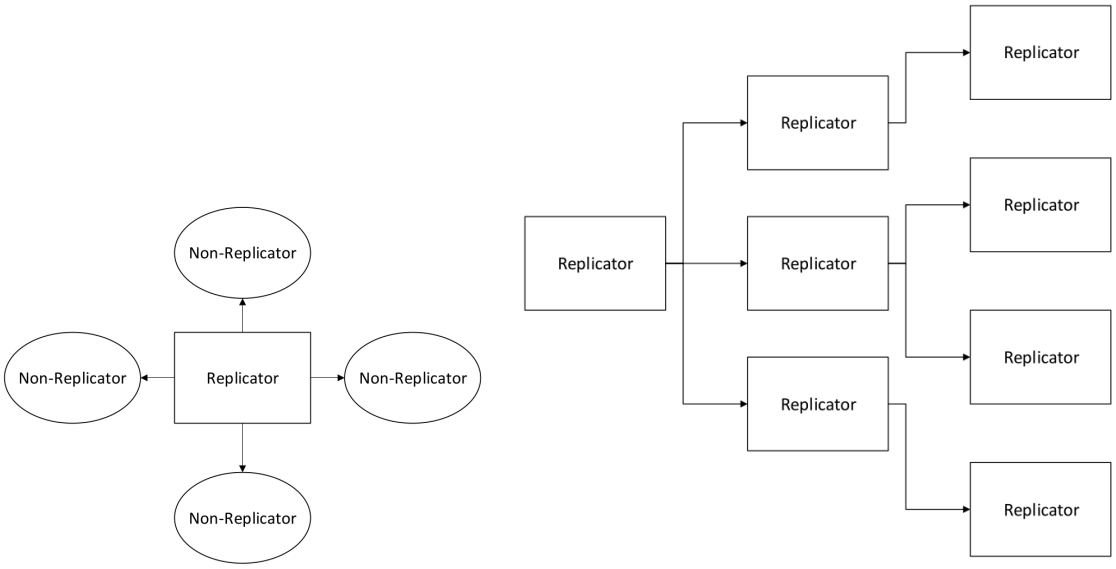

Figure 2.2: Centralized and decentralized replication [JS21]

Hierarchical replication architectures represent a hybrid approach that combines elements of both centralized and decentralized strategies. In these systems, robots with replication capabilities can produce two distinct categories of offspring: specialized robots lacking replication ability (similar to centralized systems) and fully autonomous robots capable of self-replication (similar to decentralized systems), as is shown on Figure 2.6.

This multi-tiered approach enables dynamic adaptation to environmental conditions and mission requirements. The system can produce specialized worker robots for specific tasks while simultaneously maintaining a population of replicator robots to ensure continued expansion and redundancy.

The hierarchical model allows for optimized resource allocation, where replication-capable robots can focus on expansion while non-replicating robots specialize in operational tasks, environmental sensing, or resource collection.

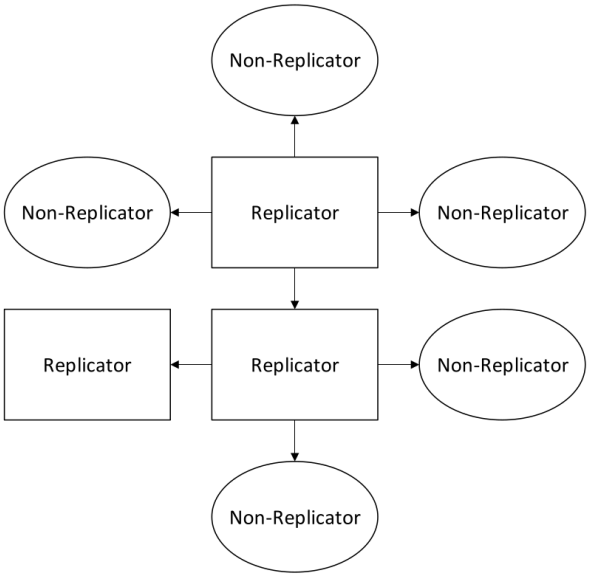

Figure 2.3: Hierarchical replication [JS21]

Population Composition Classification

The composition of robot types within an SRRS fundamentally influences system behavior, specialization potential, and adaptive capacity. This classification dimension addresses the diversity of robotic types operating within the system.

Homogeneous Systems Homogeneous SRRS consist of robotically identical units, where all entities share the same physical design, computational capabilities, and behavioral programming. This uniformity creates predictable system dynamics and simplifies coordination protocols, as all robots operate with identical sensor suites, actuator capabilities, and decision-making algorithms.

The homogeneous approach offers advantages in replication process consistency, simplified communication protocols, and predictable emergent behaviors. Resource allocation becomes straightforward when all units have identical requirements, and system-wide updates or modifications can be uniformly applied. However, homogeneous systems may struggle with task specialization and environmental adaptation, as the lack of diversity limits the system's ability to optimize for varied operational requirements.

Heterogeneous Systems Heterogeneous SRRS incorporate multiple distinct robot types, each potentially specialized for specific functions, environmental conditions, or operational roles. This diversity enables flexible division of labor, where different

robot types can be optimized for exploration, resource gathering, construction, maintenance, or replication tasks.

The heterogeneous model provides enhanced adaptability and operational efficiency through specialization. Different robot types can be designed with complementary capabilities, creating synergistic effects that exceed the performance of individual units. This approach enables the system to respond more effectively to complex environmental challenges and mission requirements.

However, heterogeneous systems require sophisticated coordination mechanisms to manage inter-type communication, resource distribution, and collaborative decision-making across diverse robotic platforms.

Interaction with environment or external system

Another classification of self-replicating robotic systems can be made based on the degree of autonomy and reliance on external systems, as it described in [LC07]. A fundamental distinction is made between passive and active self-replication:

Passive self-replicating systems do not possess inherent mechanisms to actively reproduce themselves. However, given an appropriate environment and the right configuration of components, replication can still occur. A canonical example of this is the Penrose block replicators, which demonstrate self-replication through mechanical constraints and spatial arrangement without any active control [Pen58].

Active self-replicating systems, on the other hand, are equipped with the functional capacity to carry out the replication process. These systems may still rely on passive environmental structures (such as fixtures or gravity) to assist in replication, but are actively engaged in constructing replicas of themselves.

Interaction with environment or external system, alternative classification

A more detailed and operationally useful classification is introduced by Suthakorn, Zhou, and Chirikjian [SZC02]. This framework emphasizes how SRRS interact with their environments and whether replication is accomplished directly or through intermediate stages. Based on this framework, all SRRS are categorized into two primary classes:

Directly replicating SRRS - in which a robot is capable of constructing an exact copy of itself in a single generational step. These systems can be further divided into the following subcategories, based on their reliance on fixtures and replication strategy:

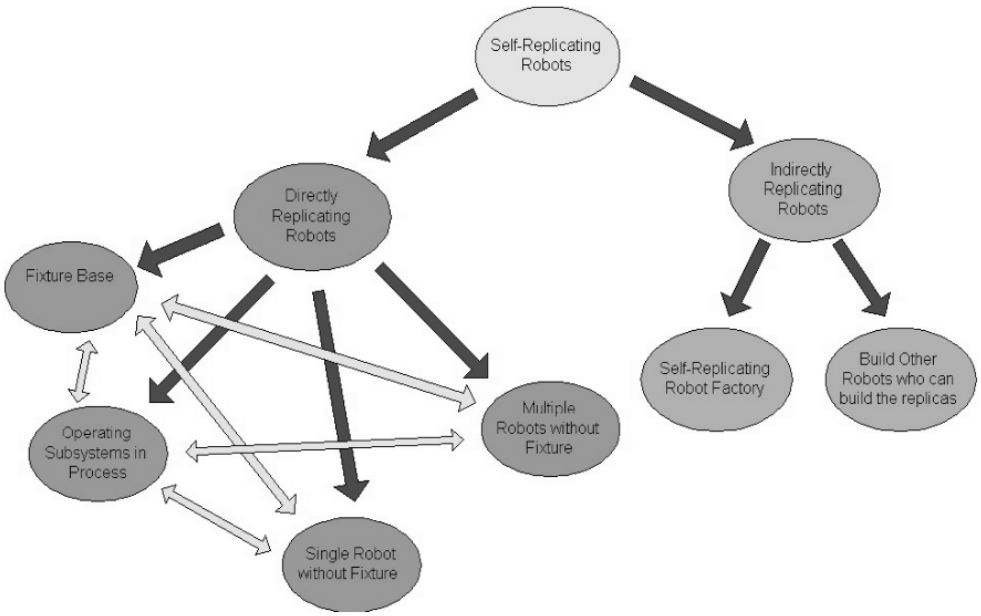

Figure 2.4: Directly and indirectly replicating robots classification [SKC03]

Directly Replicating Robots can be further classified into following groups:

- Fixture-Based Group The self-replicating robots in this group depend on external fixtures in order to complete the self-replication process. Passive fixtures are able to assist in this because of the shape constraints that they impose. In some other cases, to unify subsystems, push-pull fixtures are helpful as well.
- Operating-Subsystem-in-Process Group In this group one or several subsystems of the replica can operate before the replica itself is fully assembled. These subsystems are able to assist the original self-replicating robot during the assembly of the replica.
- Single-Robot-Without-Fixture Group In this group only one robot is used to finish the self-replication process. Thus, the robot in this group depends only on the available environment. Usually, the complexity of the subsystems or the number of subsystems in the replica is very low for this group.
- Multi-Robot-Without-Fixture Group In this group more than one robot works together in the self-replication process without the assistance of fixtures. A major advantage is the reduction of the time required for self-replication. A disadvantage is that there may be interference problems among robots. There are several possible ways that a self-replicating robot can be categorized in two or

three groups mentioned above. The combination of two or three different concepts can be incorporated in a potential design, such as a combining operating-subsystems-in-process with fixture-based robots. More categories are likely to be developed in the subsequent stages of our research in the area of self-replicating robots.

Indirectly replicating - where replication is achieved through intermediate stages rather than a direct one-to-one duplication. The original robot initiates the creation of other robots or constructs a specialized environment that facilitates replication. Indirect replication strategies include:

- Self-Replicating Robot Factory A robot or group of robots constructs a fabrication environment — a factory, that can then autonomously produce replicas of the original. This approach encapsulates the replication process into a fixed infrastructure, enabling high throughput and modular production.
- Intermediate Robot Builders Instead of replicating itself directly, a robot creates new robots that are not replicas, but are instead specialized for manufacturing. These intermediate agents then produce replicas of the original robot. This cascading replication strategy introduces flexibility and specialization but may involve increased system complexity.

2.3.3 Self-replication models

The concept of self-replicating systems originates from the foundational work of Neumann and Burks, who in 1962 proposed the first theoretical model of a self-reproducing automaton [NB62]. This model, developed in the context of cellular automata, formalized the minimal components necessary for a machine to replicate itself.

According to this model, a self-reproducing automaton consists of four key components:
A (constructor): an automatic factory that collects raw materials and processes them into outputs using a specified written instruction,
B (copier): that takes an instruction and copies it,
C (controller): that controls A and B, actuating them alternatively,
D (instructions) A coded description of the automaton's structure and function, acting as its "genetic" information.

$$(A + B + C + D) \rightarrow (A + B + C + D), (A + B + C + D)$$

And the replication process in von Neumann's model, is when the automaton $(A + B + C + D)$ produces another $(A + B + C + D)$.

However, while von Neumann's model is conceptually complete for information-theoretic or logical systems, it exhibits limitations when applied to physical robotic systems. Specifically, it doesnt describe interaction with a physical environment, resource acquisition, resource transformation and management of by-products, energy, and mechanical constraints.

To address these limitations, a more general and physically grounded model was proposed by Lee and Chirikjian [LC07]. In this extended model, the replication process is formalized as a transformation: Lee and Chirikjian proposed broader model, where a replication process can be written as

$$(R, M, E) \rightarrow (R', M', E') \tag{2.1}$$

where:
M is a multiset of available parts to be used for building replicas by an initial system
R is a multiset of an initial functional system to be reproduced. $R \neq 0$; to be a replicating system.
E is a multiset of environmental structures and/or elements involved in the replication process (but not replicated).
R', M', E', the replicated entities, the remaining or altered material and the possibly changed environment respectively.

In this model R describes the whole replicator $A + B + C + D$ from Von Neumann's model.

This transformation can also be expressed functionally as:

$$(R', M', E') = \Phi(R, M, E, t) \tag{2.2}$$

Where Φ denotes the replication function and t is time. This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and environment.

The model explicitly incorporates environmental interaction (E), which plays a critical role in physical self-replication. As proposed by Eno, Mace, Liu, et al. [EML+07], environments can be categorized based on the level of structure and predictability:

- Completely structured environment - in which every environmental structure is fixed at a precise location and there is no modification or change in any environmental structures during the self-replication process. If we define E' is the set of environmental structures after self-replication process: $E = E' \neq \phi$

- Partially structured environment is like a structured environment, but with structures whose locations can be permuted and perturbed by a small random error in pose during the self-replication process. $E \neq E \neq \phi$
- Unstructured environment has no predefined or organized structure and can dynamically change during the self-replication. Can be formally depicted as: $E = \phi$ where ϕ represents a stochastic or unknown environmental process.

2.3.4 Degree of self-replication

As shown in [Chi22]: "In an artificial self-replicating system, if the number of input parts is small, and if each part is itself a complex machine, then the effort involved in the assembly step of self-replication is relatively low compared to the effort of making the input parts. The assembly task in self-replication is then relatively unchallenging in such a scenario. In contrast, if there are many simple (easy to manufacture) parts that need to be assembled by the robot, the relative difficulty is higher. The difference between these two scenarios can be quantified using the concept of the 'degree of self replication'. The highest degree of self replication is when raw materials are harvested by a system and used to produce a replica, since fabrication and assembly are both done by the self-replicating system in that scenario."

In general as shown in [Chi22] degree of self-replication can be abstractly described as:

$$DOSR = \frac{System\ Complexity}{Parts\ Complexity}$$

In 2007 Lee and Chirikjian proposed a measure of degree of replication [LC07]: measure of how much order is created by the assembly process relative to the existing order in the unassembled parts. In this section, we first define a combined measure of system complexity distribution and relative complexity for ARS.

As a simple measure of subsystem complexity, we count the number of active elements in each subsystem. An active element is a moving mechanical part or a fundamental electronic component. Some special parts, such as a chassis and fixed mechanical parts with special purpose (e.g., a passive end- effector), are also viewed as active elements even if they are not moving mechanical parts.

For a robotic system (recall that all robotic systems are active by our definition) consisting of n subsystems, we define the degree of replication for the system, D_s , as

$$D_s = \frac{C_{min}}{C_{max}} \cdot \frac{C_{total}}{C_{ave}} \cdot \frac{1}{C_{ave}} \tag{2.3}$$

where

$$C_{ave} = \frac{1}{n}\sum_{i=1}^{n} C_i$$

$$C_{max} = maxC_1, ..., C_n$$

$$C_{min} = minC_1, ..., C_n$$

$$C_{total} = \sum_{i=1}^{n} C_i + \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} C_{ij}$$

C_i is the number of active elements in M_i
C_{ij} is the number of interconnections between the i-th and j-th subsystems when they are assembled. $C_{ij} = 0$ if the i-th and j-th subsystems are not directly connected by the assembly process.

In formula **??** the first term measures the complexity distribution throughout the subsystems, the second term measures the relative complexity of the total system to the individual subsystems, and the last term penalizes for complex subsystems. To design a robotic system capable of replication from low-level parts, the subsystem complexity should remain low relative to the total system complexity (i.e., high relative complexity).

In addition, we view a system consisting of a few complex parts as more trivial than one composed of many low-complexity parts. A higher value of D_s indicates a more truly replicating system in terms of its complexity distribution and relative complexity. Although the concept of D_s may not generalize to all robotic systems, it is useful as a measure to evaluate robotic replicating systems.

2.3.5 Implementations of kinematic self-replicating robotic systems

Low-complexity self-replication systems from specialized modules

The experimental system developed by Lee and Chirikjian demonstrates active self-replication using six simple electromechanical subsystems [LC07]. The system operates autonomously within a partially structured environment and constructs a functional replica of itself without external control or manipulation. This prototype serves as a proof-of-concept for scalable robotic self-replication using low-complexity parts and embedded environmental cues.

The system is a physical prototype of a self-replicating robot made up of six simple electromechanical subsystems (called M1 to M6) as it shown on the Figure 2.5.

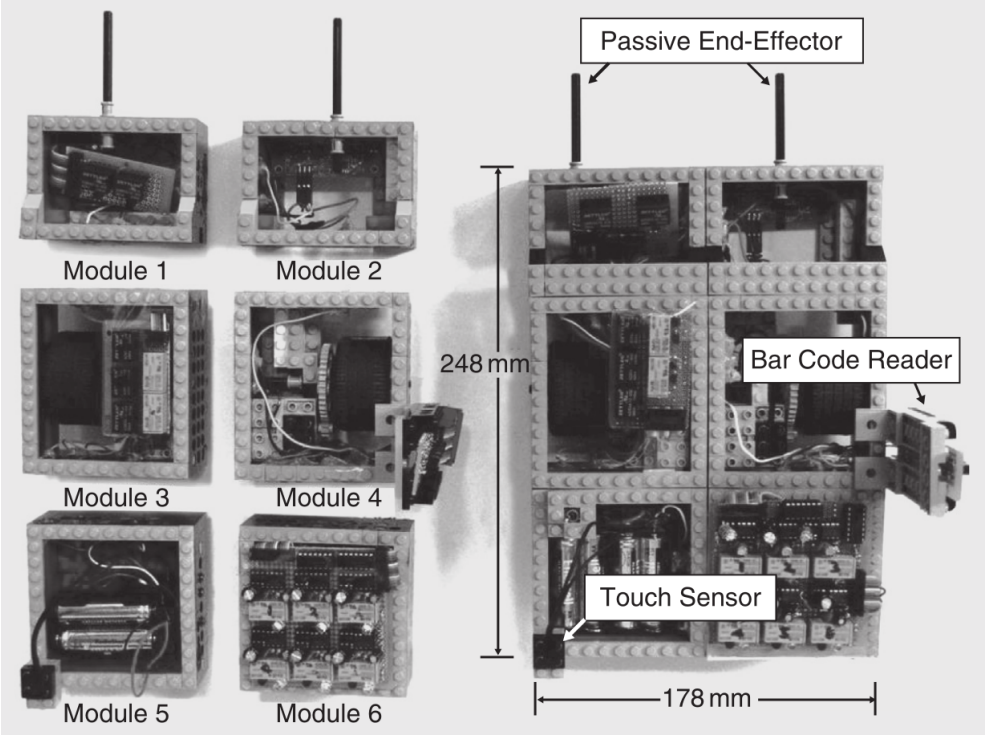

Figure 2.5: Modules (left) and complete robot (right) [LC07]

These modules, when assembled in a specific order, form a complete and functional mobile robot. The purpose of the experiment was to demonstrate that robotic self-replication is possible using relatively low-complexity parts in a partially structured environment—a space that aids replication without directly controlling it. Together, these modules allow the robot to: move along predefined paths, detect locations of components, decide what action to perform and carry and assemble parts.

The robot operates within a partially structured environment—one that contains helpful features, but does not actively participate in the replication. It includes: outer and inner tracks, crossways (branch paths), bar codes, contact codes, sensors, assembly station and wall, used to align the robot during reversing, ensuring precise placement.

The robot has six states, see Figure 2.7 and three distinctive behaviors for each state: forward line-tracking (line-tracking mode), reversing (reversing mode), and turning left while the timer is on (left-turning mode). The replication process begins with the parent robot navigating along a black track using its line-tracking sensors. As it moves, it detects barcodes placed near the modules it needs to collect; each barcode signals the robot to turn and approach a module. Using its passive end-effector, the robot captures the module and transitions to a new state based on contact sensors. It

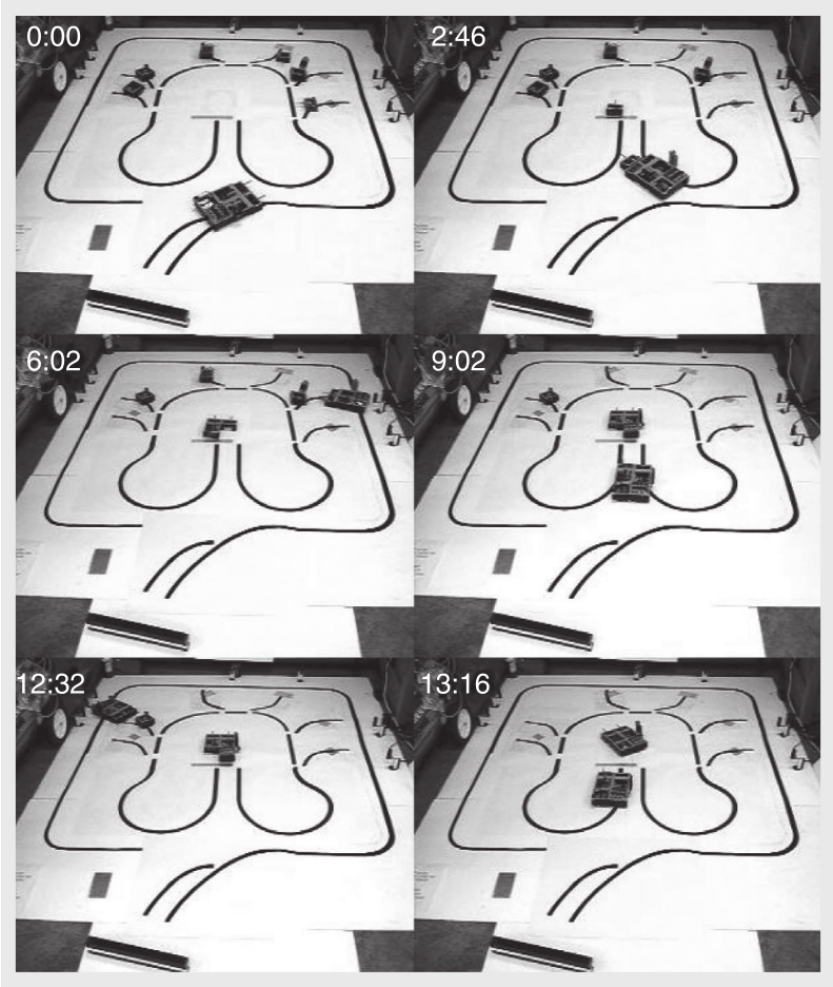

Figure 2.6: The process of self-replication [LC07]

then transports the collected module to a central assembly station marked by a metal line, which triggers the robot to reverse. The robot backs into position and places the module, using a wall as a physical guide for alignment. After delivering the module, it returns to the track and continues this process for the remaining modules. Each time, a new barcode guides it to the next module, and contact codes confirm successful transitions. The modules are assembled in a layered structure, with specific modules forming the base, middle, and top of the robot. Once all six modules are in place and properly connected, the electrical circuit is completed, and the new robot becomes fully functional. The entire process is autonomous and guided solely by the robot's internal logic and environmental cues successfully constructing a working copy of itself without any external control or assistance.

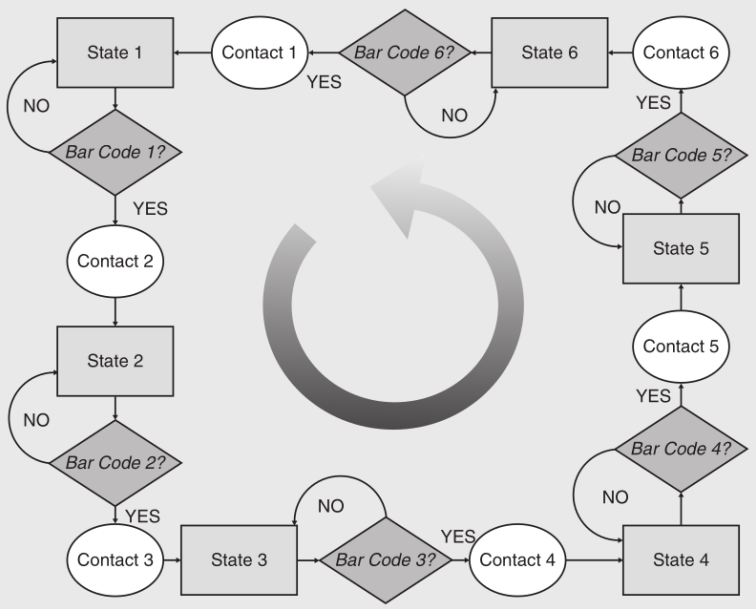

Figure 2.7: State diagram shows behaviors of the robot according to the sensor inputs [LC07].

Using formalas **??** and **??** the system can be represented by

$$R = M_1 + ... + M_6$$

$$M = M_1 + ... + M_6$$

$$E = \{\text{tracks, bar codes, contact codes, wall, metal line}\}$$

, the system becomes

$$R' = \{(M_1 + ... + M_6),\ (M_1 + ... + M_6)\}$$

$$M' = \emptyset$$

$$E' = E.$$

Kinematic self-replication from unified modules

Different authors proposed kinematic or geometrical self-replication based on unified modules. Zykov et al. in 2005 demonstrates physical self-replication using autonomous modular robots [Zyk+05].

These machines are built from identical robotic cubes—each a 10 cm module—that can physically attach, detach, and reconfigure to construct replicas of themselves.

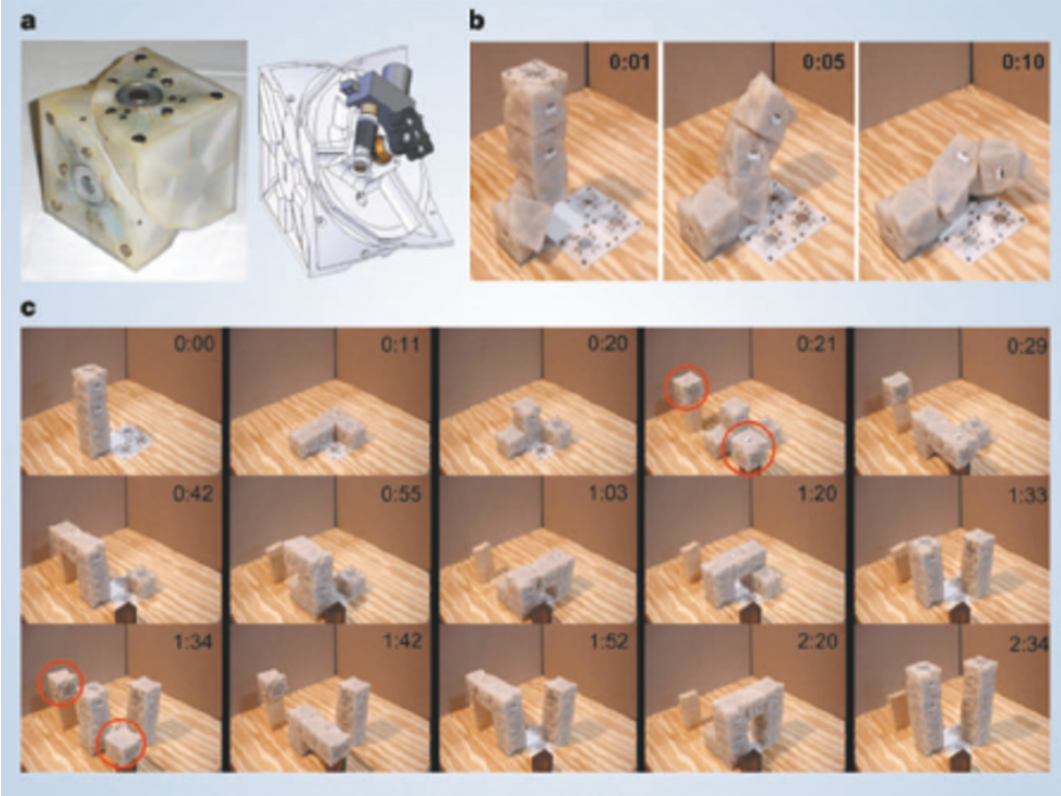

Figure 2.8: Self-replication of a four-module robot [Zyk+05]

Each cube module is split along a diagonal plane and contains internal mechanisms that allow one half to rotate relative to the other, see Figure 2.8a. This ability enables modules to cycle between different configurations by swiveling in few seconds (see Figure 2.8b), allowing connected modules to break apart and reattach in new positions. The modules use electromagnets for selective connection and disconnection, and they receive power and communication signals through conductive face-to-face contacts and a powered baseplate.

The replication process begins with a small robot, composed of four such modules, positioned near two "feeding stations" that contain spare cubes. These cubes are manually replenished, but the entire replication procedure is autonomous. Using their internal control logic and distributed microcontrollers, the robot's modules coordinate motion to pick up new cubes, manipulate them into position, and assemble them into a new, functional robot of the same structure. In one experiment, a four-module robot completed replication in approximately 2.5 minutes (see Figure 2.8), while a simpler three-module robot managed the task in just over one minute. This experiment demonstrates that even simple robotic parts can support mechanical self-replication

so that the resulting system can itself repeat the replication process.

Material-robot systems

The material-robotic system proposed by Jenett et al. in [Jen+19] and further developed by Abdel-Rahman et al. in [Abd+22] consist of passive structural lattice voxels which form a substrate for the locomotion of purpose-built inchworm modular robots build from active voxels, and capable of rearranging and placing additional voxels, see Figure 2.9.

The combination of voxels and robots forms a system, enabling precise assembly of large structures with simple robots through localized registration to the underlying lattice, and also allows of creation of modular robotic toolkit that utilizes active lattice voxels as the primary structural building blocks.

The active voxels which form the basis of the structural-robotic system are a cuboctahedral (Cuboct) unit cell which offers superior structural properties such as high stiffness at low density with geometric characteristics suited for robotic assembly and fabrication. Specifically the flat Cuboct faces enable fully- constrained connections between single-voxel pairs with four points of contact, as well as a useful decomposition for voxel fabrication.

(a) Cell **(b)** Modular robot

Figure 2.9: Material-robot system [Abd+22]

Combining these active voxels with actuators, control, and power enables unique behaviors including robotic self-replication and hierarchical robot assembly. A single

robot, tasked with constructing a large structure, can create a heterogeneous swarm of constructors tailored to maximize material throughput for a given target geometry.

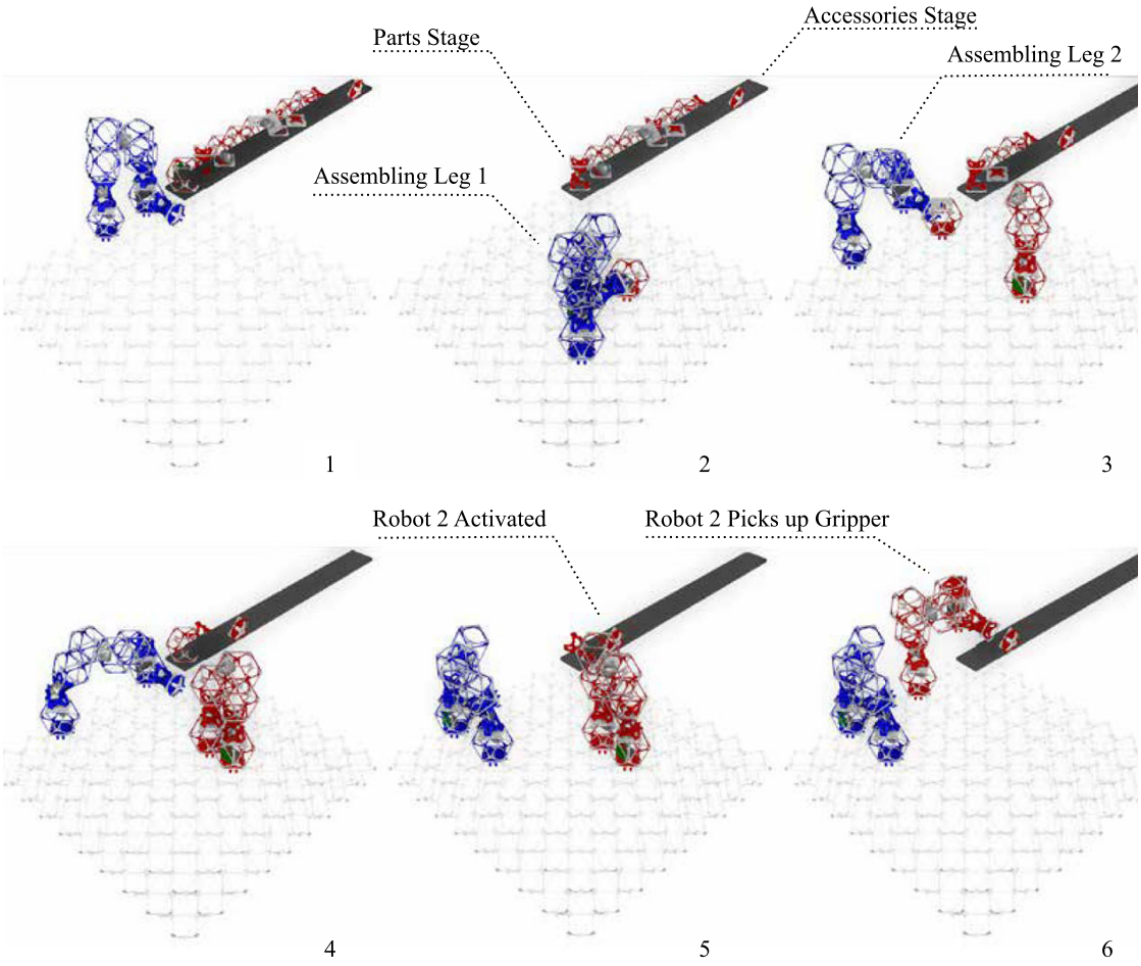

Figure 2.10: Self-replication process [Abd+22]

One of possible variations of assembler robots, designed to carry voxel cubes, and a process of self-replication from a set of modules is shown in Figure 2.10.

To perform self-replication, an initial "parent" robot uses its manipulators to pick and place voxels from a nearby pickup station. The parent robot builds a copy of itself—termed a "child" robot—by assembling voxels in a configuration that mirrors its own structure. The replication process is limited to voxels and certain key functional modules, including control units, power supplies, and actuated joints (called elbows). Components like wrists and grippers are added afterward through a procedure called "accessorizing," where the robot attaches these parts to the pre-assembled structure.

A critical constraint in the replication process is that the newly built robot must be free to move upon completion. Therefore, construction cannot proceed directly on the substrate base; instead, it extends out from an anchored platform, ensuring that once finished, the child robot can undock and function independently. The replication logic is programmed into a centralized control system, which assigns tasks, manages timing, and ensures robots do not collide. The control system guides each robot step-by-step in constructing a new robot, allowing for recursive reproduction over multiple generations.

This system's significance lies in its ability to scale construction tasks by increasing the number of robots through self-replication, rather than requiring the deployment of additional robots externally. As more robots are built, the workload is distributed, theoretically allowing exponential speed-up in large-scale assembly projects. The practical implementation uses inchworm-style locomotion, where robots move stepwise over the voxel lattice using coordinated grippers and joints, making them lightweight, simple, and adaptable to modular construction environments.

While this model is a proof-of-concept, it successfully demonstrates robotic self-replication using standardized parts and relatively low-complexity hardware. The modularity and discrete assembly approach also enhance the system's potential for error detection, robustness, and reconfiguration. Although scalability is limited by actuator strength and coordination complexity, the framework opens new avenues for autonomous robotic manufacturing systems that can grow their own workforce from a finite resource pool.

3 Research questions

This paper will address the following research questions:

1. How can automated model checking and other formal-verification techniques be used to verify different properties of self-replicating robotic systems, and what adaptations can be done for practical implementation?
2. What systematic methodology can translate the behaviors and structural constraints of self-replicating robots into expressive temporal-logic models and specifications?
3. What limitations – computational, semantic, or modelling-related – does model checking face in domain of SRRS, and how might these limitations can be mitigated?

3.1 Analysis of model checking approaches in self-replicating systems domain

The first research question involves a comprehensive survey and critical evaluation of current state-of-the-art model checking and formal verification tools, such as SPIN, NuSMV, and PRISM, focusing on their applicability to modeling and verifying the unique dynamics of self-replicating robotic systems (SRRS).

This investigation includes a review of relevant literature and existing systems in self-replicating robotics and cellular automata to identify key operational characteristics and formalizable behaviors. It also involves identifying core properties relevant to SRRS, such as liveness (ensuring successful replication within time or resource constraints), safety (avoiding resource conflicts or hazardous outcomes), as well as broader system-level guarantees like termination, deadlock-freedom, and fault-tolerance. The study will further explore how these properties can be expressed in various formal languages, including Linear Temporal Logic (LTL), Computation Tree Logic (CTL), and

probabilistic extensions. This will require examining what adaptations or abstractions are needed to make these languages suitable for SRRS modeling.

Iterative experimentation with selected tools will help assess the feasibility of encoding and verifying these properties and may lead to the discovery of more complex or emergent behaviors to integrate in later stages.

3.2 Experimental checking

The second research question aims to define a systematic formal modeling and verification workflow tailored to SRRS. This involves developing a formal representation of self-replication processes, structural rules, environmental interactions, and resource usage.

It also includes establishing a methodology to map real-world or simulated robotic behaviors - such as perception, manipulation, and actuation- onto abstract states and transitions that are amenable to formal analysis. Based on this framework, appropriate model-checking tools and specification languages will be selected and justified through experimental evaluation within simulation environments.

A prototype or proof-of-concept implementation will be developed using a simplified SRRS scenario, such modular robotic replication, to demonstrate how this methodology captures essential properties and enables formal verification.

The ultimate goal is to bridge the gap between low-level robotic simulation and control systems and high-level formal models, ensuring that the designed behaviors are both physically feasible and logically justified.

3.3 Limitations and challenges

The third research question addresses the fundamental limitations that current formal methods and model-checking tools face when applied to the complex domain of self-replicating robotic systems. It includes analyzing computational constraints such as state-space explosion, undecidability in dynamic conditions, and potential inefficiencies in scaling existing verification tools.

The study will also examine semantic mismatches between physical or simulated robot behavior and abstract models, particularly in representing kinematic and dynamic aspects, sensor errors, and asynchronous or partially observable environments.

Additionally, the research will explore modeling limitations related to the representation of dynamic structure generation and adaptive behaviors.

To address these challenges, various mitigation strategies will be proposed, including modular verification, changes in system abstractions and hybrid techniques that combine formal models with simulation-based validation.

The outcome of this part may include recommendations for extending the selected verification toolsets, adapting implementation strategies, or designing new experimental setups specifically for the analysis of SRRS.

4 Methodology

4.1 Research tools

Self-replication research in robotics involves complex interactions between assembly processes, modular design, environmental constraints, and control systems. Robotic simulation environments offer a valuable sandbox to explore these dynamics before physical implementation. The ideal simulation platform must support modularity, kinematics, sensing, and interaction with discrete components.

A set of tools for simulation and control of robotic systems including SRRS can be seen in the Table 4.1

In order to simulate kinematic self-replicating systems and to be able to implement control functions that are also applicable to a physical system, the Gazebo simulator with ROS infrastructure seems to be an optimal choice.

Gazebo supports multiple physics engines, such as ODE, Bullet, and DART, allowing to model the dynamics of contact, friction, collisions, and gravity with a high degree of accuracy.

Modular robot design can be modeled through ROS's URDF (Unified Robot Description Format) and SDF (Simulation Description Format). These formats help to define robotic structures as combinations of interconnected parts and joints, closely mimicking the principles of modular self-replicating systems. Components can be described independently and reused across different robots or configurations. It support simulating a wide array of sensors, including cameras, LiDAR, depth sensors, IMUs, proximity sensors, and contact detectors.

Structured or unstructured environments for SRRS can be modeled in great detail, placing objects with specific physical properties, and simulating real-world challenges like terrain irregularities or sensor noise.

ROS provides a powerful middleware layer that manages communication between different components of the robot and the environment. Through ROS, we can implement and test high-level control algorithms, including perception pipelines, decision-making logic, and task planning. It supports distributed systems, so that multi-robot

Simulator	Physics fidelity	Modularity support	Custom control	Use case for SRRS
Gazebo+ ROS	High	Strong via URDF/SDF	ROS integration, plugins, Python/C++ nodes	Physical replication with mobile robots and manipulators
CoppeliaSim (V-REP)	High	Native modular modeling	Lua, Python /remote APIs	Detailed control of part interaction and modular assembly
PyBullet	Medium to High	Good (via URDF /SDF)	Python API (fast iteration)	Cube/voxel-based systems and contact-rich interactions
Webots	Medium	Moderate	C/C++ /Python controllers	Lightweight simulation of small-scale replication setups
Unity+ ML-Agents	Medium	High (custom prefabs)	C#,Python with ML-agents	Learning-based or perception-driven replication strategies
Isaac Sim (NVIDIA)	High	Strong (via Omniverse)	Python scripting, ROS2	High-fidelity simulation of complex autonomous robots
Golly NetLogo CA tools	None	Tile-based, symbolic	Rule-based/ scripting	Theoretical studies (e.g. von Neumann or Langton automata)

Table 4.1: Comparison of simulation environments for self-replicating robotic systems

scenarios (such as cooperative replication) can be easily simulated. ROS also provides tools for recording and analyzing data, tuning controllers, and implementing feedback loops.

Gazebo and ROS are open-source and highly extensible. Developers can create plugins for new sensors, actuators, or control strategies. This is particularly useful in self-replication research, where novel components or interfaces may need to be modeled. Additionally, the close integration between simulation and real-world systems means that code developed in Gazebo with ROS can often be deployed directly to real hardware with minimal changes.

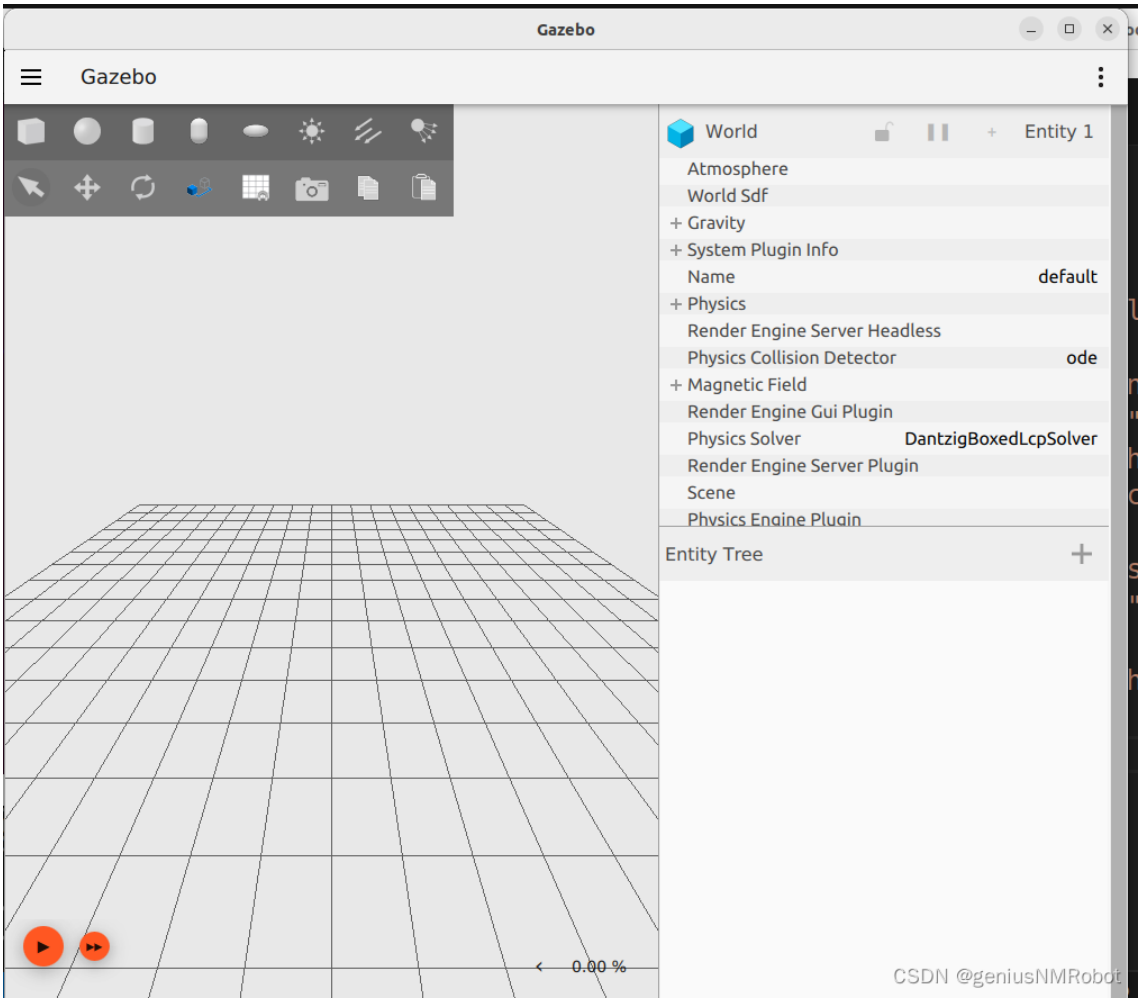

Figure 4.1: Gazebo Harmonic simulation interface

4.2 Proposed approach

Gazebo in combination with ROS can be used to create an interface between simulated robot behavior with formal verification tools, such as NuSMV. Formal methods allow researchers to rigorously verify properties of robotic systems—such as correctness, safety, liveness, and reachability —by constructing abstract models of their behaviors and validating them against logical specifications.

ROS-based systems define robotic structure and dynamics using URDF (Unified Robot Description Format) files and execute behavior through Python or C++ ROS nodes. These two sources—mechanical definitions and control logic—can be systematically translated into finite-state abstractions suitable for model checking with tools like

NuSMV.

The process typically involves analyzing the state transitions implied by the ROS node logic, especially within action sequences (e.g., part pickup, alignment , assembly), and discretizing them into states, inputs, and transitions. Each robot behavior or controller mode (e.g., "move", "grasp", "assemble") is represented as a finite state, while events and sensor conditions define transition rules. The mechanical configuration from URDF can define static properties or initial states in the system.

From this, a states diagram structure can be extracted, encoded in NuSMV's input language, and checked against temporal logic properties, described in CTL or LTL formulas. For instance, one could verify that a self-replicating robot always eventually reaches the assembly state (liveness) or that it never attempts to pick a module unless one is detected (safety).

Automating this translation—using Python scripts, that parse URDF and interpret ROS node behavior—is increasingly feasible, especially for systems designed with modular and rule-based architectures. Scripts or intermediate frameworks can analyze ROS bag files or logs to capture sequences of operations, and encode these as state transitions.

This capability enables hybrid verification workflows, where simulation is used to test behavior under dynamic conditions, and formal verification is applied to prove correctness across all possible system executions. In the context of self-replication, this allows verification of critical properties like "a complete replica is only activated after all modules are assembled" or "robots do not collide while cooperating on part assembly".

By linking Gazebo/ROS to NuSMV, can be created a powerful toolchain to not only test and observe behavior in simulation but also prove correctness and robustness in all logical conditions—an essential feature for deploying self -replicating systems in safety-critical or remote environments.

5 Implementation of model checking in SRRS

5.1 Subject description

In this research we will consider a simplified model of a relative modular robot and its corresponding environment, modeled with the potential for self-replication and based on the design, and material-robot principle, implemented by Jenett et al. [Jen+19] and Abdel-Rahman et al. [Abd+22].

In our model, as in the original work, the robot, shown on Figure 2.9b) operates by manipulating and connecting parts, shown on Figure 2.9a) that are of the same type as its own building blocks. These parts serve as material for assembling different structures and may be distributed in the environment either in ordered arrangement or arbitrary manner. Despite their functional diversity, all blocks and parts are geometrically identical and take the form of cuboctahedral voxels, a general view of it is represented in Figure 5.1.

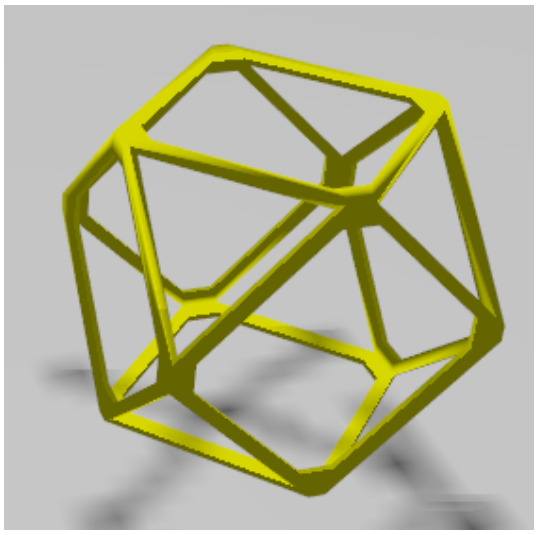

Figure 5.1: Material part or single robot block geometry in Gazebo simulation environment

Further in text, we refer to elements attached to the robot as "blocks" and to the same entities when distributed in the environment and not connected to the robot as "parts."

The robot is composed of a sequence of blocks of different functional types, an example of a robot is shown in Figure 5.2, where different types of blocks are marked with different colors.

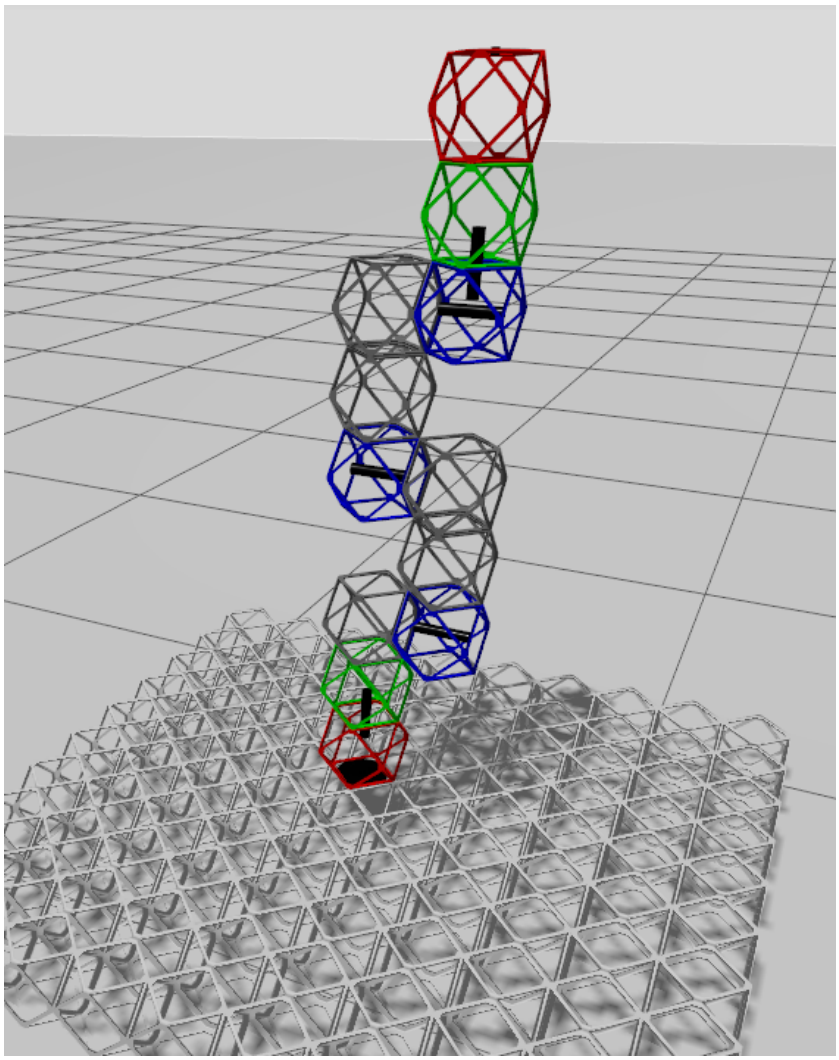

Figure 5.2: Robot structure in Gazebo simulation environment

The first and the last blocks of the sequence serve as end-effectors equipped with grippers, enabling the robot to interact with its environment. The intermediate blocks

constitute a rigid structure or act as joints, which may provide rotations around different axis, for example, enabling changing in yaw or pitch angles.

This uniform geometry facilitates a modular construction paradigm, where the same physical units can be rearranged to produce different structures and robot morphologies.

At any given time, robot in our model is capable of handling a single part, aligning it with the existing discrete environment and attaching it to some structure or new robot. Through repeated execution of this process, the robot can in principle assemble a replica of itself, thereby demonstrating the fundamental process of self-replication.

The robot operates within a structured environment defined as a lattice field that is geometrically compatible with the cuboctahedral voxel design this field represented as while base structure underneath the robot on a Figure 5.2. This lattice constrains both the robot and the individual parts to occupy discrete positions, ensuring that all spatial relations, and connections can be described in terms of transitions between well-defined integer coordinates. Such discretization not only simplifies the modeling of robot kinematics and assembly operations but also provides a natural basis for formal verification, since the system state space becomes finite and directly available for exploration by model-checking techniques.

In this chapter, we focus on methods to formally analyze the behavior of such systems. Model checking provides a rigorous framework for verifying whether the system satisfies specific properties of interest. In particular, we investigate the use of symbolic model checker - NuSMV, to evaluate both safety and liveness conditions for the modular robot using linear temporal logic (LTL) and brunching temporal logic (CTL). Safety properties ensure that undesirable states, such as collisions or inconsistent connections, are never reached. Liveness properties, on the other hand, guarantee that the system is always capable of making progress, for example, that a new block can eventually be attached to a target or that a new robot can continue self-replication process.

Beyond simple verification, we also explore the potential of using NuSMV counterexample generation as a decision-making strategy. Counterexamples provided by the model checker correspond to concrete sequences of state transitions that lead to a violation of a previously negated property. Interpreted constructively, such sequences can serve as action plans or decision sequences for the robot, indicating feasible operations that allow it to achieve its objectives. This approach establishes a bridge between formal verification and automated planning and can increase the reliability and effectiveness of the system, and also add flexibility to decision-making in complex and dynamic conditions.

5.2 Analysis of the system

According to the classification given in Chapter 2.3.2 described self-replicating system for the general configuration of a robot can be classified as active system with hierarchical type of replication and capability for direct and indirect replications.

It has hierarchical replication type because in general it can produce robots capable of self-replication as well as robots or mechanical structures lacking of such capability.

It is active since it is directly engaged in constructing child robots.

The system can be homogeneous or heterogeneous based on initial conditions and programmed behavior. Homogeneous type of SRRS consist of robotically identical units, whereas heterogeneous enables generation of multiple robot types, with potential for specialization. Therefore, our system will be homogeneous if we will use only one type of robot with predefined structure that will replicate itself and heterogeneous if there will be multiple types on start or during replication.

In this chapter we will assume the system is homogeneous but will analyze effect of different types or configuration of robots in different environments on the properties of a system.

Using model proposed by Lee and Chirikjian in [LC07] and described in Chapter 2.3.3 self-replicating process in our system can be formally represented as equation 5.1.

$$(R', M', E) = \Phi(R, M, E, t) \tag{5.1}$$

Where:
M – multiset (set with repetitions) of available parts (material) located in the environment to be used for building replicas robots by an initial system.
R – a multiset of initial robots to be reproduced. In our case one robot with predefined configuration. Each element of R includes specific robot structure in form of sequence of blocks, geometry, kinematic limitations and robot control system.
E – is a multiset of environmental structures and/or elements involved in the replication process (but not replicated). In our experiment is represented by base and optional obstacles.
R', M' – the new robots (or build structures) and the remaining and altered material respectively.
Φ – the replication function of time t.

This model is a special case of a Formula 2.2 for completely structured environment. $E = E'$ since each element of the environment (base) is either fixed at a precise location,

or described in integer coordinates and there will be no modification or change in any structures during the self-replication process.

This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and robots.

A central process is an act of self-replication – that is an assembly of a replica of itself by the initial robot from a set of parts located on a base. This process shown in state chart on Figure 5.3.

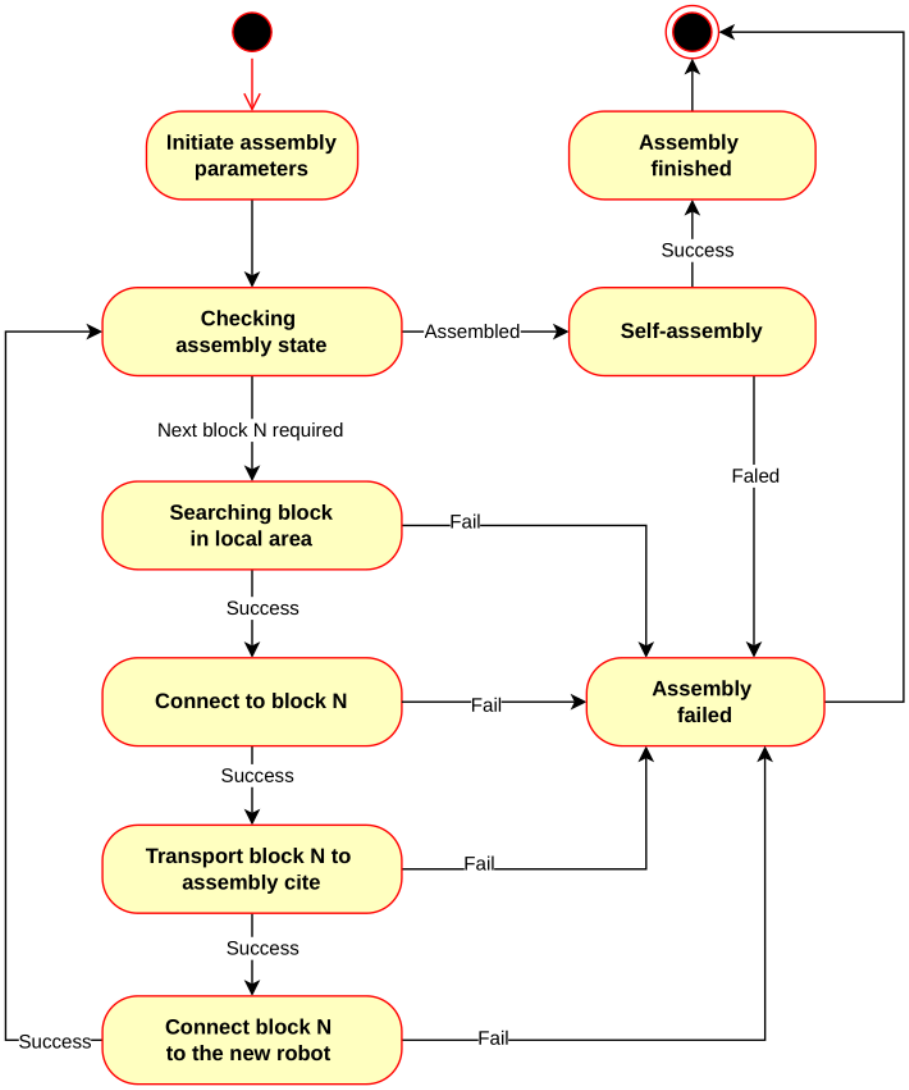

Figure 5.3: State chart of self-replication assembly process

Such process can be described as transition systems and is therefore suitable for model checking. It can be checked for specific requirements, written in form of temporal logic.

In this self-replication process we can derive following actions:

1. Connection and disconnection of the end-effector to and from the block or base.
2. Localization of the part, or receiving of part coordinates from the sensors. This action is a part of "Checking assembly state" and "Searching block in local area" in state chart.
3. Movement of the end-effector from one point to another, with or without connected parts.

Implementation of connection and disconnection actions is platform dependent be represented in a control system as boolean state.

Implementation of parts localization actions also depends on specific sensors configuration and similarly could be a represented as a boolean array. Movement action depends mainly on the robot blocks configuration, blocks parameters, environmental limitations, initial and target points.

Model checking of generalized movement, in contrast, has a potential to substantially improve the overall self-replication process and adapt it for different environment conditions.

In the following sections we will model robot, translate self-replication process in transition system and check following properties using NuSMV:

- Check local reachability of some point or a set of points (region) by the robot end-effector.
- Generate a command sequence for a robot to move the end-effector to target point, based on counterexamples.
- Research global reachability of some point or region by robot end-effector using the walking form of movements.
- Generate a path to a point using walking based on counterexamples.
- Analyze set of robot configurations – check existing configuration or find relevant configurations of blocks based on counterexamples.

5.3 Model checking based control in local space

5.3.1 Overview

A basic verification step for our model consists in checking whether a given point in space is reachable by the robot's end-effector when the blocks are arranged in a specific initial configuration. Establishing such reachability opens the way for identifying potential stages of the assembly process, as illustrated in Figure 5.4.

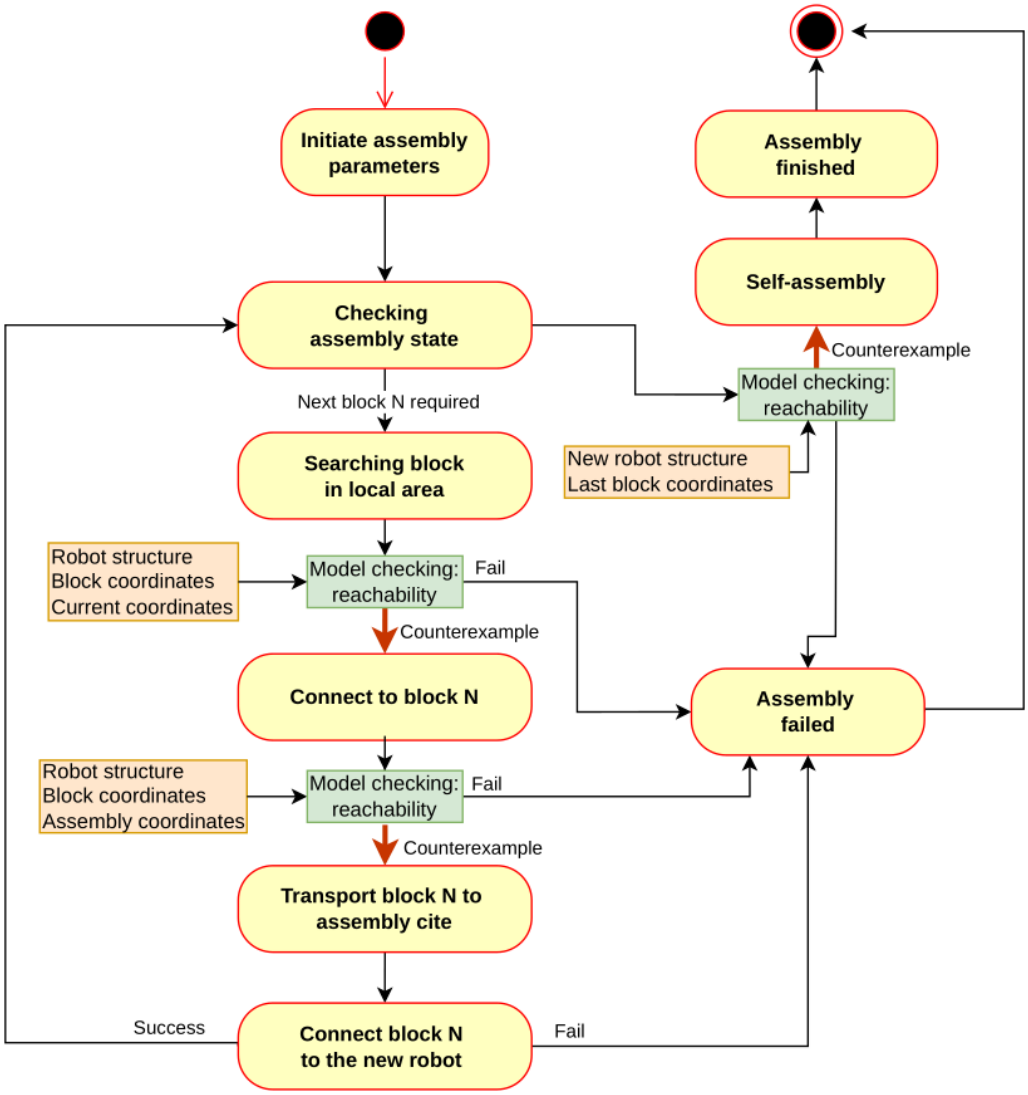

Figure 5.4: Self-replication process with potential model checking stages

In this context, model checking can be employed to confirm that the robot is capable of executing fundamental operations required for self-replication:

- moving the end-effector from its current location to the position of a target part,
- transporting the part to the designated assembly site,
- moving the newly attached end-effector to connect with the next required block.

In this chapter, we present a NuSMV model that encodes a chosen robot configuration, together with temporal logic specifications for verifying point and region reachability. Furthermore, we introduce Python-based tooling that derives counterexamples from the model checker and interprets them as executable control strategies for guiding the robot through the assembly process.

5.3.2 Robot NuSMV model

The robot structure as it shown in Figure 5.2 is represented as a modular composition of three distinct block types: a fixed base block, blocks with a yaw degree of freedom, and blocks with a pitch degree of freedom. Figure 5.5 illustrates the hierarchical arrangement of these blocks and the flow of coordinate and orientation data between them. This modular abstraction allows the robot structure to be described in a uniform and scalable manner, independent of the number of blocks.

As the lowest level of a system, as it shown in Figure 5.5 we define a block. Each subsequent block introduces a transformation of the block end position according to its type. Each block has a certain type, in this work i consider three types of blocks:
The $block\_base$ defines the fixed starting point of the robot. It receives as input the global origin coordinates (x_orig, y_orig, z_orig) and the initial orientation parameters $(yaw\_orig, pitch\_orig)$ Its role is to provide a reference frame for all subsequent blocks. The base block outputs the coordinates (x_end, y_end, z_end) and orientation values that are propagated forward to the next block in the sequence.

Block types block_yaw realise rotation around local axis Z, that oriented along the parent block The $block\_yaw$ represents a joint capable of rotating around the vertical axis. Internally, the block maintains a discrete variable ϕ, which denotes the yaw angle.

As illustrated in the Figure 5.6, ϕ can evolve in steps of +1 or -1, modulo 36, corresponding to 10-degree discretization over the full circle of 360°. Thus, the block admits cyclic transitions that allow continuous rotation in both clockwise and counterclockwise directions. The block takes as input the parent coordinates (x_par, y_par, z_par) and

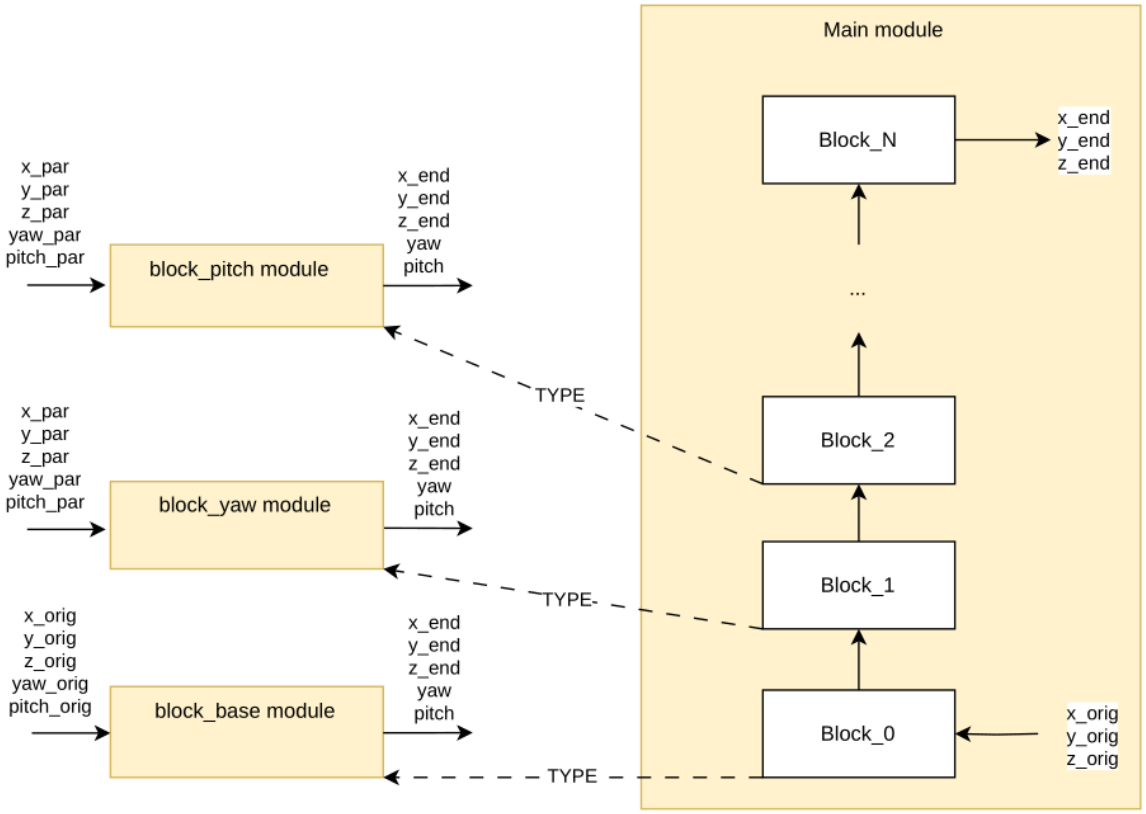

Figure 5.5: Robot configuration NuSMV model

orientation $(yaw\_par, pitch\_par)$, applies the yaw rotation, and produces updated endpoint coordinates.

$$\begin{cases} \phi \to \phi \\ \phi \to (\phi + 1)\ mod\ 36 \\ \phi \to (\phi - 1)\ mod\ 36 \end{cases} \tag{5.2}$$

Block types block_pitch realise rotation around local axis Y, that is perpendicular to Z often the parent block. Similarly, the $block\_pitch$ encodes a joint that rotates around the lateral axis of the preceding link. Its discrete state variable θ:

$$\begin{cases} \theta \to \theta \\ \theta \to (\theta + 1)\ mod\ 36 \\ \theta \to (\theta - 1)\ mod\ 36 \end{cases} \tag{5.3}$$

In combination, these transitions capture the continuous ability to rotate up or down by discrete steps.

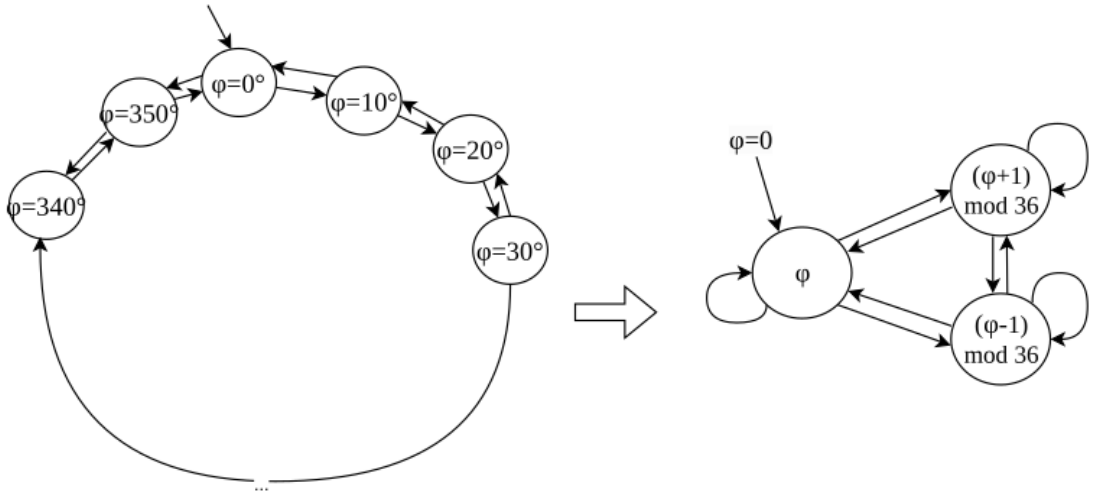

Figure 5.6: Robot block transition system for angle ϕ

At the system level, the robot is built as a chain of such blocks. After the base block, one may attach any sequence of yaw and pitch blocks in arbitrary order, depending on the desired manipulator structure. The right-hand side of Figure 5.5 shows an example with N chained blocks, where Block 0 connects directly to the base and Block N defines the robot's end-effector. The outputs of the last block are new coordinates (x_end, y_end, z_end) and updated orientation values. Each block is parameterized by the output of its predecessor, creating a cascading dependency structure.

Thus, on the NuSMV model description level we have:

- main block module - that describes a robot structure as a sequence of block types as variables in VAR section.
- block_base module - describing initial block of a sequence and locating on a ground plane.
- block_yaw module - describing rotation around the parent Z axis (along the block) and changing the yaw angle, the NuSMV code for this module is shown in Listing 5.1.
- block_pitch module - describing rotation around the parent Y axis (perpendicular to the block) and changing the pitch angle.

```
1 MODULE block_yaw(px, py, pz,
2                   xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
3                   L, initYaw)
4 VAR
5   yaw : 0..35;
```

```
  cy : cos(yaw);
  sy : sin(yaw);

DEFINE
  SCALE := 100;

  cos_yaw := cy.val;    -- [-100..100]
  sin_yaw := sy.val;

  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
  xhz2 := xhz;

  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;


ASSIGN
  init(yaw) := initYaw;
  next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

Listing 5.1: NuSMV code for block_yaw module

MuSMV modules block_base, block_pitch sin, and cos NuSMV modules shown in Appendix A. This design supports a natural inductive description of the robot in NuSMV: the same block type can be instantiated repeatedly, with only the input-output connections altered.

The modularity of the design has two important advantages.
First, it allows scalability, since additional joints can be incorporated simply by adding new instances of yaw or pitch modules.
Second, it supports module based model checking, because each block is described as a transition system with a finite number of states and transitions. The global state space of the robot is then obtained by synchronous product of the component transition systems, which can be directly analyzed using LTL or CTL specifications in NuSMV.

The dynamics of each yaw or pitch block can be represented as a finite-state transition system. Figure 5.6 illustrates this equivalence explicitly. On the left-hand side, the system is shown as a ring of discrete states corresponding to the possible angle values

$$\phi = 0^\circ, 10^\circ, \ldots, 350^\circ$$

From each state, there are transitions to the neighboring states at $\phi + 10^\circ$ and $\phi - 10^\circ$ thus modeling incremental rotations in either direction. Since the variable is defined

```
ASSIGN
    ...
    init(yaw)    := 0;
    next(yaw)    := {
                     yaw,
                     (yaw+1) mod 36,
                     (yaw+35) mod 36
                     };
```

Listing 5.2: Assign part of code for block transitions

modulo 36, the state $\phi = 350^\circ$ transitions back to $\phi = 0^\circ$, completing the cycle. The result is a finite cyclic transition system with 36 nodes, which represents the discretized continuous rotation of the block.

On the right-hand side of Figure 5.6, this same behavior is captured more compactly. Here, the internal state variable ϕ is updated according to transition rules:

$$\phi = \phi \quad \vee \quad \phi = (\phi + 1)\, mod\ 36 \ \vee \phi = (\phi - 1)\, mod\ 36$$

This concise representation preserves the semantics of the explicit ring but makes the module description more compact and scalable. When combined with coordinate update formulas, it fully captures the contribution of the block to the robot's kinematics.

This transitions and angle changing for each block reflected by the following part of assign block, shown in Listing 5.2.

For each block in a sequence of robot structure the initial value is assigned, and next state can be chosen nondeterministically from three possible transitions.

The full NuSMV model for modules: block_base, block_yaw, block_pitch and main presented in Appendix A.

5.3.3 Reachability specifications

Point reachability checking specifications:

In order to verify whether the robot is capable of reaching a desired target point with its free end-effector, in three-dimensional space, we formalize the reachability check directly within the NuSMV model.

In the DEFINE section we provide target point, end-effector coordinates variable and define error threeshold, this code fragment is shown in Listing 5.3.

```
DEFINE
...
error  := 1;
checkX := 10;
checkY := 15;
checkZ := 25;
endX_inside_limits :=(endX <=(checkX+error) & endX >=(checkX-error));
endY_inside_limits :=(endY <=(checkY+error) & endY >=(checkY-error));
endZ_inside_limits :=(endZ <=(checkZ+error) & endZ >=(checkZ-error));
```

Listing 5.3: End effector and target point coincidence condition

Here, the variables endX, endY, and endZ represent the Cartesian coordinates of the robot end-effector, which are computed elsewhere in the model based on the joint angles and link lengths.

The parameters checkX, checkY, and checkZ denote the coordinates of a target point that we wish to test for reachability.

Since we can use only countable number of states and have to represent angles as integers, each transformation of angles to linear coordinates is not precise. Correspondingly the distance between actual and target points is between zero and linear delta calculated based on descretization value for angle. For each spatial dimension, Boolean formulas $endX\_inside\_limits$, $endY\_inside\_limits$, and $endZ\_inside\_limits$ check whether the end-effector coordinate lies within a small interval around the corresponding target value, effectively defining a cube of side length 2, centered at the target.

The formal property is expressed in Linear Temoporal Logic (LTL) in Listing 5.4. states that on every execution path, it will eventually (F) be the case that the robot's end-effector is within the specified tolerance region around the target point. In other words, the property enforces that the end-effector eventually can reach the target zone.

```
    LTLSPEC
    F (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.4: Reachability condition in LTL specification

The formal property is expressed in Computation Tree Logic (CTL) in Listing 5.23.

```
CTLSPEC
EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.5: Reachability condition in CTL specification

In this specification *EF* means "there exists a path along which eventually holds following condition".

Region reachability checking specifications:

We can check reachability of some 3D region, in our case a box by defining the variables checkX, checkY and checkZ as FROZENVAR. FROZENVAR section in NuSMV declares parameters that are chosen nondeterministically at starting time and then never change along a run for an entire execution (trace). The Listing 5.6 represents an example of a region:
$x = [-10, 10], y = [-10, 10], z = [0, 20]$.

```
1 FROZENVAR
2     checkX : -10..10;
3     checkY : -10..10;
4     checkZ : 0..20;
```

Listing 5.6: Region definition for a reachability checking

Changing in the specification for the region checking using CTL logic shown in Listing 5.23 is not necessary. With the frozen variable region, the formula states: from every initial state (for every admissible target chosen at t=0), there exists a sequence of moves that eventually places the end-effector inside the region.

Because NuSMV checks CTL properties over all initial states , this specification proves reachability for every target in the declared ranges (universal over targets, existential over paths).

These specifications therefore provides a rigorous, formally verifiable mechanism to test spatial reachability of robotic manipulators in discrete state-space models. It can be generalized by altering the checkX, checkY, and checkZ parameters, and by varying the tolerance error, to systematically analyze the reachable workspace of the system.

5.3.4 Counterexample processing

Based on specifications described in previous section we use the modeling pattern in NuSMV for reachability: “eventually reach point P”, and negate that requirement to force the model checker to produce a counterexample that reaches target point P.

In our self-replication robotic system this pattern can be implementated to obtain plausible traces on following stages of the self-replication process:

1. To define the trajectory from current point to new part.

2. To define the trajectory of transportation of the part to an assembly point.
3. To define the trajectory for new robot to connect to the last part.

These stages shown on the Figure 5.7, where input parameters shown in orange squares and counterexamples depicted with red arrow.

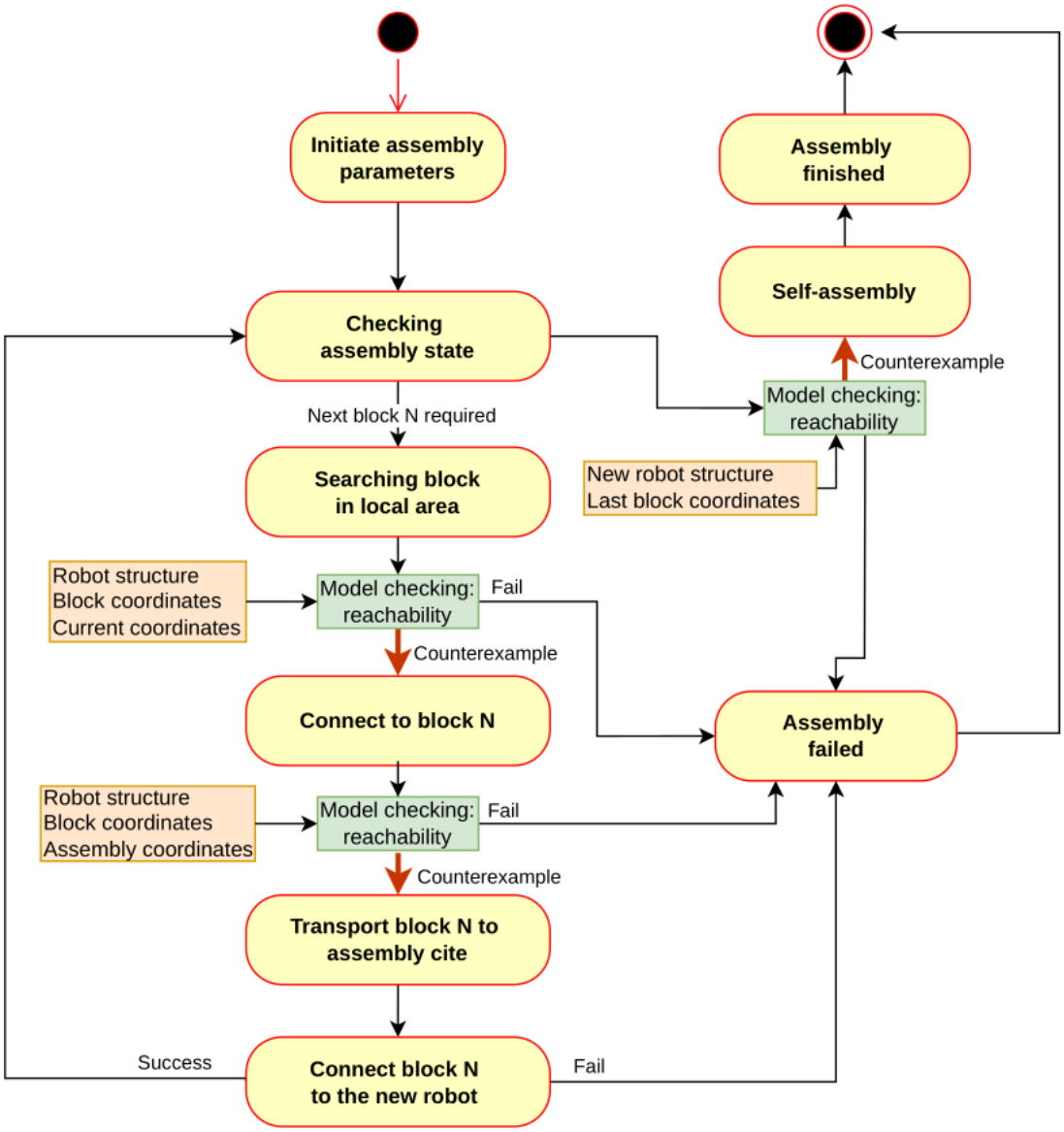

Figure 5.7: Local assembly process without model checking

The formal property is expressed in Linear Temoporal Logic (LTL) in Listing 5.7. states

that on every execution path, it will always (G) be the case that the robot's end-effector is not within the specified tolerance region around the target point. In other words, the property enforces that the end-effector infinitely often leaves the target zone, meaning the robot cannot remain in that region permanently.

```
1 LTLSPEC
2 G !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.7: Reachability condition in LTL specification

The formal property is expressed in Computation Tree Logic (CTL) in Listing 5.8.

```
1 CTLSPEC
2 AG !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.8: Reachability condition in CTL specification

The specification:

$$AG\neg(endX\_inside\_limits \wedge endY\_inside\_limits \wedge endZ\_inside\_limits)$$

is interpreted as: "On all computation paths (A), in all states globally (G), it is never the case that the end-effector simultaneously satisfies all three coordinate bounds." In other words, if the target point defined as in Listing 5.3, the model checker proves that the robot can never reach the point (10,15,25) within the given tolerance.

Conversely, if the property is violated, NuSMV produces a counterexample trace that demonstrates a specific sequence of joint configurations leading the end -effector into the defined target region. This counterexample can be directly interpreted as a feasible motion plan achieving the desired reachability.

The implementation of this process centers on the Counterexample class in python and a set of utilities for filtering variables, reconstructing geometry, and plotting results. This class parse NuSMV counterexample, extracts the variables of interest (end-effector coordinates, per-block coordinates, and joint angles), and provides means to render the counterexample trace in 3D graph. It also exposes numerically clean sequences that can hand to a controller.

Counterexample class shown in Appendix A.2.1 it takes raw NuSMV output and exposes methods to transform it into structured sequences for visualization and control. The source file is short—easy to audit and adapt—but covers the full parsing loop: reading states, carrying forward unchanged values, projecting specific variables, computing angles in degrees, reconstructing robot geometry step-by-step, and plotting.

Parsing the counterexample text presents a challenge because NuSMV prints only the variables that change between consecutive states. To reconstruct complete state vectors, the `filter_variables()` function employs a carry-forward mechanism. It

begins with a list of variable names of interest, such as the coordinates of each block or the end-effector, and then iterates over the textual representation of each NuSMV state. When a variable assignment is found, its value is updated in the active state vector; otherwise, the previous value is retained. This mechanism ensures that every time step in the resulting sequence contains a full snapshot of all relevant variables, even those that remained constant. Once the entire sequence is reconstructed, the program converts the data into numerical form, yielding a well-defined matrix of dimension steps × variables suitable for quantitative analysis. The resulting data provide a clear temporal evolution of the robot configuration.

When angular values are needed, the same mechanism is applied to orientation variables in function `get_angles`(), which are scaled by a discretization factor to convert symbolic or integer representations into physical degrees. This transformation connects the abstract state space of the model checker to the metric quantities used in real robot kinematics.

Beyond parsing, the class includes visualization function `plot_trace`() that convert the numeric data into three-dimensional figures. These plots display the complete end-effector path as well as the robot's configuration at selected steps, an example of the plot is shown in Figure 5.8. Each step is represented as a connected chain of block positions, preserving the spatial structure of the model. The visualization enforces uniform scaling across all axes to maintain geometric fidelity, and the plotted points are labeled to correspond with discrete time indices.

All paths and parameter values, such as the number of robot modules or the NuSMV executable location, are configurable. The implementation follows a transparent design that favors explicitness over abstraction, which makes it accessible to researchers who wish to adapt it to new model structures or naming conventions.

Once the counterexample has been parsed and verified, the data can be integrated into a robotic control pipeline.

Data from counterexample can be used directly as open-loop trajectories, in which each time step of the counterexample corresponds to a discrete command for the robot's end-effector or joint configuration. They may also serve as reference paths for closed-loop control (with feedback from sensors), allowing the robot to reproduce formally verified behaviors under physical constraints.

A typical workflow begins with the definition of the robot structure and target point in a NuSMV model file. The property under verification expresses the negation of the desired reachability condition – formally, the statement that the robot can never reach the point. If NuSMV finds that this negated property is false, it outputs a counterexample that shows precisely how the robot can, in fact, reach the point. The Python

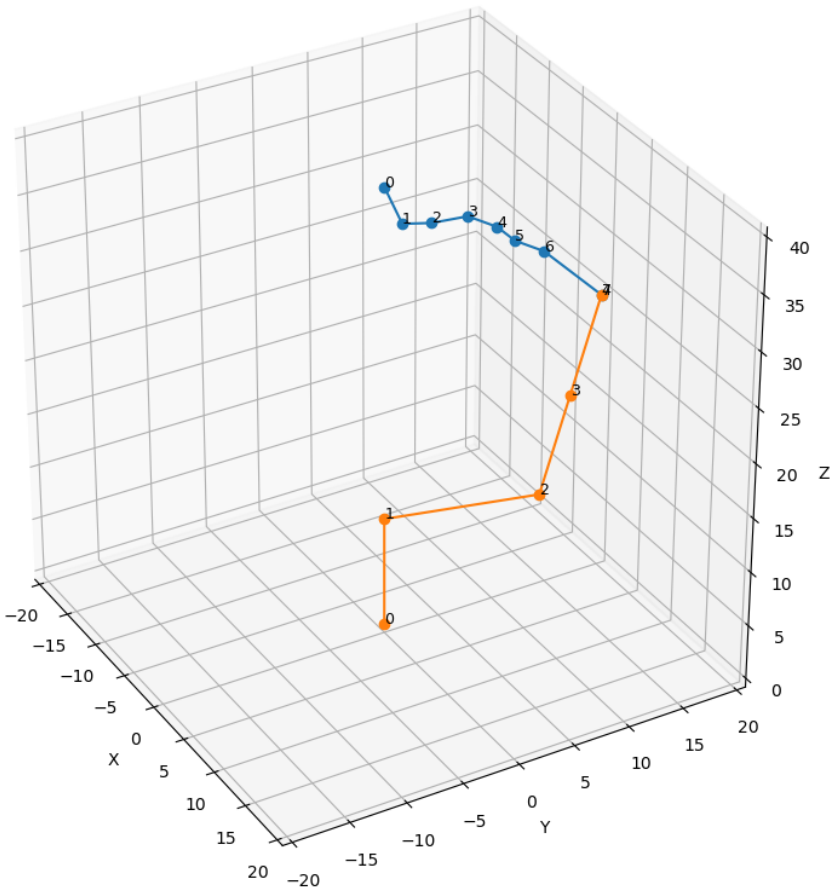

Figure 5.8: Example trace (blue) plot for target point coordinates (10,13,30) and last trace robot position (orange)

parser then extracts from this output the sequence of coordinates or angles, which can be sent to the robot controller as a reference trajectory. Visualization confirms that the motion reaches the target, and further analysis can evaluate the dynamic parameters, adjust path smoothness, or joint feasibility of the resulting motion.

By systematically reconstructing state trajectories from NuSMV output, visualizing them, and exposing them for control, the Counterexample class enables the practical reuse of verification results in robotic system experiments. Its structure is extensible for larger models, alternative naming conventions, or integration into simulation.

5.4 Reachability model checking in 2D space

In this section we will analyze another form of reachability that was described in Chapter 5.3. We will explore the reachability of various points on the field, assuming that the robot moves in steps, fixing the end effectors on the field one at a time.

In our experimental system, the robot moves on a grid of discrete cells, picking up components and assembling a new robot. Using model checking we want to ensure that the robot can successfully gather the necessary parts to replicate, or otherwise detect scenarios leading to failure.

Formal methods can verify reachability of the goal state – a completed new robot or a failure state, where replication becomes impossible and thereby increase confidence in the design. By analyzing counterexamples to these LTL properties, we can extract sequences of actions leading to success or failure.

5.4.1 Model structure for 2D space reachability

Assumptions and simplifications

To make the global reachability checking tractable, we model a simplified Self-Replicating Robotic System with several key assumptions and abstractions. These assumptions define the environment and the robot's capabilities in our NuSMV model:

1. The robot operates on a two-dimensional grid of cells –a finite field. We consider only a single end-effector that occupies one cell on this grid at any time, representing the robot's current position (with coordinates x and y). The rest of the robot's body is not explicitly modeled on the grid – instead, the structure of the robot is represented abstractly by a sequence of required part types. The end-effector is initially anchored at an assembly site, taken to be the origin cell (0,0) for convenience. All other objects (parts or obstacles) occupy discrete cell positions on the grid. An example of a grid configuration is shown on the diagram in Figure 5.9, different part types are scattered on a field and the robot must collect them.
2. The robot can move one cell at a time to a neighboring cell (up, down, left, or right), and only into cells that are unoccupied (have a type "None"). In other words, the robot's end-effector cannot move into a cell that already contains a part or an obstacle. This models a scenario where the robot must navigate around obstacles and cannot pass through or occupy the same space as existing

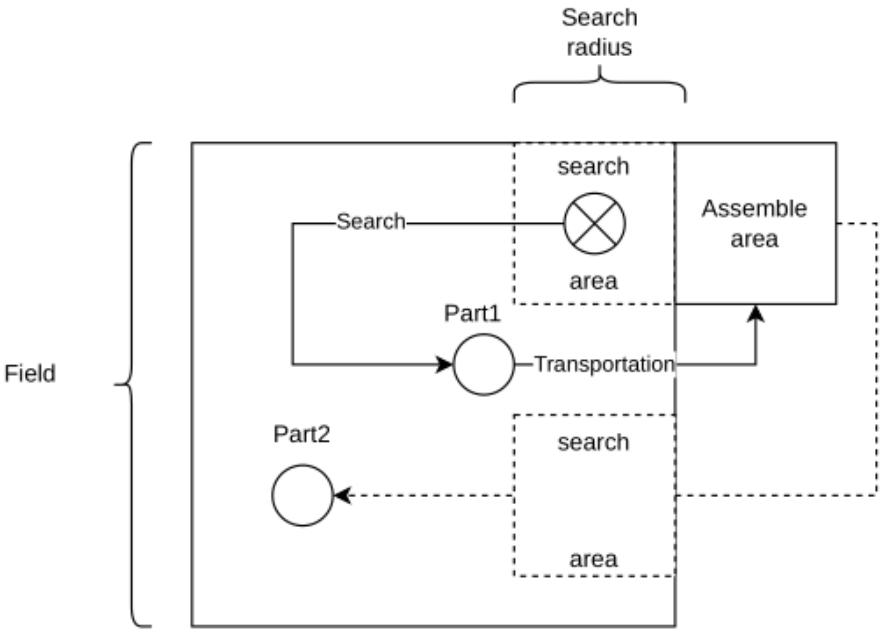

Figure 5.9: Simplified representation of a grid, robot and material parts.

objects. Movement is constrained to the four orthogonal directions on the grid (no diagonal moves). Each move is treated as a discrete time step in the model.

3. At any given position, the robot can "sense" or inspect only its immediate neighbors for parts. The reach range for checking is limited to the four adjacent cells. If a neighboring cell contains a part, the robot can decide to check that part's type. This locality assumption simplifies sensing—there is no global visibility of all parts at once, only local perception of adjacent cells.

4. When the robot finds a required part needed for replication, it will pick up that part and carry it back to the assembly location (the origin) to attach it to the new robot under construction. We abstract this entire retrieval and assembly process as a single action that does not explicitly appear as a series of steps in the NuSMV model. In other words, once a part is found, the model assumes the robot instantly returns to the assembly site with the part and the assembly index advances. This means the time and path taken to carry the part back are not modeled; the effect (having assembled one more part) is reflected in the state. This assumption significantly reduces model complexity by not requiring us to simulate back-and-forth transport or the physical attachment process. After assembly, the robot's end-effector is conceptually back at the assembly site (0,0)

and ready to search for the next part. The assembly site itself is not explicitly marked on the grid (we assume it is at the starting position). We also assume that assembling the part does not change the grid configuration in the model (the part is not removed from the grid representation when taken, since the assembly action is outside the modeled steps).

5. In a real scenario, the part would be removed from the environment once collected. However, by not altering the grid, we avoid introducing additional transitions; we rely on the assembly index to track progress rather than the physical absence of the part.

Under these assumptions, the self-replication process can be seen as a sequential loop of searching for the next required part, checking if an encountered part matches the needed type, assembling the part if it matches, which implicitly resets the search to the next part, and continuing the search for subsequent parts until either the new robot is fully assembled or no further progress is possible.

The high-level behavior can be represented by a state chart, as depicted in Figure 5.10.

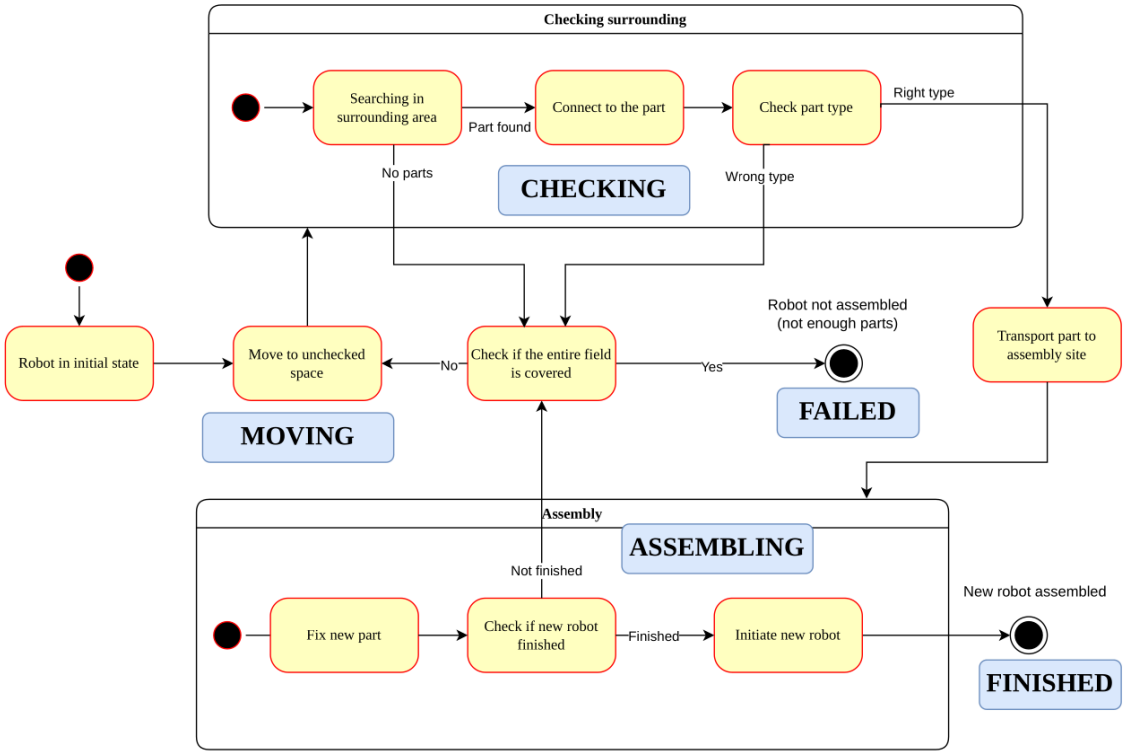

Figure 5.10: State chart of the self-replication system modeled for global reachability

In this state machine, the robot has distinct modes: MOVING, CHECKING, ASSEMBLING, and two terminal states: FINISHED and FAILED.

Initially, the robot is in the MOVING state, roaming the grid to find parts. When a neighbor cell is occupied by some object, the robot transitions to CHECKING and inspects that object's type.

If the object is of the type that the robot currently needs the next part in the assembly sequence, the robot enters the ASSEMBLING state, meaning it picks up the part and (in one abstract step) delivers it to the assembly point and attaches it.

This successful assembly of a part causes the robot to increment an internal counter of how many parts of the sequence have been assembled.

The robot then returns to MOVING to search for the next part, unless the sequence is now complete, in which case it transitions to FINISHED and means a new robot has been successfully constructed.

If the robot is in CHECKING state and finds that the adjacent part is not the type it needs, it will go back to MOVING state and continue searching (the unwanted part is left in place).

The process repeats until either the FINISHED state is reached (all required parts found in order) or a FAILURE condition occurs. A failure state is reached if the robot cannot find any required part and cannot move to any new location – essentially, the robot is stuck with remaining parts needed, the assembly sequence isn't complete, but with no available moves or accessible needed parts. In our model, this condition is captured by the situation where the robot is in MOVING state, still has parts to assemble, and every adjacent cell is either occupied (not NONE) by something (which might not be the needed part) or is out of bounds of the grid. In such a case, the model transitions the robot to FAILED, a terminal state indicating assembly cannot be completed.

Both FINISHED and FAILED are terminal states in the model, that means once reached, the robot remains in that state. This state logic forms the basis of our NuSMV model.

5.4.2 NuSMV Formal Model Structure

We constructed a formal model in NuSMV to capture the described behavior. The model is centered around the robot's position and state, the grid environment, and the assembly progress. In following sections we describe the main components of the model.

State variables and initial conditions

The NuSMV model defines discrete state variables corresponding to the robot and environment. Key variables include: grid, x and y coordinates, robot_state,sequence and checked_cells.

Grid or field is a 2D array representing the environment, indexed from 0 to field_max_index in both dimensions. Each cell in the grid can hold either "NONE" (if the cell is empty) or a part type or obstacle type. We define a set of possible cell values, e.g., NONE, TYPE1, TYPE2, TYPE3, OBSTACLE1,... . In a given scenario, some of these types will be present and others not, but the model is capable of representing any configuration of parts. We initialize grid with a specific configuration where a handful of cells have parts or obstacles and the rest are NONE. An example of a small 4 x 4 size grid initialized as it shown in Listing 5.9:

```
    initial_grid := [
    [NONE,  NONE, NONE, NONE, NONE, NONE],
    [NONE,  NONE, NONE, NONE, NONE, NONE],
    [NONE,  NONE, NONE, NONE, NONE, NONE],
    [NONE,  NONE, NONE, TYPE2,  NONE, NONE],
    [TYPE1, NONE, NONE, NONE, NONE, NONE],
    [NONE,  NONE, NONE, NONE, TYPE3,  NONE]
];
```

Listing 5.9: Initial representation of grid in NuSMV model

This example corresponds to a field where three part objects (of types TYPE1, TYPE2, TYPE3) are placed at specific coordinates, and the rest are empty cells (NONE). An obstacle could similarly be placed by setting a cell to OBSTACLE. The robot's starting position is (0,0), which in this example is also the assembly site and is empty.

2D coordinates x and y represent the robot's current position, integers ranging from 0 to field_max_index, where field_max_index = N-1 if the grid is N x N. These coordinates represent the cell where the robot's end-effector is located.

Robot state represented by robot_state variable – the mode of operation of the robot, which can take values MOVING, CHECKING, ASSEMBLING, FINISHED, FAILED. This enumerated type encodes the state machine described earlier.

An integer variable assembly_index tracking how many parts of the target assembly sequence have been completed, or equivalently, the index of the next required part in the sequence. Initially, assembly_index = 0 meaning no parts have been assembled yet, and when assembly_index exceeds robot_size the assembly is complete. In practice, reaching the last part triggers the FINISHED state as described below.

An array sequence, holds the required part type for each index of the robot's assembly sequence. This array is of length (robot_size + 1). For example, if the robot must assemble parts of types TYPE1, TYPE2, TYPE2, TYPE3 in that order, then robot_size = 3 and sequence[0]= TYPE1, sequence[1]=TYPE2, sequence[2]=TYPE2, sequence[3]=TYPE3. In the model initialization, we explicitly set each element of sequence to define the blueprint of the robot we want to assemble. This sequence is a parameter to the model, different robot designs can be specified by changing these initial values.

An auxiliary 2D array checked_cells (has the same dimensions as grid) used to record which cells have been visited during the search. Each element of checked_cells can be either UNKNOWN or VISITED. Initially, all cells are UNKNOWN. The intention is to mark cells as VISITED once the robot has explored them, to prevent the robot from infinitely revisiting the same empty cells in a loop. In the model, we will constrain movements such that the robot does not move into a cell that is already marked VISITED. This prevents trivial cycles and helps the model checker focus on new areas. The checked_cells array is updated when the robot moves: when transitioning to a new position, that position can be marked as visited. In our NuSMV model, this is encoded implicitly through transition constraints rather than an explicit assignment.

At the beginning, we initialize the model variables as follows. The robot starts in state MOVING, with x = 0 and y = 0 (assembly point). The assembly index is 0, which means no parts assembled yet. The grid is set to the predefined configuration of parts and obstacles for that scenario. The sequence array is initialized with the desired part sequence for the robot to build. All cells in checked_cells start as UNKNOWN, except the starting cell which can be considered implicitly visited.

State transition

The core of the model is the state transition logic, which we encode using ASSIGN and TRANS blocks. We break down the transitions by robot state, and the simplified transition system shown in Figure 5.11

In the NuSMV model, the logic with robot_state is realized with a structured case statement for next(robot_state) in ASSIGN section. When the robot is in MOVING state, it is actively changing the coordinates. We implement two key behaviors for MOVING:

If the robot is in MOVING and a required part is adjacent (in any of the four neighbors), then it should transition to CHECKING, as it shown in Listing 5.10.

```
1  (robot_state = MOVING &
2   ((x > 0                & grid[x - 1][y] != NONE) |
3    (x < field_max_index  & grid[x + 1][y] != NONE) |
4    (y > 0                & grid[x][y - 1] != NONE) |
```

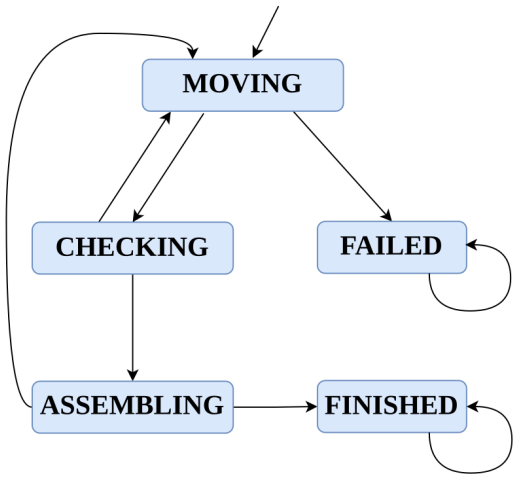

Figure 5.11: Simplified transition system relative to robot_state variable

```
    (y < field_max_index  & grid[x][y + 1] != NONE))
) : CHECKING;
```

Listing 5.10: Asign for transition from MOVING to CHECKING

If the robot is in MOVING and no required part can ever be found, the robot should give up and enter FAILED, this part shown in Listing 5.11. The condition we use to approximate "no required part exists or no moves possible" is: the robot is MOVING, still needs parts (assembly_index <= robot_size), and all adjacent cells are blocked (no empty neighbor to move into).

```
(robot_state = MOVING &
(assembly_index <= robot_size) &
((x = 0                  | grid[x - 1][y] != NONE ) &
  (x = field_max_index   | grid[x + 1][y] != NONE) &
  (y = 0                 | grid[x][y - 1] != NONE ) &
  (y = field_max_index   | grid[x][y + 1] != NONE ))
) : FAILED;
```

Listing 5.11: Assign for transition MOVING to FAILED

If at the current position none of the four directions offers an empty cell (i.e. the robot is surrounded by occupied cells or edges), then the robot cannot move. captures dead-end situations (e.g., the robot is boxed in by obstacles or irrelevant parts that are not the one it needs).

The actual movement to a new cell is not decided in the next (robot_state), instead, movement is governed by separate TRANS constraints described later. In essence, if

the robot stays in MOVING for the next step, we allow its (x, y) to change to an adjacent empty cell that hasn't been visited yet. We encoded movement using additional transition constraints rather than in the main case, to separate the logic of where it moves from when it moves in the state machine.

When in robot_state is in CHECKING, the robot is examining a specific neighboring part's type to see if it's what we need, the formal description is in Listing 5.12.

```
    (robot_state = CHECKING &
    assembly_index <= robot_size) : case
    grid[partX][partY]  = sequence[assembly_index] : ASSEMBLING;
    grid[partX][partY] != sequence[assembly_index] : MOVING;
    esac;
```

Listing 5.12: Assign for transition from CHECKING state and changing of auxiliary partX and partY variables

We need to decide whether to go to ASSEMBLING ,if the part matches the requirement or back to MOVING if it does not.

To formalize this, we introduced variables partX and partY to record the coordinates of the part being checked, the conditions are in Listing 5.13.

```
    next(partX) := case
      ( robot_state = MOVING & next(robot_state) = CHECKING &
        x > 0 & grid[x - 1][y] = sequence[assembly_index]
      ) : x - 1;  -- found in left neighbour cell (
        robot_state = MOVING & next(robot_state) = CHECKING &
        x < field_max_index & grid[x + 1][y] = sequence[
            assembly_index]
      ) : x + 1;  -- found in right neighbour cell
      TRUE : x;  -- otherwise, retain previous (or irrelevant outside
          checking context)
    esac;
    next(partY) := case
      ( robot_state = MOVING & next(robot_state) = CHECKING &
        y > 0 & grid[x][y - 1] = sequence[assembly_index]
      ) : y - 1;  -- found in upper neighbour cell
      ( robot_state = MOVING & next(robot_state) = CHECKING &
        y < field_max_index & grid[x][y + 1] = sequence[
            assembly_index]
      ) : y + 1;  -- found in down neighbour cell
      TRUE : y;
    esac;
```

Listing 5.13: Assign for transition from CHECKING state and changing of auxiliary partX and partY variables

When the robot transitions from MOVING to CHECKING, we simultaneously set partX and partY to the coordinates of the neighbor that it decided to inspect. In the model, we achieve this by writing next(partX) and next (partY) with conditions mirroring the state change.

For example, one rule is: if robot_state = MOVING and next(robot_state) = CHECKING and the left neighbor cell (x-1, y) contains the needed part type (grid[x-1][y] = sequence[assembly_index]), then next(partX) = x-1 and next (partY) = y. Similar rules set partX and partY for the right, up, or down neighbor if that neighbor is the target part.

If the part at (partX, partY) matches the required type for the current assembly index (i.e., grid[partX][partY] = sequence[assembly_index]), then the robot will proceed to pick it up and assemble it.

If the part is not the needed type (grid[partX][partY] != sequence[assembly_index]), then the robot will return to MOVING to keep searching. So we have next(robot_state) = MOVING for that case. We coded this using a nested case within the CHECKING branch in NuSMV (so that from CHECKING , the next state is a sub-case that depends on a comparison of the part 's type).

The ASSEMBLING state represents that the robot has successfully fetched a part and is adding it to the assembly. We consider that the assembly of one part is completed within this state, so the next step depends on whether there are more parts to assemble. The implementation of this transition shown in the Listing 5.14

```
1     (robot_state = ASSEMBLING
2     ) : case
3     assembly_index < robot_size : MOVING;
4     assembly_index = robot_size : FINISHED;
5     esac;
```

Listing 5.14: Assign for transitions from ASSEMBLING state

If the robot was assembling a part, and it was not the final part (i.e. assembly_index < robot_size before assembly), then after assembling, the robot should go back to MOVING to search for the next part. We model this by next(robot_state) = MOVING when robot_state = ASSEMBLING and assembly_index < robot_size. Additionally, we will update the assembly_index: while in ASSEMBLING, if it's not the last part, we increment assembly_index by 1 for the next state (so the needed part type will advance to the next in the sequence).

If the robot was assembling and this part was the final one needed (assembly_index = robot_size, meaning we just placed the last part), then the assembly is now complete. Therefore, we transition to FINISHED. In this scenario, we also increment as-

sembly_index (although it's not strictly necessary once we mark FINISHED, but it could be used to indicate assembly_index = robot_size+1 meaning " done").

In summary, ASSEMBLING is a transient state: it will always go either to MOVING (with the next part index) or to FINISHED, in one step.

FINISHED and FAILED are the absorbing terminal states. We encode in Listing 5.15 that if the robot is already in FINISHED, it stays in FINISHED, and similarly for FAILED.

```
      robot_state = FINISHED   : FINISHED;
      robot_state = FAILED     : FAILED;
```

Listing 5.15: Terminal conditions for FINISHED and FAILED

Thus, once a terminal condition is reached, no further transitions occur and the trace effectively ends.

In addition to the robot_state updates, the model includes constraints on how the x and y coordinates can change between steps. The movement and visit tracking logic described in TRANS block of the logic.

If the robot will remain (or go) in the MOVING state at the next step then we require that next(x), next(y) is one of the neighboring cells of (x, y) that is empty and not yet visited, this part shown in Listing 5.16.

```
  TRANS
    (next(robot_state) = MOVING ) -> (
        ( next(x) = x - 1 & x > 0
        & next(y) = y
        & grid[x - 1][y] = NONE
        & checked_cells[x - 1][y] != VISITED
        ) |
        ( next(x) = x + 1 & x < field_max_index
        & next(y) = y
        & grid[x + 1][y] = NONE
        & checked_cells[x + 1][y] != VISITED
        ) |
        (next(y) = y - 1 & y > 0
        & next(x) = x
        & grid[x][y - 1] = NONE
        & checked_cells[x][y - 1] != VISITED
        ) |
        (
        next(y) = y + 1 & y < field_max_index
        & next(x) = x
        & grid[x][y + 1] = NONE
        & checked_cells[x][y + 1] != VISITED
        ));
```

Listing 5.16: Transition of x and y coordinates during MOVING

This big disjunction covers each of the four directions. It essentially means: if we are continuing to move, the robot must move by one step into a free cell that has not been visited before. It prevents illegal moves (e.g., jumping multiple cells or moving into occupied cells) and also prevents staying in place during a MOVING step.

If the robot's next state is not MOVING (next(robot_state) != MOVING), that implies the robot is engaging in checking or assembling, so we constrain that the robot's coordinates do not change in that step, see Listing 5.17

```
1    TRANS (next(robot\_state) != MOVING) -> (next(x) = x & next(y) =
         y)
```

Listing 5.17: Condition that robot position doesnt change if not MOVING

This ensures that during a CHECKING or ASSEMBLING step, the robot stays at the same location.

We also add TRANS (next(grid) = grid) because we assume the environment's configuration of objects doesn't spontaneously change between steps (no external agent moves the parts or obstacles during the process). The only change to the grid would theoretically be removal of a part that was picked up, but as per our assembly abstraction, we are not updating the grid for that, so indeed the grid remains static in the model. Similarly, TRANS (next(sequence) = sequence) is added because the required sequence doesn't change over time (it's a fixed goal).

For completeness, we enforce that unless in ASSEMBLING (where assembly_index increments), the assembly_index stays the same across a step: TRANS (robot_state != ASSEMBLING) -> (next(assembly_index) = assembly_index). Combined with the earlier assignment rule that in ASSEMBLING state we do increment, this ensures assembly_index only goes up exactly when a part is being assembled.

In our implementation, we simplified matters by only constraining moves to not go into VISITED cells, and we implicitly assume the robot doesn't immediately revisit its last position because it's now marked visited. The initial cell (0,0) could be considered visited from the start to avoid the robot just stepping out and back in repeatedly. In practice, because the robot always has the option to move into an unvisited cell (and the model checker tends to find a path to the goal if it exists, rather than looping), we found that the first counterexample trace it finds is typically a simple path without revisiting. Nonetheless, a more rigorous model would include an explicit update of checked_cells[next(x)][next(y)] := VISITED and possibly mark the start as visited in the init section for completeness.

Specification for checking 2D reachability

To perform reachability analysis of the terminal states (FINISHED or FAILED), we frame the problem as checking LTL properties that assert that those states never happen. By checking the negation of the reachability, the model checker will provide a counterexample path when the state is in fact reachable. We used two LTL formulas, first - successful assembly never occurs, in Listing 5.18

```
1    LTLSPEC G F (robot_state != FINISHED).
```

Listing 5.18: Terminal conditions for FINISHED and FAILED

This LTL formula reads as "always eventually, the robot_state is not FINISHED," which is a somewhat indirect way of saying "robot_state is never stuck in FINISHED forever."

If this property is false, it means there exists a path where eventually robot _state = FINISHED and remains FINISHED thereafter (since once finished, it stays). In simpler terms, a counterexample to this property is a finite trace that ends in the FINISHED state (the model checker will actually produce a loop or repetition after FINISHED, but the interesting part is up to reaching FINISHED). Thus, a counterexample to this formula provides a concrete sequence of actions leading the robot to successfully assemble the new robot.

Another possible specification shown in Listing 5.19

```
1    LTLSPEC G F (robot\_state != FAILED).
```

Listing 5.19: Terminal conditions for FINISHED and FAILED

This formula asserts "it is always eventually not FAILED," i.e. the robot never gets permanently stuck in FAILED. If this is false, there is a scenario where the robot ends up in FAILED state. The counterexample trace for this formula will show how the robot ended up unable to complete the assembly.

In our NuSMV model file, we typically use first of these LTL specifications. For example, to find a successful path in 2D. But if we want to find a potential failure path, we check the second.

In all interesting cases for our scenarios, the properties are false because we specifically set up scenarios where eventually the robot does finish (we ensure all needed parts are actually present somewhere on the grid) or could fail (if we design a scenario where a part is absent or unreachable).

We leverage these traces as a source of information for analyzing and interpreting the robot behavior.

5.4.3 Model generation and verification

Writing a NuSMV model for one specific grid configuration and robot sequence is useful for a single-case analysis, but our goal was to explore many scenarios (different field sizes, different placements of parts, different robot part sequences) without manually coding each model variant. To facilitate this, we developed a Python toolchain to automate model generation, execution, and counterexample processing.

The workflow can be summarized in the following steps:

We prepare a general NuSMV model template based on the NuSMV model described in section 5.4.2. with placeholders for scenario-specific parameters.

The template, stored in a file Assemble_initial_template.smv, presented in Appendix **??**, it includes formatting placeholders such as field_max_index, object_types, init_robot_sequence, initial_grid, etc. in place of concrete values. This template captures the structure of the model (variables, state transition logic, LTL specification), leaving certain details to be filled in.

We built a simple graphical user interface,shown in Figure 5.12, using Tkinter library, to allow interactive configuration: the user can specify the field size and robot length. Then user have to press button "Generate field".

The program opens the grid with buttons for each cell, where the user can select a cell's content from a dropdown (e.g. None, or specific part types), this part shown in Figure 5.13.

The robot's required part sequence can be set via dropdowns in a side panel. This interface makes it easy to set up custom scenarios without directly editing code. The tool ensures the field and sequence lengths are consistent (it will create as many sequence slots as the robot size requires, etc.).

Once the scenario is configured, user press "Generate model" button and the program uses the provided inputs to generate a concrete NuSMV model file. This is done by taking the template and substituting in the actual values. The Python class takes user input or configuration for the scenario, such as the field size, the list of part types and obstacle types to consider, the initial positions of all objects on the field, and the robot's assembly sequence (which defines robot_size and the sequence array values).

The set of object types is derived from all items present on the grid (plus the "NONE" empty type); The robot's part sequence (a list like ["TYPE1", "TYPE2", "TYPE2", "TYPE3"]) is used both to set robot_size (length minus one) and to fill in the init(sequence[i]) lines in the model. The initial grid configuration is formatted as a matrix literal for NuSMV.

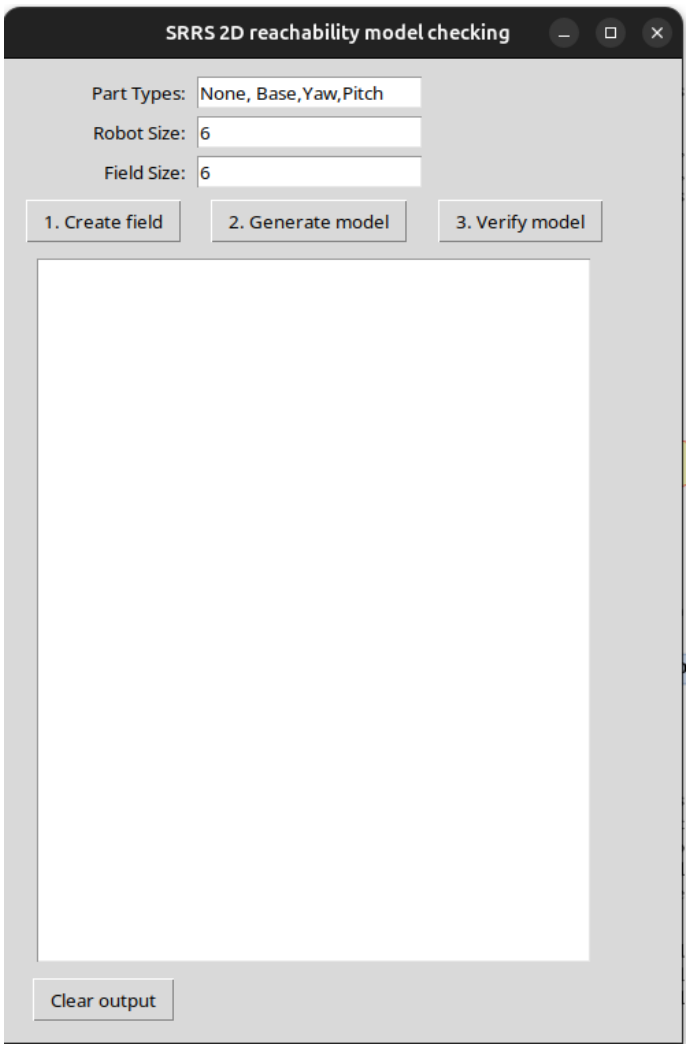

Figure 5.12: Graphical interface for 2D reachability checking

This model file is essentially the same structure for any scenario, but with different initial conditions and parameters reflecting that scenario. The Python generator prints a summary to the user, confirming the model generation and showing the initial grid and sequence it encoded, for verification purposes.

After generating the model, with "Verify model" button we verify it using NuSmv. We automate this by invoking the nuSmv binary through a subprocess call on the generated model file. The output will indicate whether the LTL specification was satisfied or violated.

The significant part of our program is parsing the NuSmv counterexample to extract the robot's movement path and key events. We focus on the variables of interest: x, y,

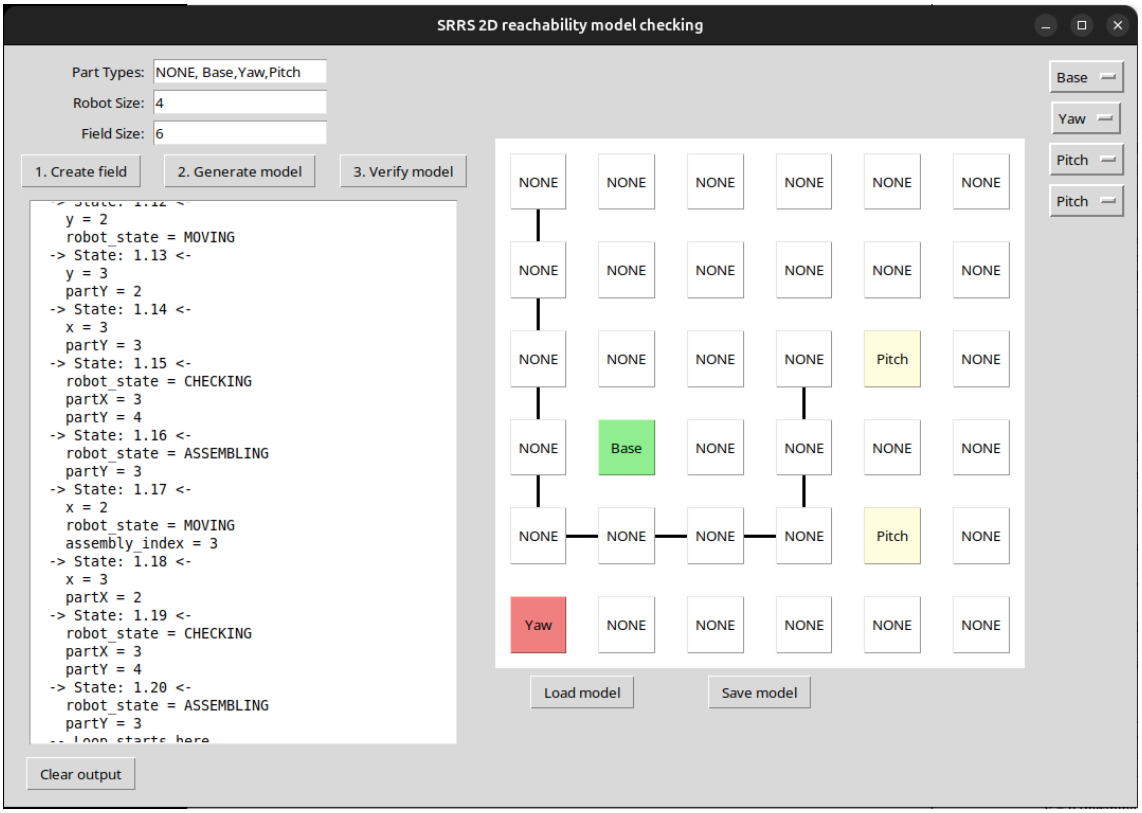

Figure 5.13: Graphical interface for 2D reachability checking with field

and the robot_state (particularly noting when it reaches FINISHED or FAILED). Our Python script reads the output line by line and filters those lines that indicate a change or value of x or y or mention the robot_state becoming FINISHED. By doing this, we reconstruct the sequence of coordinates the robot went through.

The trace format usually lists assignments in a single step as separate lines. By capturing x and y and assembling them in order, we build a list of (x, y) positions visited at each step of the counterexample path. The path indicates the cells the robot effector occupied over time. The Python GUI then takes this path of coordinates and visually draws it on the grid for the user: connecting the sequence of cell centers with lines (in chronological order) so one can see the trajectory. The output text field of the GUI also shows a cleaned-up version of the nuSmv output and/or the path coordinates.

By automating this workflow, we can iterate quickly over different scenarios and immediately get the robot's path if assembly is possible, or see where it fails. The counterexample trace effectively serves as a simulated execution of the self-replication process under the given scenario. Thus, we explore the state space with model checker and provide an example execution leading to the outcome of interest.

5.5 Robot configuration model checking

In this part we will check the model of the robot of the certain configuration for the reachability property introduced in

5.5.1 Sequential checking of possible configations

In order to systematically evaluate which robot structures are capable of reaching a target coordinates, implemented an automated configuration checking procedure that integrates model generation, model checking with NuSMV, and result filtering. The overall workflow is illustrated in Figure 5.14.

The process of checking robot configurations is performed by integrating Python - based model generation with the NuSMV model checker. The workflow begins with two primary inputs: the set of available block types, such as base, yaw-rotating, and pitch-rotating blocks, and the maximum robot length expressed as the number of blocks. These inputs are used to generate the complete set of allowed configurations of length N, ensuring that every possible arrangement of building blocks within the specified limit is considered. Each configuration is represented as an ordered sequence of blocks, starting with the mandatory base block and followed by a selection of yaw and pitch blocks. Once the set of configurations is created, the process iterates over each configuration individually.
The block sequence is extracted and combined with:

- coordinates of an initial block of a robot (x_{orig},y_{orig},z_{orig})
- assembly site coordinates relative to a robot(x_{asm},y_{asm},z_{asm})
- target block coordinates relative to a robot: [Block(x_1,y_1,z_1), Block2(x_2,y_2,z_2), ...]

This information forms the basis for generating a concrete NuSMV model through a Python script. The script instantiates the appropriate modules corresponding to each block type, establishes the connections between their input and output variables, and integrates the positional and orientation parameters across the robot chain. In addition , the script embeds the temporal logic specifications provided as LTL or CTL formulas. These specifications define the verification goals, such as whether the end-effector can reach a given target coordinates and avoids certain unsafe areas.

The generated model is then passed to NuSMV, which executes symbolic model checking to evaluate whether the specifications are satisfied. The model checker systematically explores the entire state space of the configuration, taking into account the discrete transitions of yaw and pitch blocks as well as the coordinate propagation along

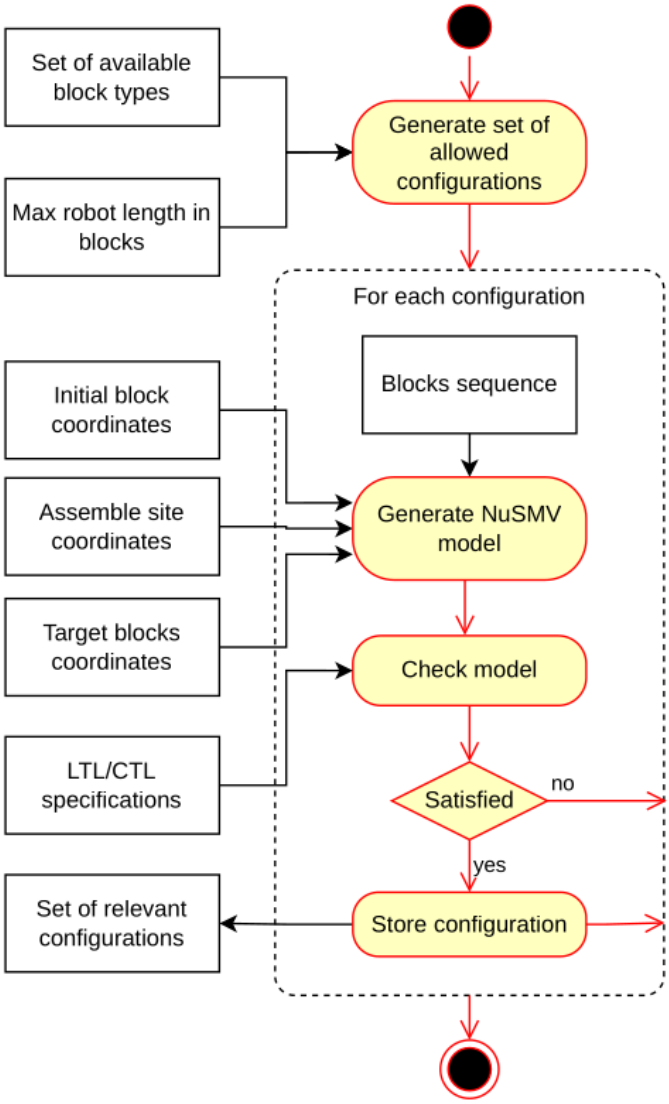

Figure 5.14: Activity diagram of searching for relevant configurations

the chain.
If the specification is not satisfied, the algorithm discards the configuration and proceeds to the next candidate in the sequence.
If the specification is satisfied, the configuration is stored in a set of relevant configurations, which represents the final output of the process.

5.5.2 Automated template-based model generation

The implementation of the workflow described in Figure 5.14 was performed in configurations checking system written in Python. The architecture of the design corresponds directly to the class diagram shown on the Figure 5.15, where the class *Configuration_checking* orchestrates the process, the class *Robot* assembles a sequence of blocks, object of the class *Block* encapsulates the description of an individual block.

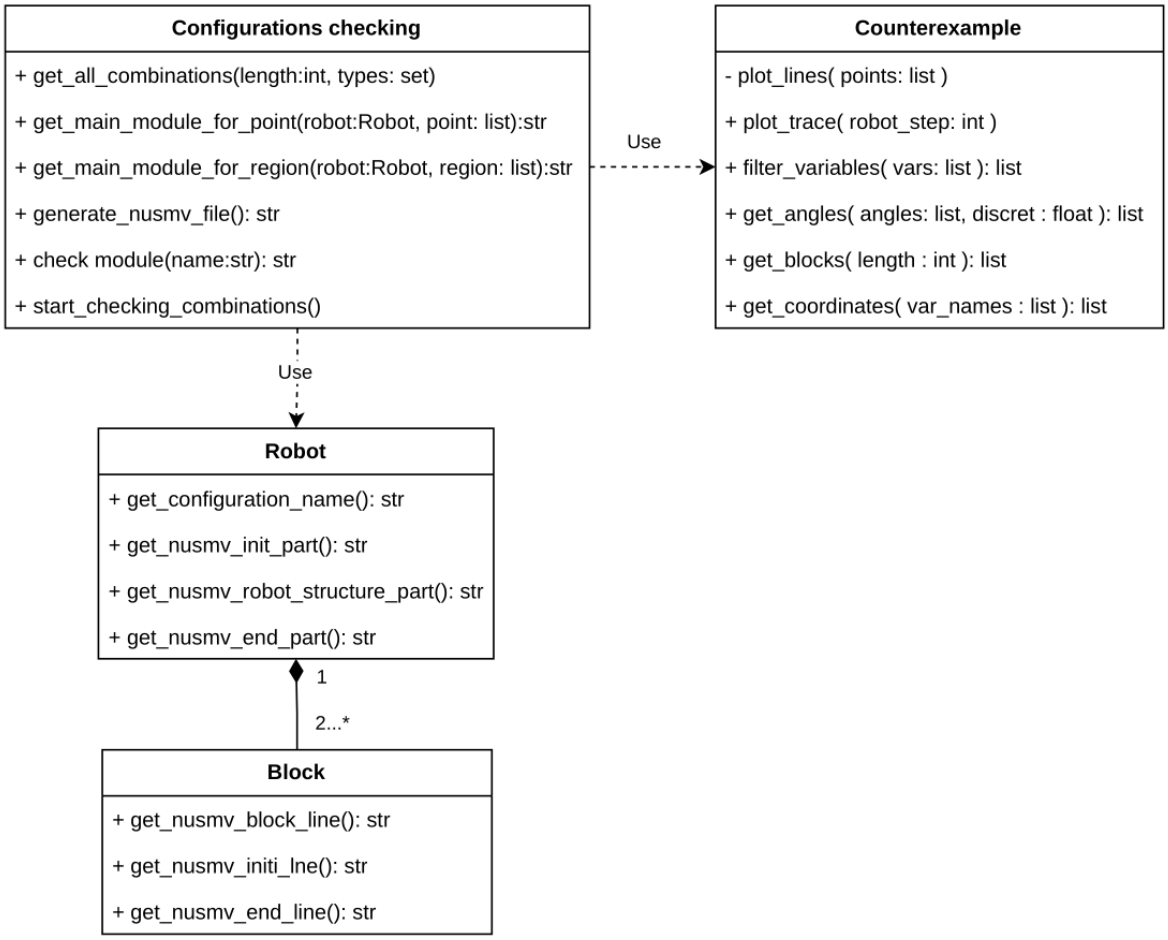

Figure 5.15: Class diagram of searching for relevant configurations

A *Block* class is the minimal self-contained representation of a kinematic segment in the robot block sequence. Each block is identified by an integer number and a type such as “base”, “yaw”, or “pitch”, and it stores the index of its parent block in order to define the topology and orientation. Geometric information is reduced to a single parameter length, while all pose and orientation information is described later through

the NuSMV model. The block provides three critical outputs in the form of NuSMV code fragments: a structure line that instantiates the variable with appropriate block module and wires it to its parent, an initialisation line that sets link length and default joint angle values, and, for the final block, assignment lines that expose the end-effector coordinates. For non-base blocks, angle variable names are generated systematically (e.g., yaw2, pitch3), guaranteeing uniqueness and readability. In this way, each block fully describes its structural role without knowledge of the entire robot, making the class simple and robust.

The Robot class acts as a container and assembler for blocks. Constructed from a list of block types, it automatically creates a chain of Block instances , numbers them, and links each to its parent. It maintains the list of blocks as well as a handle to the final block, which represents the robot's end-effector. The Robot then concatenates the fragments provided by its blocks into complete sections suitable for insertion into a NuSMV model. It can return the structure of the chain as a sequence of module instantiations, the initialisation of link lengths and angles, and the assignment of global end-effector coordinates. Additionally, it encodes the configuration in a readable name such as "base_yaw_pitch_yaw", which is used both in logging and file management. The Robot therefore translates an abstract sequence of types into a structured collection of blocks, and from there into concrete model code that describes the robot in full.

The third class, Configuration_checking, governs the entire process. Its role is to generate and test all possible configurations of a specified chain length composed of a given set of block types. The first block is always fixed to a base, and the remaining positions are filled with every possible combination of the admissible types. This results in an exponential search space, for example two types and four positions yielding sixteen unique robots. Configuration_checking takes as additional input a target point in space and stores both this point and the error tolerance around it, against which reachability is later checked. For each generated morphology, the class constructs a Robot, retrieves its NuSMV code, and splices it into a template file. The method create_nusmv_from_template searches for the region between "MODULE main" and a predefined end marker, replaces that region with a new main module generated for the current robot , and leaves the rest of the file untouched. This ensures that all library code, specifications, and fairness constraints in the template remain intact . The new module contains the structure lines from the robot, the initialisation of lengths and angles, the fixed base pose and basis vectors, the end -effector definitions, and the error-band predicates that compare the end coordinates to the desired target. This layering is explicit in the class diagram Figure5.15: Configuration_checking depends on Robot, which is composed of Block instances.

Instead of hard-coding a complete NuSMV model, the system rewrites the MAIN mod-

ule fragment inside a reusable template file. This makes the pipeline resilient to changes in property specifications or library modules—the template remains the source of truth, while the classes inject only the structure and constants.

The overall workflow combines automatic model generation, model checking, and storing of satisfiable robot configurations. By using a Python script to handle tasks of configuration expansion and NuSMV code generation, the approach ensures scalability and reduces the potential errors. The inclusion of temporal logic specifications provides a basis for determining whether a configuration is functionally capable of achieving the task requirements. The final set of relevant configurations captures only those robot designs that satisfy the given conditions, providing a reliable foundation for further analysis or deployment in robotic applications.

The result of the robot with length 4 and 5 blocks for reachability of point $x = 5, y = 5, z = 10$ shown in the Listing 5.20

```
Checking self.X=5, self.Y=5, self.Z=10
self.length=4
number of combinations:8
1/8:base_pitch_pitch_pitch_:     not reachable    [0.377735 sec]
2/8:base_pitch_pitch_yaw_:       not reachable    [0.039332 sec]
3/8:base_pitch_yaw_pitch_:       not reachable    [0.576335 sec]
4/8:base_pitch_yaw_yaw_:         not reachable    [0.01432 sec]
5/8:base_yaw_pitch_pitch_:       reachable        [0.242966 sec]
6/8:base_yaw_pitch_yaw_:         not reachable    [0.034844 sec]
7/8:base_yaw_yaw_pitch_:         not reachable    [0.062835 sec]
8/8:base_yaw_yaw_yaw_:           not reachable    [0.013127 sec]

Checking self.X=5, self.Y=5, self.Z=10
self.length=5
number of combinations:16
1/16:base_pitch_pitch_pitch_pitch_: not reachable   [15.916151 sec]
2/16:base_pitch_pitch_pitch_yaw_:   not reachable   [0.344371 sec]
3/16:base_pitch_pitch_yaw_pitch_:   not reachable   [35.386106 sec]
4/16:base_pitch_pitch_yaw_yaw_:     not reachable   [0.041149 sec]
5/16:base_pitch_yaw_pitch_pitch_:   reachable       [24.917136 sec]
6/16:base_pitch_yaw_pitch_yaw_:     not reachable   [0.727521 sec]
7/16:base_pitch_yaw_yaw_pitch_:     not reachable   [1.013776 sec]
8/16:base_pitch_yaw_yaw_yaw_:       not reachable   [0.028078 sec]
9/16:base_yaw_pitch_pitch_pitch_:   reachable       [4.10417 sec]
...
```

Listing 5.20: The output of configuration model checking for lengths of robot 4 and 5 and target region x=-5..5, y=-5..5, z=0..5

We found one configuration for the robot with 4 blocks and two configurations for the robot with 5 blocks. For these configurations we can plot the trace and robot pose, they are on the Figure 5.16

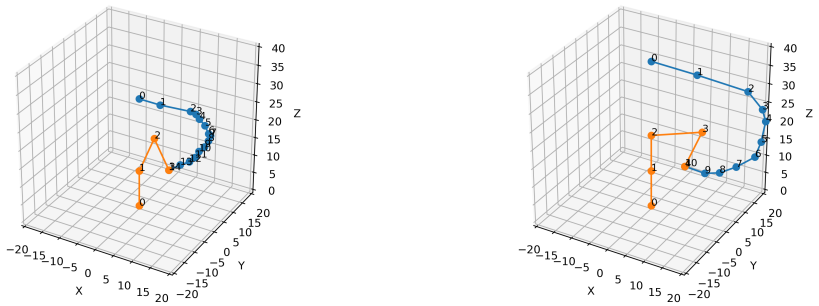

Figure 5.16: Trace for robot length 4 (left) and length 5 (right) for configurations that can reach the target region

5.6 Aggregated configuration checking in NuSMV

Alternative approach is to check all possible combinations of blocks in a robot in NuSMV itself. To implement this we add a new block to the NuSMV model described in Chapter 5.3.2. That block works as a universal interface between main module and modules for different types of blocks, the code for this block module shown in Listing 5.21.

```
1  MODULE block_general(type,px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
       zhx, zhy, zhz, L, angle)
2  VAR
3      block_y : block_yaw( px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
           zhx, zhy, zhz, L, angle );
4      block_p : block_pitch(px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
           zhx, zhy, zhz, L, angle );
5  DEFINE
6      xhx2    := case type=0: block_y.xhx2; TRUE : block_p.xhx2; esac;
7      xhy2    := case type=0: block_y.xhy2; TRUE : block_p.xhy2; esac;
8      xhz2    := case type=0: block_y.xhz2; TRUE : block_p.xhz2; esac;
9      yhx2    := case type=0: block_y.yhx2; TRUE : block_p.yhx2; esac;
10     yhy2    := case type=0: block_y.yhy2; TRUE : block_p.yhy2; esac;
11     yhz2    := case type=0: block_y.yhz2; TRUE : block_p.yhz2; esac;
12     zhx2    := case type=0: block_y.zhx2; TRUE : block_p.zhx2; esac;
13     zhy2    := case type=0: block_y.zhy2; TRUE : block_p.zhy2; esac;
14     zhz2    := case type=0: block_y.zhz2; TRUE : block_p.zhz2; esac;
15     px2     := case type=0: block_y.px2;  TRUE : block_p.px2; esac;
16     py2     := case type=0: block_y.py2;  TRUE : block_p.py2; esac;
17     pz2     := case type=0: block_y.pz2;  TRUE : block_p.pz2; esac;
```

Listing 5.21: General block serves as interface to choose and access specific types of modules

We also introduced a new variable in FROZENVAR part of the MAIN module that defines type for each block during model checking. In NuSMV, FROZENVAR are variables whose values are fixed throughout the entire execution of the model. They are assigned a value at the initialization step but do not change during state transitions, effectively acting as constant parameters that can differ between model instances. This construct is useful for representing static elements of our system that remain invariant while the system evolves, such as robot configuration and target region coordinates.

The robot structure part in MAIN module changes accordingly, to link to a general_block and pass var_block variable that specifies which type of block to use. These changes shown in the Listing 5.22.

```
MODULE main
FROZENVAR
    var_block1 : {0,1};
    var_block2 : {0,1};
    ...
VAR
    block0 : block_base(base_px, base_py, base_pz,
        base_xhx, base_xhy, base_xhz,
        base_yhx, base_yhy, base_yhz,
        base_zhx, base_zhy, base_zhz, L0);
    block1 : block_general(var_block1, block0.px2, block0.py2, block0
        .pz2,
        block0.xhx2, block0.xhy2, block0.xhz2,
        block0.yhx2, block0.yhy2, block0.yhz2,
        block0.zhx2, block0.zhy2, block0.zhz2, L1, angle1 );
    block2 : block_general(var_block2, block1.px2, block1.py2, block1
        .pz2,
        block1.xhx2, block1.xhy2, block1.xhz2,
        block1.yhx2, block1.yhy2, block1.yhz2,
        block1.zhx2, block1.zhy2, block1.zhz2, L2, angle2 );
    ...
DEFINE
    ...
    angle1     := 0;
    angle2     := 0;
    ...
```

Listing 5.22: Changes in main module to go over different types of blocks

```
CTLSPEC
EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.23: Reachability condition in CTL specification

6 Simulation of the SRRS

In this chapter, we describe a simulation-based platform designed and implemented for self-replicating robotic systems, combining the ROS 2 middleware, Gazebo physics simulation, and a modular set of Python controllers and sensor nodes. The principal goal of this system was not only to design a single robot simulation, but also to create an experimental platform that allows us to test, compare and validate different control approaches, including based on NuSMV, for self-replicating robotic systems. We structured this simulation system as a collection of ROS 2 nodes and launch files that collectively define the robotic environment, the robot itself, the material in form of parts, and the control interface.

6.1 System Architecture

For the simulation we used ROS 2 Jazzy middleware [Rob25b], Gazebo Harmonic simulator [Rob25a], a set of ROS2 launch files and nodes in Python 3.12. The full code base and list of used libraries shown in [Bab25]

The simulated environment consists of three categories of entities:

- The main robot, defined in src/srrs_sim/urdf/robot.xacro, which acts as an assembler of the new robot.
- Material parts, defined in src/srrs_sim/urdf/part.xacro, which represent the components that the robot manipulates to assemble new robots.
- The environment represented by base model, described in src/srrs_sim/sdf/base.sdf, which provides a fixed reference plane, define optional obstacles and a physical structure for attachment operations of robots.

All these elements are spawned into the Gazebo Harmonic simulation using the ROS2 launch mechanism. 3D models for these entities were created in SolidWorks 2023 modeling software based on the design proposed by [Jen+19], and converted to STL mesh files.

In order to control this simulation we developed following architecture, shown in Figure 6.1.

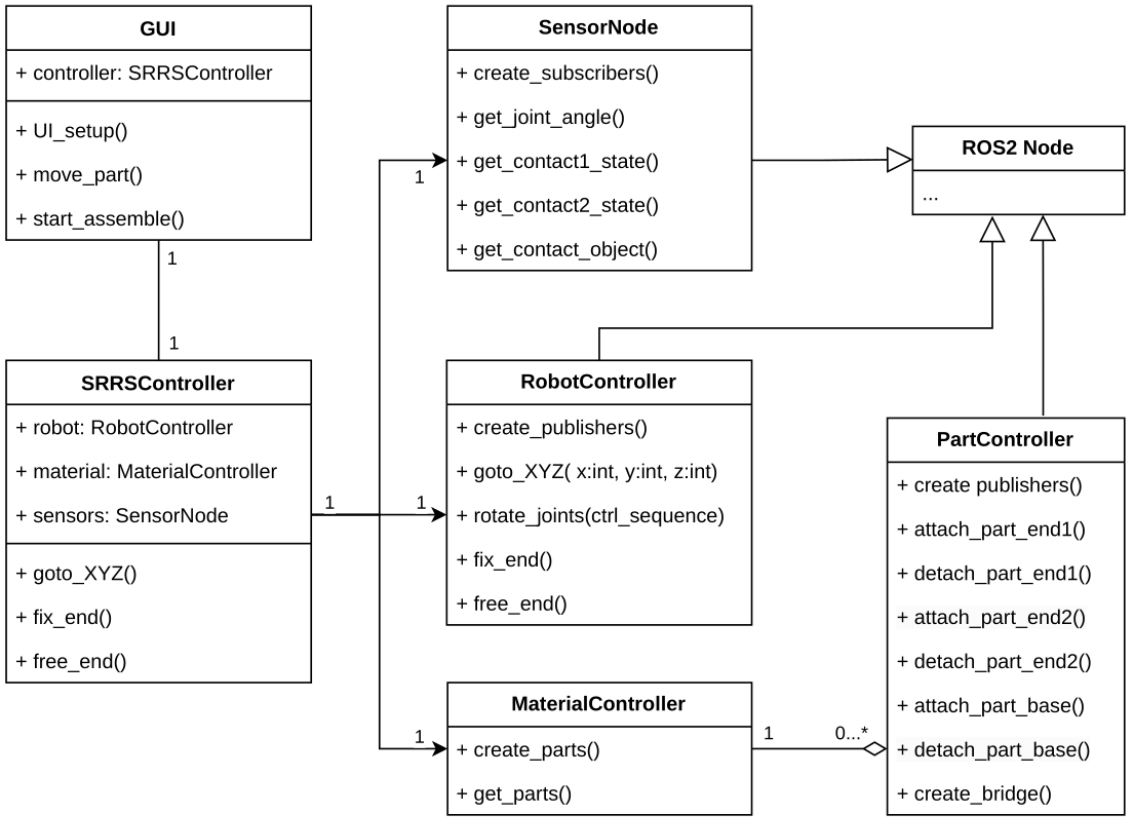

Figure 6.1: Class diagram of the simulation system

The system is designed with a layered approach, enabling interaction between the user interface, high-level control modules, and ROS2 nodes communicating with Gazebo via the ros_gz_bridge library. The architecture allows for coordinated manipulation, sensing, and part assembly in a simulated self-replicating robotic scenario.

We organized the software in several layers, from low-level motion control to high-level behavior orchestration:

- Initial launch file src/srrs_sim/launch/robot.launch
- Graphical user interface: GUI class
  located in src/srrs_sim/srrs_sim/robot_controller_gui.py file.
- Supervisory control node: SRRScontrollerNode,
  located in src/srrs_sim/srrs_sim/SRRScontrollerNode.py file.

- Low-level sensor acquisition node: SRRSsensorsNode class
  located in src/srrs_sim/srrs_sim/SRRSsensorsNode.py file.
- Low-level robot control node: RobotController class, shown
  located in src/srrs_sim/srrs_sim/robotController.py.
- Low-level material and part control nodes: MaterialController and PartController classes, located in src/srrs_sim/srrs_sim/partController.py and
  in src/srrs_sim/srrs_sim/MaterialController.py.

We start the simulation by running robot.launch file, it initiates Gazebo world and starts the graphical user interface.

GUI class provides the main entry point for user interaction. It designed to enable manual and semi-automated control of the robotic movement and assembly process. The main functions included in GUI class are: `UI_setup()`, which initializes the user interface layout, `move_part()` to command manipulations of individual elements, and `start_assemble()` to trigger the assembly sequence. Internally, the GUI maintains a reference to the SRRSController, representing the supervisory control logic of the entire system. Through this connection, user commands are propagated down to robot controller, material manager, and sensor interfaces.

The SRRSController class acts as the coordination layer that integrates the functional subsystems. It maintains three key associations: the RobotController , MaterialController, and SensorNode. These relationships form the backbone of the system's modularity, where each controller is responsible for a well-defined subset of functionalities. The SRRSController provides functions like `goto_XYZ()` - to move the free end-effector to given coordinates or `fix_end()` to fix on the base or on the part for transportation, that abstract the complexity of low-level motion and interaction control. When the user initiates an action through the GUI, the SRRSController orchestrates the appropriate response by invoking methods from subordinate controllers.

An object of RobotController class handles direct motion control of the robot within Gazebo. It designed to generate and publish control commands to the simulation environment through ROS2 topics. This class represents the bridge between high-level assembly planning and low-level joint- space control, encapsulating motion primitives into callable functions accessible by the SRRSController.

The SensorNode component provides perception capabilities. It functions as a ROS2 node with several subscribers to Gazebo topics relayed through the ros_gz_bridge. These topics provide information about joint angles, contact states, and detected objects in the simulation.

We implemented methods such as `get_joint_angle()`, `get_contact1_state()`, `get_contact2_state()`, and `get_contact_object()` to access structured sensory information. By maintaining real-time synchronization between the Gazebo simulation and the ROS2 communication layer, the SensorNode allows the control system to react dynamically to physical interactions occurring during assembly— such as contact detection when two parts align.

The MaterialController manages the set of parts available for assembly. It gives a possibility to create and manipulate parts in the Gazebo world, using functions `create_parts()` and `get_parts()`. These parts are represented as Gazebo models that can be spawned, attached, or detached through dedicated ROS2 launch file. The MaterialController establishes a one-to-many association with the PartController, reflecting that a single material manager may handle multiple part controllers simultaneously. Each PartController instance corresponds to a specific simulated component within the environment.

The PartController provides fine-grained control over individual parts in the simulation. It includes methods for creating ROS2 publishers and service clients, as well as specialized attachment and detachment procedures for various connection points, with corresponding detachment methods. The `create_bridge()` function establishes the communication link with Gazebo through ros_gz_bridge, ensuring that changes to part states in Gazebo (such as attachment or detachment) are mirrored within the ROS 2 ecosystem.

Collectively, this architecture enables the robot in simulation to perform complex movements and assembly tasks manualy or according to predefined program. The GUI layer allows human supervision and intervention, while the SRRSController coordinates motion and assembly actions. Since each class corresponds to a logical unit of functionality, the framework can easily integrate additional controllers, such as vision modules or planning algorithms, by extending the existing class hierarchy.

6.2 Gazebo simulation elements

6.2.1 Base model

Base model spawn by initial launch file (robot.launch) and represented by static and fixed in world 3D model of lattice. In our simulation the base described in SDF file base.sdf and has a dimension of 10x10 cells, it is shown in Figure 6.2.

The Base code provided in Appendix A.4.1 and consist of a link with geometry, and fixed joint that connect link to a world.

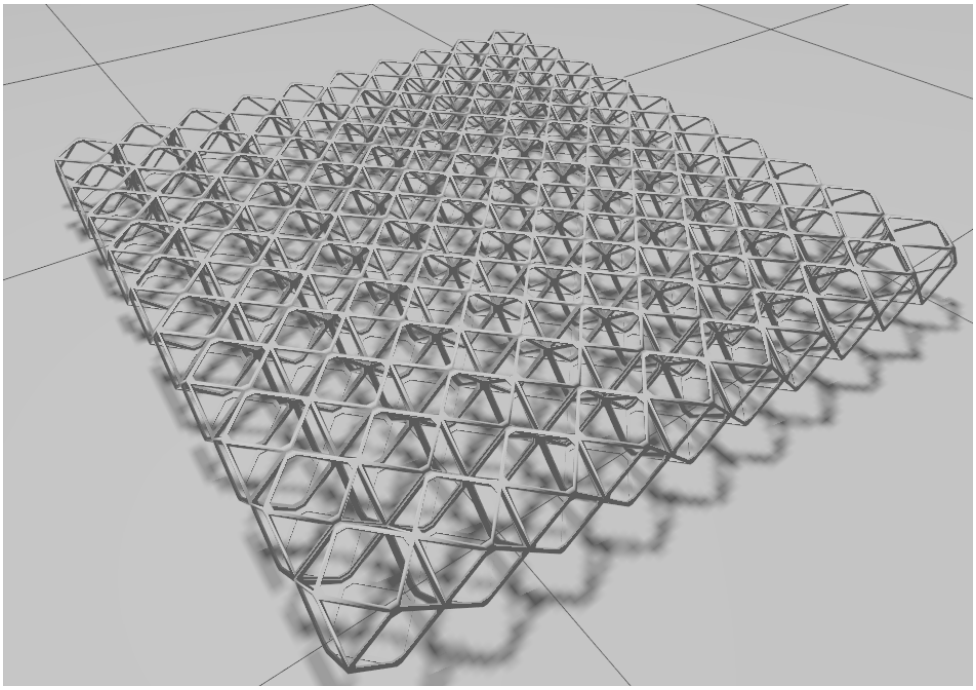

Figure 6.2: Base model in Gazebo simulator

The base represents environment and can be supplemented with additional static structures, by spawning parts with fixed joint from the same robot.launch file.

6.2.2 Parts simulation and behavior

In our simulation part represents the simplest element of a material, used to assemble a new robot. It is defined in part.xacro file and consist of a main link, shown in Listing 6.1, and detachable plugins that describe connection to base, robot end-effectors and other parts.

```
1  <link name="part">
2      <visual>
3          <geometry>
4              <mesh filename="meshes/Voxel.stl" scale="0.001 0.001
                   0.001"/>
5          </geometry>
6      </visual>
7      <collision name="part_collision">
8          <geometry>
9              <mesh filename="meshes/Voxel.stl" scale="0.001 0.001
                   0.001"/>
10         </geometry>
```

```
    </collision>
    <inertial>
        <mass value="0.1"/>
        <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0"
            izz="1.0"/>
    </inertial>
</link>
```

Listing 6.1: Part link code inside of part.xacro

One of plugins for detachable joint is shown in Listing 6.2.

The detachable joint controlled from a ROS2 with two topic messages of type Empty, separately for attach and detach actions. In xacro file we defined topic names, and child model and link in URDF tree for connection.

```
    <gazebo>
        ...
        <plugin
            filename="libgz-sim8-detachable-joint-system.so"
            name="gz::sim::systems::DetachableJoint">
            <parent_link>part</parent_link>
            <child_model>robot</child_model>
            <child_link>block0</child_link>
            <attach_topic>attach_end1_part${xacro.arg('part_num')}</
                attach_topic>
            <detach_topic>detach_end1_part${xacro.arg('part_num')}</
                detach_topic>
            <output_topic>detachable_obj_end1_state${xacro.arg('
                part_num')}</output_topic>
            <suppress_child_warning>true</suppress_child_warning>
        </plugin>
        ...
        </gazebo>
```

Listing 6.2: Detachable joint example from partẋacro

For low level control of a part we created a PartController class that is an extension of ROS2 Node, the code shown in Listing A.3.2

The PartController object provides an individual control over specific part in the simulation. It has following functionality:

- Create publishers for all topics for mechanical attachment and detachment of the part to base and both end-effectors of a robot.
- Provides methods to publish control topics in publishers.
- Starts ros-gz-bridge that connect topics (control messages) in distributed system of ROS2 with Gazebo simulated environment.

6.3 Robot simulation and control

6.3.1 Gazebo representation of a robot

As the robot configuration for further simulation we take a five-joint manipulator with two fixed ends, enabling alternating attachment and detachment to the base during locomotion and assembly, the initial configuration is shown in Figure 6.3.

The modularity of this structure allows us to simulate replication cycles where the robot assembles a set of parts into a new robot instance. This provides a flexible testbed to evaluate different control policies-deterministic sequences, feedback-based behaviors, or Model checking based strategies-without modifying the core simulator.

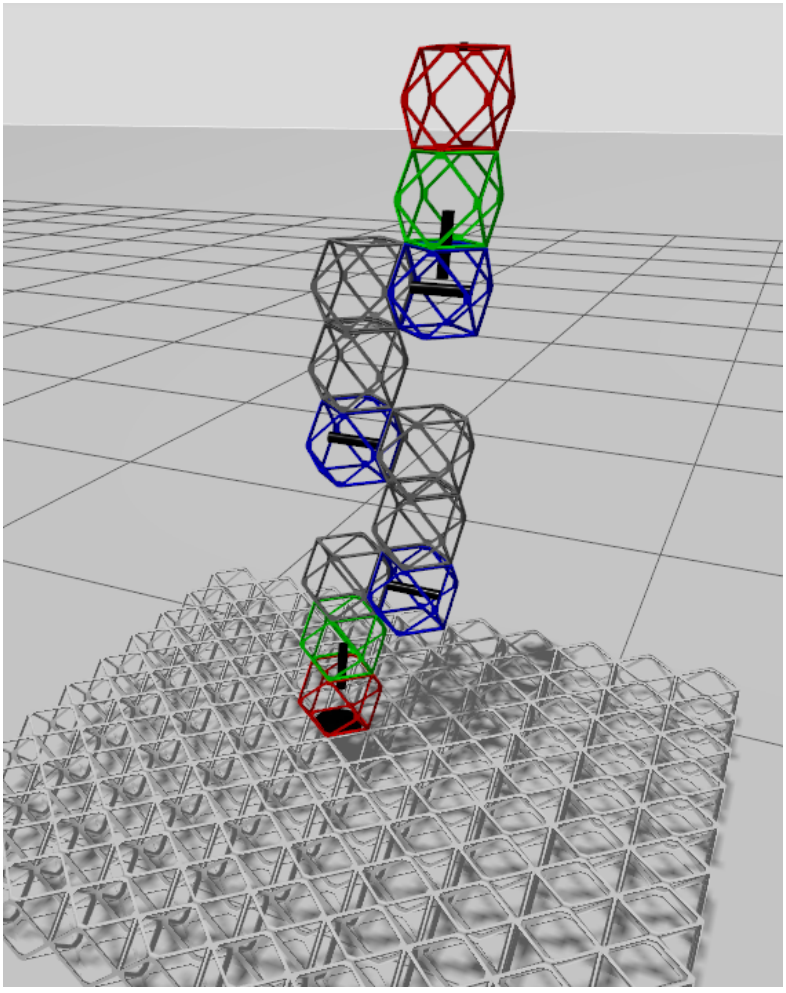

Figure 6.3: Robot structure in Gazebo simulation environment

This design emulates the behavior of a linear self-reconfiguring structure similar to a modular relative robot.

The physical description of a robot encoded in robot.xacro file and defines the structural and kinematic model of the self-replicating robot used in our simulation. It serves as a expanded description of the robot geometry, joints, sensors, and actuation interfaces, compatible with both ROS2 and Gazebo. The use of Xacro (XML Macros) provides modularity and reusability, enabling to easily modify or replicate robot configurations during simulation. Each link and joint is described using standard URDF elements, wrapped in XACRO macros to support parameter substitution,

The main structure of a robot as a sequence of blocks described in Listing 6.3. Each line in it represents a block of type: block_hor, block_ver, block_fix or block_end (block_end is equivalent to "voxel" type of block0). All of the blocks have a number, parent, coordinates relative to a parent and their types described in separate macros.

```
<xacro:voxel name="block0" origin="0 0 0" color="${red}"/>
<xacro:block_ver block_num="1"  parent_num="0" x="0" y="0" z="0"/>
<xacro:block_fix block_num="2"  parent_num="1" x="0" y="0" z="1"/>
<xacro:block_hor block_num="3"  parent_num="2" x="0" y="1" z="1"/>
<xacro:block_fix block_num="4"  parent_num="3" x="0" y="0" z="0.5"/>
<xacro:block_fix block_num="5"  parent_num="4" x="0" y="0" z="1.5"/>
<xacro:block_hor block_num="6"  parent_num="5" x="0" y="-1" z="1.5"/>
<xacro:block_fix block_num="7"  parent_num="6" x="0" y="0" z="0.5"/>
<xacro:block_fix block_num="8"  parent_num="7" x="0" y="0" z="1.5"/>
<xacro:block_hor block_num="9"  parent_num="8" x="0" y="1" z="1.5"/>
<xacro:block_ver block_num="10" parent_num="9" x="0" y="0" z="-0.5"/>
<xacro:block_end block_num="11" parent_num="10" x="0" y="0" z="0"/>
```

Listing 6.3: Robot configuration as a sequence of blocks

In simulation and on a Figure 5.2 we define following color convention:

- red - base block `block_end` or `voxel`, NuSMV equivalent: block_base,
- black - fixed block block_fix, modeled in NuSMV by length parameter of a previous block,
- blue - initially horizontal rotating block `block_hor`, NuSMV equivalent: block_pitch,
- green - initially vertical rotating block `block_ver`, NuSMV equivalent: block_yaw.

Some blocks represent static structures and are fixed to a previous blocks or base like block_fix in the Listing 6.4.

```
    <xacro:macro name="block_fix" params="block_number parent_number
       x y z " >
```

```
        <xacro:voxel name="block${block_number}" origin="${x*
            cell_step} ${y*cell_step} ${z*cell_step}" color="${grey}"
            />
        <xacro:joint_fix name="fixed${parent_number}_${block_number}"
             parent="block${parent_number}" child="block${block_number
            }"/>
    </xacro:macro>
```

Listing 6.4: Block type block_fix as a macros

Others have rotating joints, that can be controlled, the example of a block_hor macros shown in Listing 6.5, its

```
    <xacro:macro name="block_hor" params="block_number parent_number
        x y z " >
        <xacro:joint_revolution name="rev${parent_number}_${
            block_number}" parent="block${parent_number}" child="
            block${block_number}" axis="0 1 0" origin="${x*cell_step}
            ${y*cell_step} ${(z+0.5)*cell_step}" rpy="1.57 0 0"/>
        <xacro:voxel name="block${block_number}" origin="${0} ${0} $
            {-0.5*cell_step}" color="${blue}"/>
    </xacro:macro>
```

Listing 6.5: Block type block_hor as a macros

Each block defined with a name, parent block relative to which it rotates, rotation axis, coordinates and color.

Each joint_revolution is connected to a ros2_control plugin, described in the Listing 6.6 and connected joints to ROS2 system.

```
    <ros2_control name="GazeboSimSystem" type="system">
        <hardware>
            <plugin>gz_ros2_control/GazeboSimSystem</plugin>
        </hardware>
        <joint name="rev0_1">
            <command_interface name="position"/>
            <state_interface name="position"/>
        </joint>
        <joint name="rev2_3">
            <command_interface name="position"/>
            <state_interface name="position"/>
        </joint>
        <joint name="rev5_6">
            <command_interface name="position"/>
            <state_interface name="position"/>
        </joint>
        <joint name="rev8_9">
            <command_interface name="position"/>
            <state_interface name="position"/>
```

```
20          </joint>
21          <joint name="rev9_10">
22              <command_interface name="position"/>
23              <state_interface name="position"/>
24          </joint>
25      </ros2_control>
```

Listing 6.6: Block type block_hor as a macros

Robot also contains global parameters, that define physics and interaction with parts and base. Global parameters allow consistent scaling of geometry, mass, and joint limits across the entire robot. We consider a link_length parameter to be a unified length unit in our discrete environment, it is a size of a voxel side, and one step on a base lattice. shown in Listing 6.7

```
1 <xacro:property name="link_length" value="0.134"/>
2 <xacro:property name="link_mass" value="0.2"/>
3 <xacro:property name="joint_limit" value="3.1416"/>
```

Listing 6.7: Parameters of a robot in robot.xacro

6.3.2 Low level motion control

As it described in before, Gazebo model of a robot provides an ability to connect to parts or base and to rotate around multiple revolute joints.

We developed a separate class RobotController, shown in Appendix A.3.1 that extends a ROS2 Node in order to control this functions on a low level by generating and publishing control commands through ROS2 topics. The most important methods of a class are:

- goto_XYZ(x, y, z) for commanding Cartesian movements
- calculate_joint_angles() for calculation of joint angles based on robot geometry
- rotate_joints(ctrl_sequence) for performing articulated joint rotations

The function goto_XYZ(x, y, z) defines the high-level command for moving the robot's end-effector to a position with given linear coordinates. The function recursively decomposes the trajectory into steps, on each step if the point is not reachable from the current point (length is more than a step length) robot perform a striding movement in target direction, lines 6-28 in Listing 6.8. When it came closer to a target it invokes the lower-level function calculate_joint_angles() to calculate joint angles for necessary target point, line 30 in Listing 6.8. And these angles then used as an input for rotate_joints() function.

```
def goto_XYZ(self, x, y, z, step_size=3):
    x = float(x)
    y = float(y)
    z = float(z)
    # Choose if target point in near or far form (0,0)
    if abs(x) > step_size or abs(y) > step_size:
        # going to a far located point
        if x > step_size:
            stepX = step_size
            x = x - step_size
        elif x < -step_size:
            stepX = -step_size
            x = x + step_size
        else:
            stepX = 0
        if y > step_size:
            stepY = step_size
            y = y - step_size
        elif y < -step_size:
            stepY = -step_size
            y = y + step_size
        else:
            stepY = 0
        # Step algorithm:
        self.goto_XYZ(stepX, stepY, 1)
        self.goto_XYZ(stepX, stepY, 0)
        self.swap_fix_block()  # swap "legs"
        self.goto_XYZ(x, y, z, step_size)
    else:  # near pose calculation
        command_sequences = self.calculate_joint_angles(x, y, z)
        self.rotate_joints(command_sequences)
```

Listing 6.8: Function goto_XYZ() of RobotController class

The function calculate_joint_angles() translates the target Cartesian coordinates (x, y, z) relative to the fixed base to the target angles for each of joints, and returns these angles.

Method rotate_joints() allows high-degree-of-freedom manipulation by publishing target angles in topics by name of joints, used for joints definition in xacro file, in our example it is: rev0_1, rev2_3, rev5_6, rev8_9, rev9_10, as it shown in Listing 6.6.

Joint angle calculation function and internal function relative_angles(), shown in Listing A.3.1 on lines 97-144, serves to calculate joint angles based on chosen combination of blocks and translate Cartesian coordinates to angles only for our robot geometry.

Mathematically during this translation, we compute three internal angular parameters for a given target point (x, y, z):

$$\alpha = \arcsin(\frac{\sqrt{r_0^2 + z^2}}{4})$$

$$\beta = \arctan(\frac{y}{x})$$

$$r = \sqrt{y^2 + x^2}$$

$$\gamma = \arctan(\frac{z}{r_0})$$

,where:

α - is a pitch of middle block without correction

β - yaw of a first block

γ - correction of α

$r_0 = \sqrt{x^2 + y^2 - 1}$ is a correction term depending on the robot geometry.

These angles are used to derive the five joint commands, as it shown on the Figure 6.4:

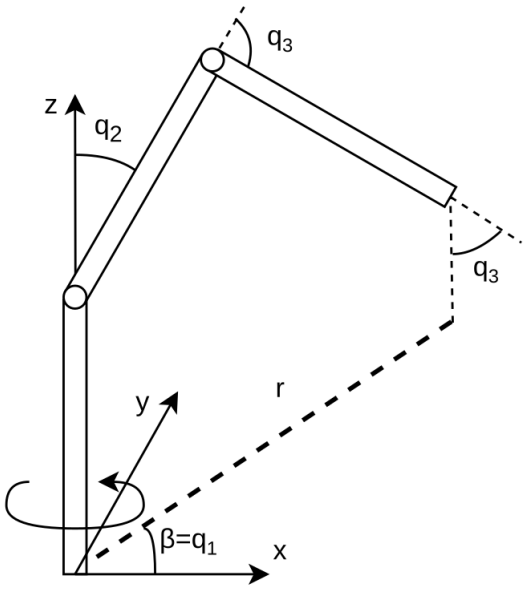

Figure 6.4: Joint angles calculation

$$
\begin{aligned}
q_1 &= \beta_0 \\
q_2 &= \alpha - \gamma \\
q_3 &= 180^0 - 2\alpha \\
q_4 &= \alpha + \gamma - \sigma^z \\
q_5 &= \beta_0
\end{aligned}
$$

with σ^z being a vertical correction term (0^o or 90^o depending on z). The controller converts these degrees to radians and publishes them as Float64MultiArray messages to each joint topic "position_controllerN" commands. The motion is considered complete when the difference between the target and current joint angles is below a threshold, in our example it is defined as $\epsilon = 2^o$. And the described calculation of angles ensure that the last block of our robot is vertically oriented and directed downwards to manipulate the underlying part.

The described approach constitutes an open-loop control with feedback verification: we do not continuously adjust the trajectory, but we ensure that each commanded position is reached before the next step. The convergence condition is implemented through the function `wait_movement_finish()` using ROS 2 timers and events.

6.3.3 Base and Part Interaction

To emulate self-assembly and replication, we introduced attach/detach mechanisms between the robot's end effectors, parts, and base. In our Gazebo simulation, attachment is realized through service calls bridged from ROS to Gazebo's internal link-joining mechanism. The following attachment states are defined:

$A_{R1,B}$: end-effector 1 attached to base

$A_{R2,B}$: end-effector 2 attached to base

$A_{R1,Pi}$: end-effector 1 attached to part i

$A_{R2,Pi}$: end-effector 2 attached to part i

Each of these is represented by a ROS topic such as /attach_end1_part1 or /attach_part1_base. Detachment events are the complementary /detach_* topics. The PartController node, shown in Listing A.3.2, automatically creates and maintains all necessary publishers for these topics. When a new PartController(number) instance is created, it initializes a corresponding Gazebo bridge through the ros_gz_bridge parameter bridge:

Each part is modeled as a single block that can attach to either of the robot's two end effectors or to the base via virtual "attachment joints." These joints are controlled through ROS topics using the std_msgs/msg/Empty message type to send attach/detach commands, bridged to Gazebo via ros_gz_bridge.

/attach_end1_partnum@std_msgs/msg/Empty@gz.msgs.Empty

/detach_end1_partnum@std_msgs/msg/Empty@gz.msgs.Empty

and similarly for other combinations. Each bridge connects a ROS 2 topic to a Gazebo message interface, ensuring low-latency coupling between control and simulation.

The base model, spawned from base.sdf file, serves as an inert reference body with defined attachment points aligned with the robot's initial configuration. It also defines the inertial frame for all coordinate calculations described in previous section.

6.3.4 Sensor Integration

The SRRSsensorsNode provides the sensory feedback needed for validation of contact and joint dynamics. This node subscribes to the following topics from Gazebo:

- /joint_states – to obtain joint positions and velocities,
- /robot/contact1 and /robot/contact2 – to detect physical interactions between the end effectors and other objects,
- /robot/imu1 – for inertial measurements of base acceleration and orientation.

The SRRSsensorsNode class, located in src/srrs_sim/srrs_sim/SRRSsensorsNode.py file, interprets messages of type ros_gz_interfaces/ msg/Contacts, parses them to extract collision partner names, and publishes simplified string notifications on /contact1/change_state and /contact2 /change_state. These allow the controller and GUI to display real- time contact information.

The contact state logic is time-based: if no new contact message is received for more than $\Delta t = 0.5s$, the system resets the contact flag. This ensures robustness against transient or spurious collision events.

The joint angle information from this node is also made available to the GUI for continuous visual updates. Through this feedback loop, we can evaluate not only the logical success of assembly sequences but also the physical consistency of movements and the timing behavior of the control algorithms.

6.4 Supervisory Control and GUI Integration

The starting point of the simulation is the launch file `robot.launch`, which initiates the Gazebo world, spawns the robot and the base environment, and launches the graphical interface for control (`robot_controller_gui.py`). The design ensures that the entire system can be initialized with a single command and that all subsystems–control, sensing, visualization – operate concurrently under ROS2 multithreaded executor.

High-level behavior orchestration is handled by the SRRScontrollerNode class. This node creates instances of both RobotController and PartController and coordinates their interactions. It exposes simple methods such as fix_end1_part(num), free_end2_part(num), fix_end1_base(), and others, which are directly callable from the GUI.

The graphical user interface (robot_controller_gui.py), shown in Figure 6.5, constitutes a crucial part of the system. The graphical interface is implemented in Tkinter, providing manual and automated control capabilities, live feedback of joint states, and camera streams.

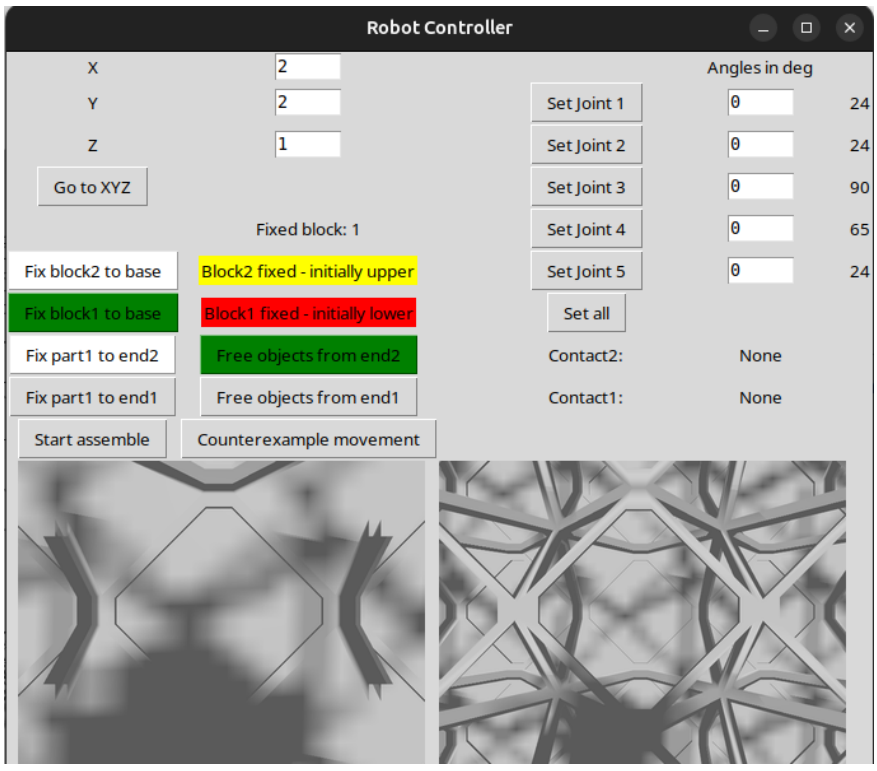

Figure 6.5: Graphical user interface for simulation control

It serves as both a human control console and an experiment orchestration tool. Built in Tkinter, it allows the operator to input target coordinates and issue movement commands, fix or release robot ends or parts through dedicated buttons, observe current joint angles, contact states, and camera feeds, execute the automatic assembly process.

The GUI continuously updates sensor readings using scheduled callbacks, ensuring a refresh rate of approximately 30–100 ms for angles and contact information. Camera frames are displayed using Pillow ImageTk interface, showing two synchronized camera streams from Gazebo topics /camera1/image and /camera2/image.

6.5 Assembly simulation workflow

When we launch robot.launch file it initiates the Gazebo world, spawns the robot and the base environment, and launches the graphical interface for control. In a typical experiment, we initialized the simulation with the robot fixed at the base and parts distributed at different coordinates.

When the "Start Assemble" button is pressed, the GUI calls start_assemble() function, which executes an automated sequence of motions to assemble a new robot from scattered parts. The GUI executes a predefined sequence:

1. Fix one of the end-effectors of the robot to the base.
2. Set assembly coordinates.
3. Move to current part, pick it up, and place it at the assembly location.
4. Increment the z-coordinate for each subsequent layer and repeat from step 1.

This sequence is implemented in the method move_part(num, x, y), which demonstrates a loop of high-level pick-and-place actions using the low-level goto_XYZ() function calls and attachment commands. Simulation steps of the assembly process shown in Figure 6.6

Each movement is performed in small steps to avoid collisions, and the GUI waits for the movement threads to complete before issuing the next command. This process is visualized in Gazebo and confirmed through GUI feedback of contact states.

After the execution we can return to the initial pose or pose with arbitrary joint angles by manualy entering angles in degrees into input fields and pressing "Set Joint" buttons. We also can move end-effector to arbitrary Cartesian coordinates (x,y,z) entering it in corresponding fields and pressing "Go to XYZ" button.

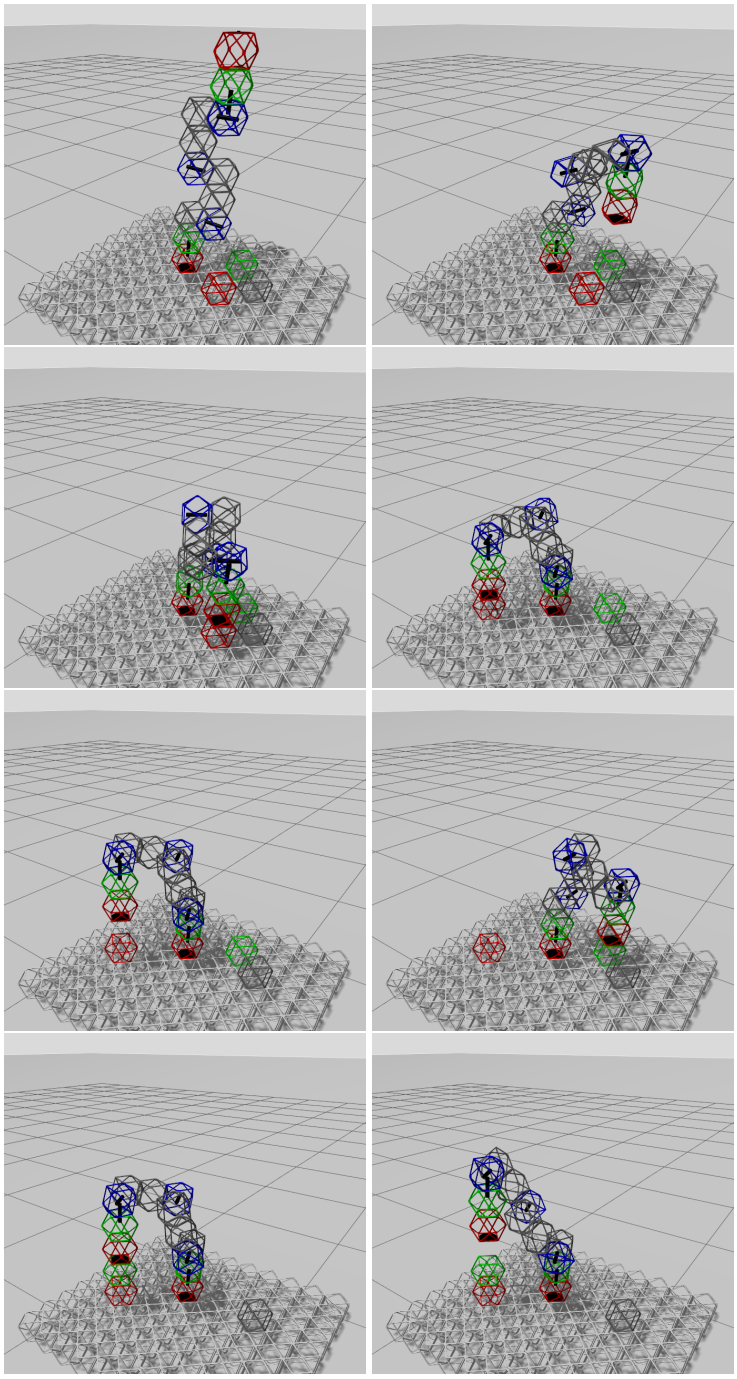

Figure 6.6: Assembly process steps for two initial blocks of a new robot

Additionally, we can move to predefined coordinates using counterexample-based control, this approach was presented in Chapter 5.3.4 and the simulation will be used for

validation in Chapter 7.1.

6.6 Implementation challenges and design considerations

Developing this system required addressing several challenges intrinsic to both ROS 2 and Gazebo:

- Synchronization of simulated physics and ROS 2 communication. Gazebo Harmonic introduces slight delays in joint updates. Therefore, we implemented event-based waiting (threading.Event) rather than time-based sleeps to detect motion completion.
- Attachment semantics. Because Gazebo joint creation for attachments can introduce instabilities, we used lightweight Empty message topics rather than service calls, improving responsiveness.
- GUI concurrency. To prevent blocking of ROS 2 threads by Tkinter, we ran the ROS 2 executor (MultiThreadedExecutor) in a separate daemon thread, allowing GUI and simulation updates to coexist without race conditions.
- Scalability. The number of parts can be increased dynamically using spawn_part function. This function invokes a new ROS 2 process through a Bash command that calls ros2 run ros_gz_sim create, ensuring that each part receives a unique name and topic namespace.
- Extensibility. Because each node is encapsulated, new sensors or controllers can be added by extending the base class rclpy.node.Node.

Through the development of this project, we established a modular and extensible simulation environment for studying self-replicating robotic systems. Our implementation demonstrates that the integration of ROS2 and Gazebo provides a sufficiently realistic and flexible platform to test control strategies involving multi-body coordination. The structure of the codebase with clear separation between RobotController, PartController, and SRRSsensorsNode enables straightforward substitution of alternative algorithms. Furthermore, because all messages are standard ROS 2 types, the same system could be deployed on physical robots with appropriate hardware. The self-replication process would then move from simulation to reality, enabling experimental validation of robotic “growth” behaviors.

7 Evaluation and validation

In this chapter we will verify approaches described in Chapter 5 and perform evaluation of their computation time.

To verify approaches and validate the results of reachability and configuration checking and to demonstrate the feasibility of the counterexample-guided control approach, we complement the formal analysis with the simulation. We will use the simulation platform described in Chapter 6. By comparing the outcomes of formal verification with simulation results, we aim to assess both the soundness and the practical utility of model-checking-based methods for modular self-replicating robots.

7.1 Validation of the counterexample based robot control

To validate the counterexample based approach, introduced in chapter 5.3.4 we will integrate the counterexample generation pipeline into ROS2-Gazebo simulation environment. We choose robot blocks configuration described in Chapter 6.3, robot initial position and target point with coordinates $[50, 50, 10]$. These initial parameters reflected in the lines 1-3 of the Listing 7.1 lines.

Listing 7.1: Function goto_XYZ() of RobotController class

```
1 angle_vars = ["block1.yaw", "block2.pitch", "block3.pitch", "block4.
    pitch", "block5.yaw"]
2 sequence = ["base", "yaw", "pitch", "pitch", "pitch", "yaw"]
3 lengths = [10, 20, 30, 30, 10, 20]
4 robot = Robot(sequence, lengths)
5 generator = RobotNusmvGenerator(robot, output_smv_path="./smv/")
6 generator.set_target_point(50, 50, 10)
7 generator.generate_from_template("project_robot")
8 result = Counterexample.run_nusmv_file("./smv/robot.smv")
9 counterexample = Counterexample(result, len(sequence))
10 angles = counterexample.get_angles(angle_vars)
11 counterexample.plot_trace(-1, show_plot=True)
```

Trace derived from the counterexample consist out of 9 steps and shown in Figure 7.1

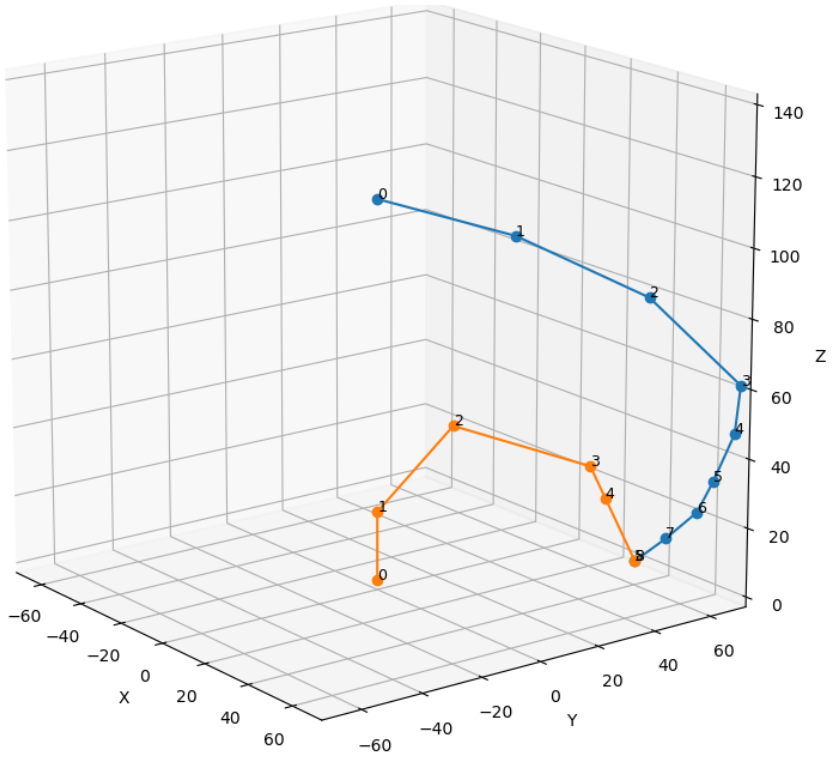

Figure 7.1: Mean time of model checking with number of blocks in robot

Resulting joint angles derived from the trace presented as a list steps, where each step is a list of control angles for each joint:

$$[0, -10, -10, -10, -10]$$

$$[0, -20, -20, -20, -20]$$

$$[0, -30, -30, -30, -30]$$

$$[0, -30, -40, -40, -40]$$

$$[0, -30, -50, -50, -50]$$

$$[0, -30, -60, -50, -60]$$

$$[0, -30, -70, -50, -60]$$

$$[0, -30, -80, -50, -60]$$

These angles were published as a sequence of control messages for the robot's joints using the rotate_joints() function implemented in the RobotController class, which is described in Section 6.3. This function ensures that each joint of the simulated robot receives a command corresponding to the angles obtained from the verification results, thereby reproducing in simulation the trajectory.

The result of this is shown in Figure 7.2, which illustrates the robot successfully performing the movement sequence derived from the counterexample.

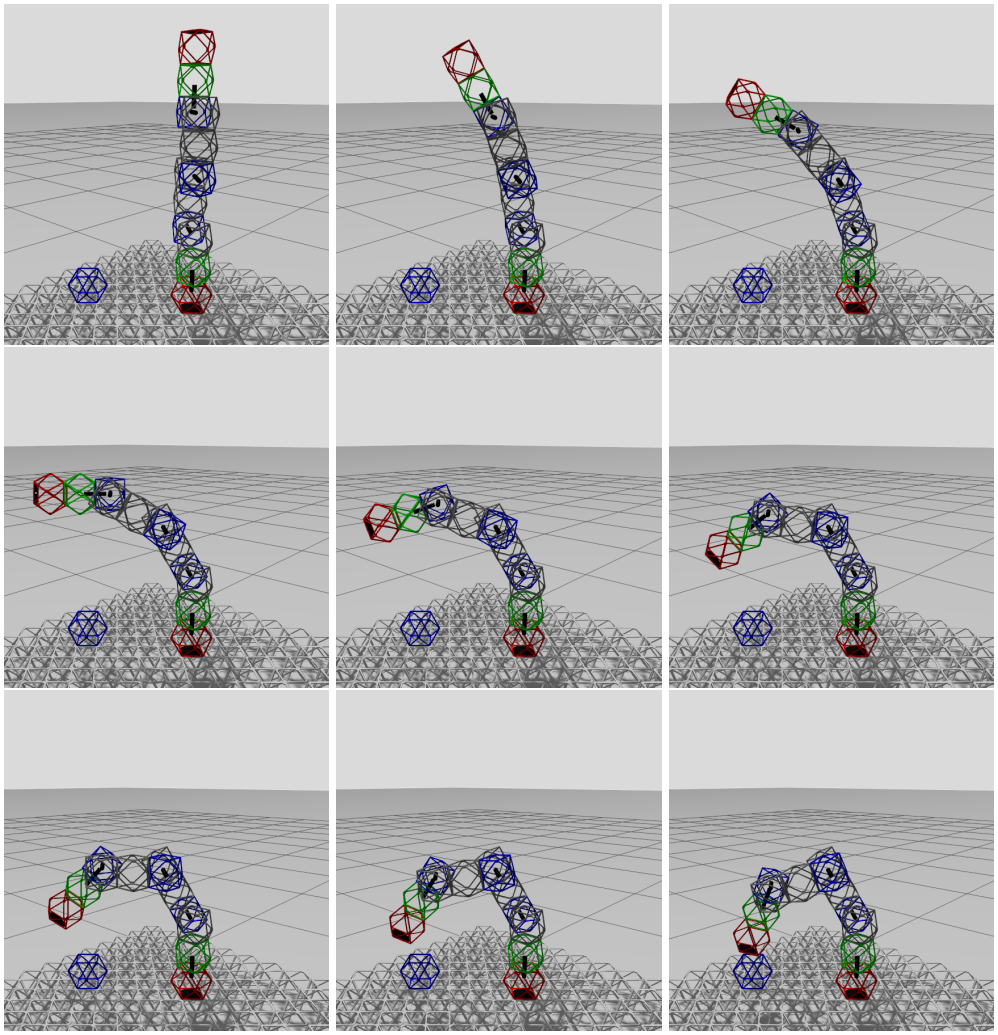

Figure 7.2: Simulation steps to achive point [5,5,1] from counterexample

Before launching the simulation in Gazebo, it was necessary to align the coordinate systems between the counterexample data and the simulation environment. The coordinate system orientation used in the NuSMV model differs from that of Gazebo.

Therefore, a preliminary transformation step was applied to adjust the rotation of the robot and the target. This alignment was critical for correctly mapping the counterexample trajectory to the simulated robot's physical motion. Once this transformation was established, the robot was able to execute the motion plan as described by the verified specification.

For the target point considered in this experiment, the resulting distance between the robot's end effector and the target position after motion completion was approximately 0.16 simulation units. This residual distance represents the cumulative error introduced by several factors.

Total discrepancy between the target point and the center of the end effector consists of the angle discretization error in NuSMV and the geometric offset of the blocks in the simulation model. Maximal discrepancy during simulation of ten different reachable points was 27% of length unit in the simulation (size of the voxel).

The simulation process also revealed several challenges and limitations in the current implementation of the counterexample- based control mechanism. The main observed issues are summarized below:

1. **Non-orthogonality of the end effector during mechanical attachment.**
   During attachment actions, the end effector did not remain orthogonal to the side surfaces of the parts being assembled. This misalignment reduce the effectiveness of physical attachment or cause mechanical stress on the joints.
2. **Potential self-intersection of robot segments during movement.**
   While following the motion trace generated in NuSMV, the robot occasionally approached poses that would lead to self-intersection in the physical model. This issue arises because our model used in NuSMV lacks explicit spatial collision constraints.
3. **Positional error due to angle discretization.**
   The angular variables in the NuSMV model were represented with a discrete resolution (e.g., in 10° or smaller increments). This quantization introduces rounding errors, which accumulate along multi-joint kinematic chains and lead to noticeable deviations in end-effector positioning.
4. **Geometric inaccuracy due to simplified structural modeling.**
   The NuSMV model employs a simplified geometric abstraction of the robot structure, which does not fully capture the precise spatial offsets between certain block types. Particularly, in our model blocks with a rotational axis, offset along the longitudinal direction.

To overcome these limitations and improve the accuracy of counterexample- based control, several refinements and future system enhancements are proposed:

1. The NuSMV model can be extended with additional constraints that enforce orthogonal alignment of the terminal blocks of the robot relative to the surfaces of the parts being attached. This adjustment will ensure that the resulting trajectories maintain proper contact geometry during assembly.
2. By incorporating geometric consistency checks or collision-avoidance predicates into the model generation process, it is possible to prevent self-intersecting configurations from being produced during verification. Such constraints could be encoded as temporal logic formulas that prohibit conflicting joint combinations.
3. The angular discretization can be progressively refined in an iterative process. The model-checking trace from a coarse discretization can be used as a baseline, followed by repeated verification steps with smaller angle increments. This refinement strategy will balance computational complexity and accuracy.
4. The robot physical structure in the NuSMV representation should be further refined to reflect the exact spatial arrangement of the blocks, including any offsets or rotations in the mechanical joints. This would reduce systematic discrepancies between the verified and simulated trajectories.

In summary, the integration of counterexample-based motion generation with Gazebo simulation demonstrates the feasibility of using formal verification outputs to control physical or simulated self-replicating robots. Although the current implementation still exhibits measurable deviations due to model simplifications and discretization, the results confirm that verified behaviors can be meaningfully reproduced in a physical simulation environment. Continued refinement of the model fidelity and constraint formulation will further improve the correspondence between the symbolic verification domain and the physical reality of robotic assembly.

7.2 Validation of the robot configuration checking

In order to prove consistence of configurations checking workflow described in Chapter 5.5 we will first compare model checking results for both sequential checking with python and aggregated checking with NuSMV. And then we will simulate one of reachable configurations in Gazebo, using simulation platform described in Chapter 6 to validate the results and prove that the target point is reachable for chosen configuration.

7.2.1 Comparison of individual and aggregated model checking

We will perform sequential configuration model checking for robots which consists of block of types: "yaw", "pitch", for target point $[20, 20, 20]$ and target region:

$x = \{-10, 10\}, y = \{-10, 10\}, z = \{0, 10\}$

We will use `configuration_checking` class, described in Chapter 5.5.2. and the working script which is located in `configuration_checking/ validation_individual_conf` script.

For robot consisting of 3 blocks using individual checking we have no reachable configuration, the list of all possible configurations are given in Table 7.1.

N	Configration	Point	Time,sec	Region	Time,sec
1	base_pitch_pitch	not reachable	0.0276	not reachable	0.0117
2	base_pitch_yaw	not reachable	0.0118	not reachable	0.0144
3	base_yaw_pitch	not reachable	0.0162	not reachable	0.0232
4	base_yaw_yaw	not reachable	0.0101	not reachable	0.0131

Table 7.1: Configurations reachability checking for robot size 3 for target point and target region.

For the aggregated model checking with NuSMV model adjusted for robot length 3 we also had no reachable configurations, and the computation time was 0.047455 sec. This result corresponds to individual checking outcome.

For robot consisting of 4 blocks, using individual checking we get five reachable configuration, the list of all possible configurations are given in Table 7.2.

N	Configration	Point	Time,sec	Region	Time,sec
1	base_pitch_pitch_pitch	reachable	0.1265	not reachable	0.3181
2	base_pitch_pitch_yaw	reachable	0.0358	not reachable	0.0402
3	base_pitch_yaw_pitch	reachable	0.1876	not reachable	0.2137
4	base_pitch_yaw_yaw	not reachable	0.0123	not reachable	0.0164
5	base_yaw_pitch_pitch	reachable	0.1074	not reachable	0.1253
6	base_yaw_pitch_yaw	reachable	0.0240	not reachable	0.0235
7	base_yaw_yaw_pitch	not reachable	0.0624	not reachable	0.0654
8	base_yaw_yaw_yaw	not reachable	0.0112	not reachable	0.0171

Table 7.2: Configurations reachability checking for robot size 4 for target point and target region.

And for the aggregated model checking with NuSMV model adjusted for robot length=4 we will have a following result for the reachability of point $[20, 20, 20]$:

['base', 'pitch', 'pitch', 'yaw'] Time: 0.699504 sec

This result corresponds to the robot configuration number 2 in Table 7.2 for individual checking.

Results for model checking of robots consisting of 5 blocks, shown in Table 7.3.

N	Configration	Point	Time,sec	Region	Time,sec
1	base_pitch_pitch_pitch_pitch	reachable	1.6375	not reachable	7.0634
2	base_pitch_pitch_pitch_yaw	reachable	0.2747	not reachable	0.3352
3	base_pitch_pitch_yaw_pitch	reachable	2.0740	not reachable	3.4262
4	base_pitch_pitch_yaw_yaw	reachable	0.0443	not reachable	0.0442
5	base_pitch_yaw_pitch_pitch	reachable	2.1243	reachable	8.6275
6	base_pitch_yaw_pitch_yaw	not reachable	0.2090	not reachable	0.2357
7	base_pitch_yaw_yaw_pitch	reachable	1.0638	not reachable	1.1828
8	base_pitch_yaw_yaw_yaw	not reachable	0.0126	not reachable	0.0183
9	base_yaw_pitch_pitch_pitch	reachable	0.9875	reachable	1.3594
10	base_yaw_pitch_pitch_yaw	reachable	0.2242	reachable	0.2629
11	base_yaw_pitch_yaw_pitch	reachable	2.1728	reachable	4.3590
12	base_yaw_pitch_yaw_yaw	not reachable	0.0224	not reachable	0.0462
13	base_yaw_yaw_pitch_pitch	not reachable	0.4550	not reachable	0.5033
14	base_yaw_yaw_pitch_yaw	not reachable	0.0656	not reachable	0.0753
15	base_yaw_yaw_yaw_pitch	not reachable	0.1228	not reachable	0.1382
16	base_yaw_yaw_yaw_yaw	not reachable	0.0117	not reachable	0.0177

Table 7.3: Configurations reachability checking for robot size 5 for target point and target region.

For the aggregated model checking with NuSMV model adjusted for robot length 5 we will have a following result for the reachability of target point:

['base', 'pitch', 'yaw', 'pitch', 'pitch']

Calculation time was 22.020454 sec

All aggregated model checking results corresponds to the individual checking, that proves the consistency of the developed approach.

The important observation is that in configurations where the first block after base is pitch block, the configuration is equivalent to a base-yaw-pitch..., as if we would have an additional yaw block after base, this property defined by base block structure, and reflects the arbitrary initial rotation of a robot.

7.2.2 Simulation of eligible configurations

From the list of reachable configurations presented in Table 7.3, one configuration was selected for validation using the simulation platform. The chosen configuration, highlighted in green in the table, corresponds to the following sequence of blocks: `[base, yaw, pitch, yaw, pitch]`. A robot with this structure is capable of reaching the specified target point according to the model-checking results.

To implement this configuration in the simulation, a dedicated xacro file was created, and located in `/urdf/robot_yaw_pitch_yaw_pitch.xacro`, in [Bab25]. The primary modification in this file concerns the block sequence of the robot, as shown in Listing 7.2. This modification ensures that the simulated robot matches the verified structural configuration used in NuSMV.

```
<xacro:voxel name="block0" origin="0 0 0" color="${red}"/>
<xacro:block_ver block_num="1" par_num="0"  x="0" y="0" z="0" />
<xacro:block_fix block_num="2" par_num="1"  x="0" y="0" z="1" />
<xacro:block_hor block_num="3" par_num="2"  x="0" y="1" z="1" />
<xacro:block_ver block_num="4" par_num="3"  x="0" y="0" z="-0.5" />
<xacro:block_fix block_num="6" par_num="4"  x="0" y="0" z="1" />
<xacro:block_hor block_num="7" par_num="6"  x="0" y="-1" z="1" />
<xacro:block_fix block_num="8" par_num="7"  x="0" y="0" z="0.5" />
<xacro:block_end block_num="11" par_num="8" x="0" y="0" z="0.5" />
```

Listing 7.2: Function goto_XYZ() of RobotController class

A counterexample trace for this configuration and the corresponding target points was obtained using the script `counterexample_yaw_pitch_yaw_pitch_robot.py` available in [Bab25]. The resulting joint angles extracted from the counterexample are: `[10, -70, -80, -70]`. These values represent the rotation commands for each joint in degrees and were later applied in the simulation to reproduce the verified trajectory.

For the purpose of simulation and validation, a dedicated ROS 2 launch file named `robot_config_validation.launch.py` was developed. This file initializes the Gazebo environment, loads the robot description, and executes the counterexample-based control sequence to move the robot toward the target point.

The results of the simulation are presented in Figure 7.3. The figure illustrates the initial configuration of the robot and its final pose after performing the movement sequence derived from the counterexample trace. As can be observed, the robot successfully reached the target point, confirming the feasibility of the configuration identified through model checking.

In summary, this section demonstrates that the outcomes of both individual and aggregated verification of NuSMV models for all robot configurations up to a length of five

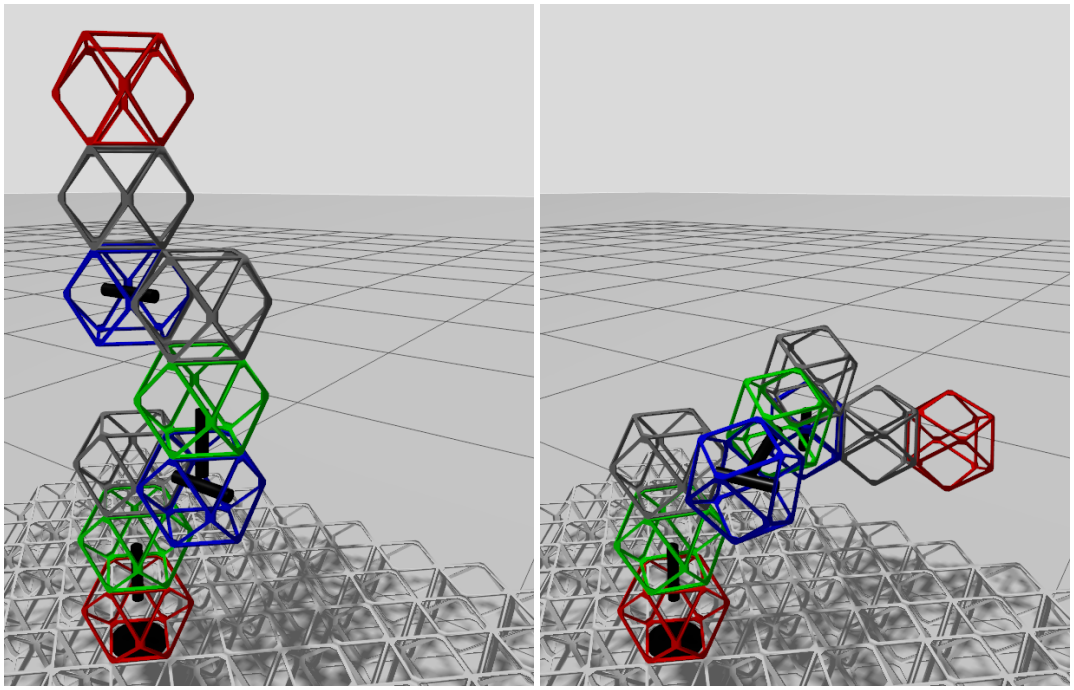

Figure 7.3: New configuration of a robot and the pose to reach target point

blocks are consistent with each other. Furthermore, the provided simulation example validates one of the verified configurations, confirming that the model-checking results accurately correspond to physically realizable robot movements within the simulation environment.

7.2.3 Comparison of computational time for model checking

To analyze the computational complexity of configuration checking and to compare the aggregated and sequential verification approaches, a series of experiments were conducted. In the first part of the experiment, a single target point with coordinates $x = 20$, $y = 20$, $z = 20$ was used for reachability verification. The results of these experiments were presented in Tables 7.1–7.3.

In the second part, the total model-checking time was evaluated for a target region defined by the coordinate ranges $x \in \{-10, 10\}$, $y \in \{-10, 10\}$, and $z \in \{0, 10\}$. The variation of the total verification time for point reachability with respect to the number of blocks in the robot is shown in Figure 7.4.

Similarly, the dependence of the total verification time for region reachability on the robot size is illustrated in Figure 7.5.

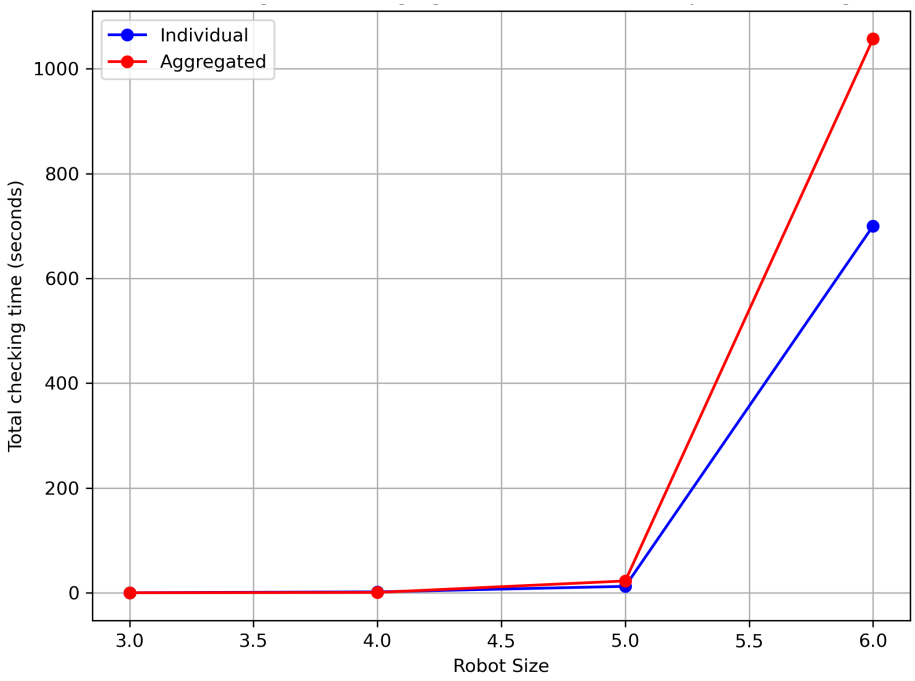

Figure 7.4: Total time of model checking with number of blocks in robot for target point.

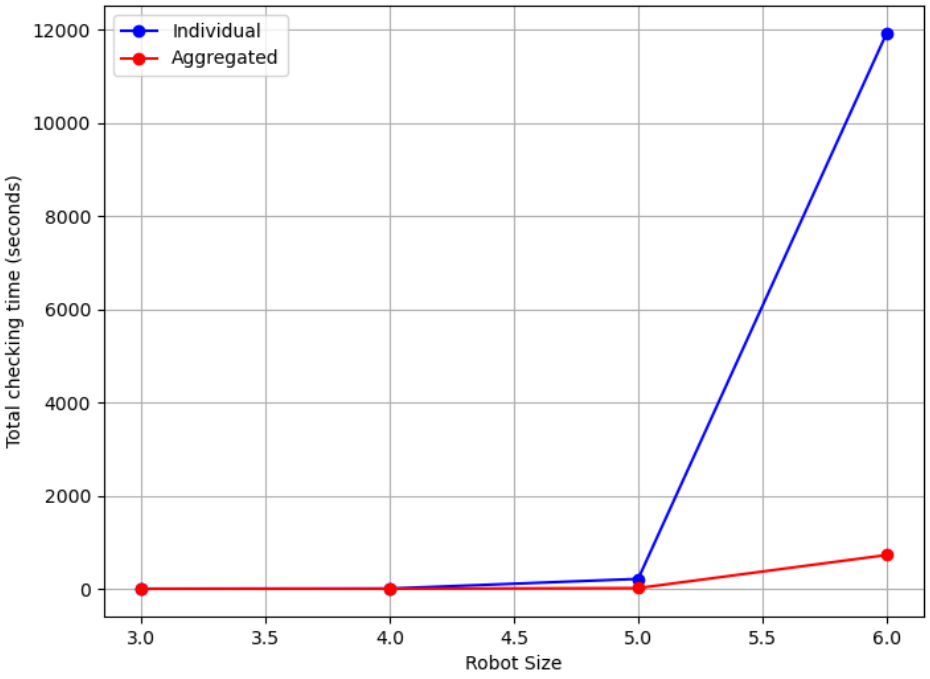

Figure 7.5: Total time of model checking with number of blocks in robot for target region.

In both graphs, the model checking time grows exponentially with robot size. This is a typical behavior for model checking problems, as the state space explosion becomes dominant when system complexity increases. For smaller robot sizes (3–4 modules), the total checking time is nearly negligible in both point and region cases. However, as the robot size reaches 5 and especially 6 modules, the computation time rises in the point-target case up to approximately 700 seconds, and in the region-target case exceeding around 2 hours for the largest configuration.

The experiments reveal that the sequential and aggregated approaches to configuration checking in NuSMV exhibit distinct advantages that determine their suitability for different verification tasks.

The sequential checking method systematically explores each configuration. Its advantage lies in its completeness: it provides a comprehensive list of all reachable configurations, ensuring that no valid state is omitted. This nature makes the sequential approach particularly valuable for analysis, where the generated configurations can be further examined using simulation to assess structural properties, other criteria. However, this completeness comes at a computational cost. Since each configuration is verified independently, the overall process tends to be slower than the aggregated approach, especially as the number of configurations increases.

In contrast, the aggregated configuration strategy check multiple configurations in single verification model. This integrated formulation enables NuSMV to handle the problem more efficiently, resulting in faster verification times in most cases. However, the aggregated model typically returns only the first reachable configuration that satisfies the given specification. Discovering additional configurations requires iteratively negating previously found solutions and re-running the verification process, adds procedural complexity. On the other hand, this approach supports fine-grained control over the structure of configurations, as it allows the inclusion of constraints on specific blocks within the sequence through type-defining variables in NuSMV. This feature makes aggregated checking well-suited for analysis on design stage, where constraints play a central role.

The choice between these two methods should therefore depend on the verification objective: whether the goal is to obtain a full configuration space for detailed analysis or to achieve rapid validation of specific structural constraints within a large and complex model.

7.3 Evaluation of 2D reachability checking computation time

We performed a number of verification experiments using the workflow described in Chapter 5.4 to validate the model and to analyze the robot's behavior in various conditions.

We ensured that the automatic generation itself was correct by comparing an automatically generated model (for a simple scenario) to the hand-written initial model. They matched in structure, and running both yielded the same outcome.

Using a simple Python script, we measured the time required for model generation, model checking, and counterexample processing under different scenario sizes. We were particularly interested in how the computation grows with field size and with robot size.

Field Size

We kept the robot's sequence length fixed and increased the grid size (4x4, 5x5, 6x6, and 7x7) to see how the state space explosion affects runtime. The results, shown in Figure 7.6, indicated that the verification time grows quickly with the area of the grid.

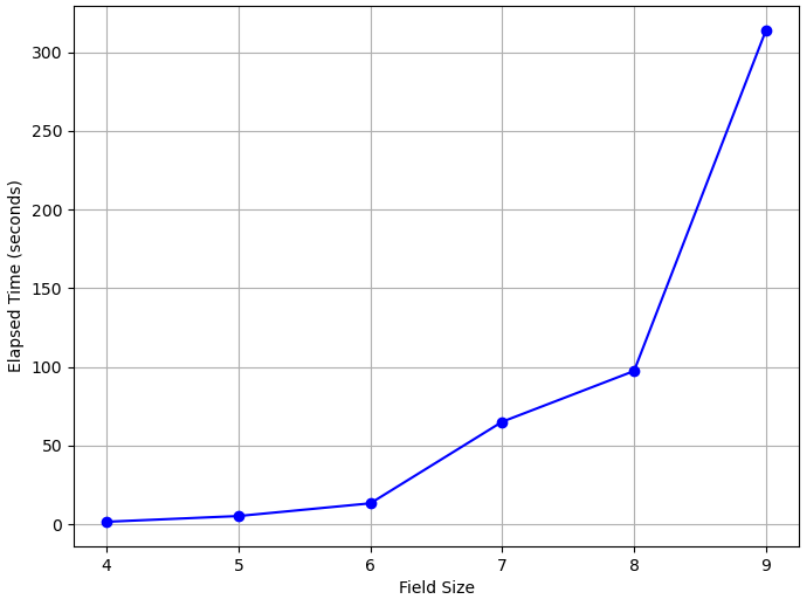

Figure 7.6: Model checking time with field size

A larger field means more possible positions and more possible paths for the robot, which dramatically increases the state space that the model checker must explore. We found that going beyond a 7x7 grid with a 4-part robot made NuSmv execution considerably slower in our model. The relationship was not strictly exponential in field size for our tested range, but it was steep—significantly higher runtime for each increment in field dimension.

Robot Length

We then fixed a field size to 6 and varied the number of parts in the robot's sequence length. The computation times for model generation, verification, and trace processing as a function of robot size are plotted in Figure 7.7.

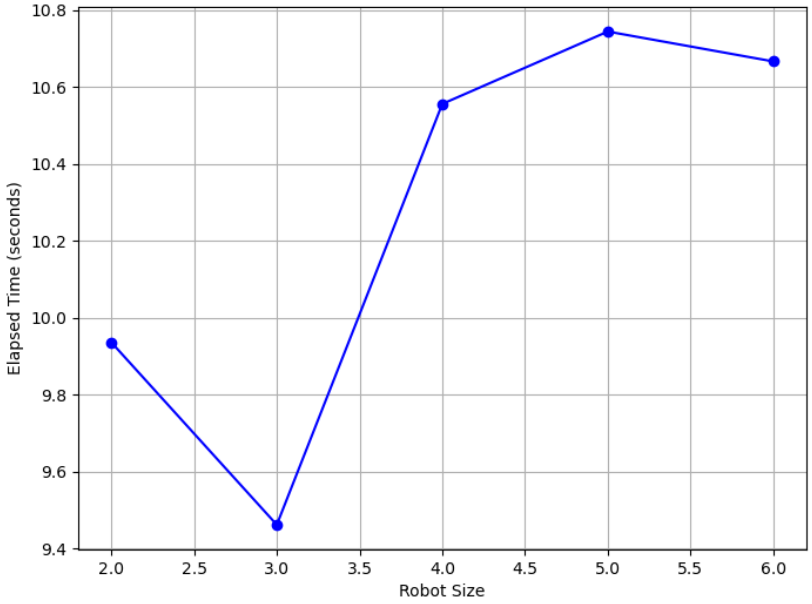

Figure 7.7: Model checking time with robot size

As expected, longer sequences also increase the state space, because the assembly_index has a larger range and more parts need to be found in order. This tends to increase the length of counterexample traces. With a 6x6 field, going from a robot requiring 3 parts to one requiring 5 parts made the verification slower, but the impact was much less pronounced than during increasing the field size. This suggests that spatial complexity (field area) was the more limiting factor in our model than the temporal length of the task.

The performance data was collected on a standard laptop with Intel Core i7 2.4 GHz, running Ubuntu 24.04. We recorded the times for model generation, which were negligible for these sizes, on the order of fractions of a second up to a couple of seconds for the largest cases, model checking, which dominated the time, ranging from under a second for very small cases to many seconds or minutes for the larger ones, and counterexample processing, also very fast, since parsing a text trace is trivial compared to the model checking.

One qualitative observation from all our experiments is that the counterexample produced by NuSmv was often the shortest possible path to reach the goal or failure. That is, the model checker tends to find an optimal or near-optimal solution in terms of number of steps. For example, if there was a more direct route for the robot to gather parts, the first counterexample usually reflected that direct route rather than a circuitous one. In our tests, for all evaluated configurations up to length-4 robots in 5x5 grids (and many in 6x6), the first counterexample found corresponded to a strategy where the robot made efficient moves, not visiting irrelevant areas. This suggests that, at least for these sizes, our approach could be leveraging the model checker as not just a verifier but also a planner that yields shortest paths. However, we did not formally guarantee optimality, and it is possible for a model checker to output a non-shortest counterexample in some cases. A more rigorous analysis could be done to confirm path optimality or lack thereof.

In summary, the verification experiments confirmed that the NuSMV model correctly captures the reachability of both success and failure conditions in a self-replicating robot scenario. The automated workflow significantly reduced the effort to test different scenarios, and the results provided confidence in the model Figures 7.6 and 7.7 provide a high-level view of how scaling the problem size impacts the computational cost, which is important when considering more complex or larger-scale applications of this verification approach.

7.4 Threats to validity

In our work, the integration of formal verification through NuSMV, simulation in ROS2 and Gazebo, and the conceptual modeling of self-replicating robotic system introduces several sources of uncertainty and possible threats to validity. The purpose of this section is to critically evaluate these threats and to define their implications. Understanding these potential weaknesses is essential not only for interpreting the results but also for guiding future work aimed at reducing their impact.

Three main sources of validity threats were identified in the developed system:

1. Limited generalization of the developed system — the applicability of the proposed verification methods is confined to a specific class of SRRS systems.
2. Discretization errors and computational trade-offs — the accuracy and performance of the model are constrained by the discretization parameter.
3. Inconsistencies between NuSMV models, simulation environments, and real-world – abstractions and simplifications lead to mismatches between modeled and physical behaviors.

Limited generalization of the developed system

The approaches and implementations presented in Chapter 5 and 6 were specifically designed and validated on modular, self-replicating robotic systems with relative robot (robots using relative motion between their parts and their environment to move and build structures). These systems are characterized by discrete structural blocks, that can be mechanically connected, detached, and rearranged to form new robots. The model checking framework assumes a relative coordinate representation, where all positional and kinematic relations are expressed relative to the robot's local reference frame.

While this abstraction enabled a compact and computationally tractable formal model, it limits external validity. In particular, the proposed verification methods may not directly generalize to non-modular SRRS architectures, such as continuous-body or deformable robots, self-replicating swarm systems, where replication emerges from collective behavior or hybrid robotic systems, where the replication process involves chemical or additive-manufacturing operations.

For such systems, the assumptions about structure, connectivity, and discrete transitions used in this work would not necessarily hold. Additionally, the temporal logic specifications we used depend on the ability to enumerate discrete states. SRRS with other type of structure would therefore require reformulation of the model semantics.

Therefore, the findings of this work should be interpreted as representative of a specific class of SRRS rather than as a universal model checking methodology for all self-replicating robots.

Discretization errors

The second threat to validity arises from the discretization parameter employed during model generation. In formal verification of motion-related properties, the continuous workspace of the robot must be discretized into a finite set of states to make model checking computationally feasible. In our system, each step of discretization defines a quantization of angular and linear space, determining how finely the robot's configuration space is represented.

This choice introduces the accuracy–efficiency contradiction:

A coarse discretization with a large step size reduces the number of states, leading to faster model checking, but at the expense of precision. A fine discretization with a small step size increases model fidelity but causes exponential growth in the number of states, leading to extremely long verification times.

These discretization-related errors directly threaten the validity, as the model's discrete representation might fail to capture the intended continuous behaviors.

This problem can be mitigated by performing multi-resolution analysis, where verification is performed at multiple levels of discretization and results are compared for consistency. Using abstraction refinement techniques, in which coarse verification results guide selective refinement of critical regions in the state space.

Future research should therefore formalize the relationship between discretization, verification precision, and computational time for SRRS, to develop flexible model parameterization.

Inconsistencies between NuSMV model, simulation, and reality

Another threat to validity stems from inconsistencies between the used NuSMV model, the ROS2–Gazebo simulation, and the real-world physical implementation of the robot. While the verification model captures the logical and structural relations among robot components, certain geometric and kinematic simplifications were made to keep the model tractable.

A concrete example arises in the representation of blocks of type block_pitch, where in the simulation and real robot design an axis of rotation is shifted along the rotation axis relative to the connected block. This shift introduces a small translational displacement during rotation— an effect that is physically significant for precise assembly but was omitted in the NuSMV model for simplicity. As a result, the formal model assumes perfectly aligned rotational joints, whereas the physical robot exhibits a small offset.

The potential consequence is that a configuration deemed reachable in the formal model might not be physically realizable due to geometric interference or misalignment.

To mitigate this threat we used an alternating shift direction in the simulation, and in future this and similar aspects can be incorporated into the NuSMV model.

Performing cross-validation between model checking and simulation, as partially implemented in this thesis in Chapters 7.1 7.2, allows detecting inconsistencies by replaying verified sequences in the simulation environment.

Summary

The three identified sources of validity threats represent the main factors that may constrain the implementation to other sysems and interpretation of the results presented in this thesis. None of these threats invalidate the fundamental conclusions of the research, but they define the operational boundaries within which those conclusions remain valid.

From a methodological perspective, these threats highlight the necessity of maintaining a multi-layered validation pipeline, where formal verification, simulation-based testing, and physical experimentation complement one another. Model checking provides exhaustive logical guarantees within its abstract domain, while simulation validates these results under physical dynamics, and real-world testing confirms them under environmental uncertainty. By addressing these aspects, subsequent studies can move toward fully validated, scalable frameworks for self-replicating robotic systems.

8 Conclusion

This work investigated the application of automated model checking and to the domain of self-replicating robotic systems. Self-replicating systems, by their nature, involve dynamic configuration changes, high structural and behavioral complexity. Traditional analysis, simulation or real-based experiments often fails to guarantee the correctness of such systems due to the large number of states with logical errors or contradictions including interactions between robot, materials and environment. The work presented here aimed to fill this gap by employing NuSMV symbolic model checker, to rigorously verify selected properties of SRRS under varying structural and operational conditions.

The thesis demonstrated how automated model checking can be applied to verify three aspects of SRRS behavior. Each aspect corresponds to a different level of system abstraction – local, structural, and global, illustrating the generality of the approach.

The first application, described in Chapter 5.3, focused on verifying point and region reachability during the local assembly process of modular robots. By encoding the robot actuation and attachment logic in NuSMV and defining reachability goals as temporal logic specifications, the system could determine whether a particular configuration or spatial position was achievable. When a specification was violated, NuSMV generated a counterexample trace, which then can be used to guide movement. This approach provided not only verification but also a foundation for reactive control based on model-checking feedback.

The second verification task addressed the global dynamics of the self -replication process by modeling the distribution of material parts in a 2D workspace was demonstrated in Chapter 5.4. Here, temporal logic was used to verify whether the system preserved liveness – the ability to eventually complete replication given finite resources. The model checking results confirmed that certain distribution conditions led to deadlocks, while others guaranteed successful replication cycles.

The third aspect of SRRS model checking shown in Chapter 5.5 and concerned the feasibility of robot configurations. Using the same modeling framework, the approach evaluated multiple block sequences to determine which configurations were valid and functionally reachable within the system mechanical and environmental constraints.

This enabled an automated selection of eligible robot designs – a task traditionally requiring extensive simulation or manual inspection. The results showed that the NuSMV checker could systematically reject infeasible

One of the purposes of the research centered on developing a systematic methodology to represent SRRS behavior and constraints in a form suitable for temporal-logic analysis. To achieve this, in Chapter 5.5.2 we introduced a template-based NuSMV model generation approach, implemented through Python scripts that automatically translate high-level robot specifications into NuSMV code.

The approach used parameterized templates to encode repeated components for robot blocks, and to define their logical interactions. This enabled scalable generation of models representing different robot structures without manual redefinition. By combining this template-based method with developed temporal-logic specifications for reachability and liveness, demonstrated in Chapter 5.3.3, it became possible to evaluate entire families of designs under specified verification criteria.

Furthermore, in Chapter 6 of this paper we integrated ROS2 and Gazebo simulations to validate the results of model checking. Counterexamples and verified traces were interpreted as executable motion sequences and simulated using robot, parts and environment, developed based on the design principles described in [Jen+19].

This verification loop – from formal model to physical simulation – ensured that verified properties held not only in abstract state space but also in realistic dynamic conditions. This methodological framework provides a reproducible foundation for integrating formal verification into the development workflow of modular and self-replicating robots.

Model checking suffers from the well-known state explosion problem, which becomes bigger in modular robot systems with multiple blocks and many degrees of freedom. Experiments in Chapter 7.2.3 showed that the state space grows exponentially with each added component, but proposed aggregated and sequential configuration checking approach is feasible for moderately complex systems, even using personal computer.

Future work in this area can focus on several directions aimed at enhancing both the theoretical framework and practical deployment of model-checked SRRS systems. Extending the verification framework to encompass different kinds of modular robots, material parts properties, and environmental conditions. This will allow comparative studies of alternative self-replication paradigms. Developing higher-level controllers capable of using counterexamples from model checking to dynamically guide assembly or repair actions, enabling self-correcting replication behaviors. Incorporating adaptive verification directly into the replication cycle, allowing the robot to modify its

configuration or verification goals in response to environmental feedback or detected errors. Extending the current single-system framework to collective SRRS, where multiple robots cooperate in material collection and assembly. Formal methods will be applied to verify coordination and resource-sharing properties within such distributed systems.

In conclusion, this work makes both scientific and engineering contributions to the field of autonomous manufacturing and self-replicating robotics. Scientifically, it provides a novel framework for integrating formal verification into the analysis of self-replicating processes. Technically, it delivers a working ROS2–Gazebo implementation of a modular self- replicating robot whose behavior can be verified, tested, and reproduced. The developed methods and codebase – from NuSMV model generation scripts to simulation orchestration—constitute a reproducible experimental platform for future research in self-replication, modular robotics, and formal verification. This integration of model checking and robotic simulation paves the way toward reliable, verifiably correct systems, capable not only of constructing themselves but also of proving the correctness of their own actions. The combination of model checking, simulation validation, and adaptive control proposed in this thesis represents a step toward closing the gap between theoretical models of self-replication and their practical realization in Self-replicating robotic systems.

Bibliography

[Pen58] L. S. Penrose. “Mechanics of Self-Reproduction”. In: *Annals of Human Genetics* 23.1 (Nov. 1958), pp. 59–72. DOI: 10.1111/j.1469-1809.1958.tb01442.x (cit. on p. 17).

[NB62] John von Neumann and Arthur W. Burks. *Theory of Self-Reproducing Automata*. Edited and completed by Arthur W. Burks. Urbana, IL: University of Illinois Press, 1962 (cit. on p. 19).

[San80] NASA Santa Clara Space Initiative. *Advanced Automation for Space Missions*. Space Initiative/XRI Santa Clara, California: NASA Conference Publication 2255, NASA Ames Research Center Moffett Field, California, 1980 (cit. on p. 1).

[Azi+96] Adnan Aziz, Kumud Sanwal, Vigyan Singhal, and Robert Brayton. “Verifying Continuous Time Markov Chains”. In: *Proceedings of the 8th International Conference on Computer Aided Verification (CAV)*. Ed. by Rajeev Alur and Thomas A. Henzinger. Vol. 1102. Lecture Notes in Computer Science. New Brunswick, NJ: Springer, 1996, pp. 269–276 (cit. on p. 9).

[Sip98] Moshe Sipper. “Fifty Years of Research on Self-Replication: An Overview”. In: *Artificial Life* 4.3 (1998), pp. 237–257. DOI: 10.1162/106454698568576 (cit. on p. 13).

[SZC02] Jackrit Suthakorn, Yu Zhou, and Gregory S. Chirikjian. “Self-Replicating Robots for Space Utilization”. In: (2002) (cit. on pp. 1, 17).

[YS02] Håkan L. S. Younes and Reid G. Simmons. “Probabilistic Verification of Discrete-Event Systems Using Acceptance Sampling”. In: *Computer Aided Verification (CAV ’02)*. Vol. 2404. Lecture Notes in Computer Science. Copenhagen, Denmark: Springer, 2002, pp. 223–236. DOI: 10.1007/3-540-45657-0_19 (cit. on p. 9).

[SKC03] Jackrit Suthakorn, Yong T. Kwon, and Gregory S. Chirikjian. “A Semi-Autonomous Replicating Robotic System”. In: *Proceedings of the 2003 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*. IEEE International Symposium on Computational Intelligence in Robotics and Automation, Vol. 2. Kobe, Japan: IEEE, July 2003, pp. 776–781. DOI: 10.1109/CIRA.2003.1222279 (cit. on p. 18).

[HR04] Michael Huth and Mark Ryan. *Logic in computer science Modelling and Reasoning about Systems*. 2nd. Cambridge, New York, Melbourne, Madrid, Cape Town, Singapore, São Paulo: Cambridge University Press, 2004 (cit. on pp. 3–7, 9).

[Zyk+05] Victor Zykov, Efstathios Mytilinaios, Bryant Adams, and Hod Lipson. "Self-reproducing machines". In: *Nature* 435.7038 (2005). Brief Communication, p. 163. DOI: 10.1038/435163a (cit. on pp. 25, 26).

[You06] Håkan L. S. Younes. "Statistical Probabilistic Model Checking with a Focus on Time-Bounded Properties". In: *Information and Computation* 204.9 (Sept. 2006), pp. 1368–1409. DOI: 10.1016/j.ic.2006.03.002 (cit. on p. 9).

[EML+07] Steven Eno, Lauren Mace, Jianyi Liu, et al. "Robotic Self-Replication in a Structured Environment without Computer Control". In: *Proceedings of the 2007 IEEE International Conference on Robotics and Automation (ICRA)*. 2007 (cit. on p. 20).

[LC07] Kiju Lee and Gregory S. Chirikjian. "Robotic Self-Replication: A Descriptive Framework and a Physical Demonstration from Low-Complexity Parts". In: *IEEE Robotics & Automation Magazine* 14.4 (2007), pp. 34–43. DOI: 10.1109/MRA.2007.910 (cit. on pp. 13, 14, 17, 20–25, 40).

[KNP11] Marta Kwiatkowska, Gethin Norman, and David Parker. "PRISM 4.0: Verification of Probabilistic Real-Time Systems". In: *Computer Aided Verification (CAV 2011)*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 585–591. DOI: 10.1007/978-3-642-22110-1_47 (cit. on p. 10).

[HSZ14] Timothy L. Hinrichs, A. Prasad Sistla, and Lenore D. Zuck. "Model Check What You Can, Runtime Verify the Rest". In: *HOWARD-60: A Festschrift on the Occasion of Howard Barringer's 60th Birthday*. Ed. by Andrei Voronkov and Margarita V. Korovina. Vol. 42. EPiC Series in Computing. EasyChair, 2014, pp. 234–244. DOI: 10.29007/slnn. URL: https://easychair.org/publications/paper/tq7 (cit. on p. 10).

[DGL16] Stéphane Demri, Valentin Goranko, and Martin Lange. *Temporal Logics in Computer Science Finite-State Systems*. 2nd. University Printing House, Cambridge CB2 8BS, United Kingdom: Cambridge University Press, 2016 (cit. on pp. 7, 8).

[LST16] Axel Legay, Zacharias R. M. Sedwards, and Louis-Marie Traonouez. "Plasma Lab: A Modular Statistical Model Checking Platform". In: *ISoLA 2016 – Leveraging Applications of Formal Methods, Verification and Validation*. Ed. by Tiziana Margaria and Bernhard Steffen. Vol. 9952. Lecture Notes in

Computer Science. Springer, 2016, pp. 77–93. DOI: 10.1007/978-3-319-47166-2_6 (cit. on p. 10).

[Deh+17] Christian Dehnert, Sebastian Junges, Joost-Pieter Katoen, and Matthias Volk. "A Storm is Coming: A Modern Probabilistic Model Checker". In: *Computer Aided Verification (CAV 2017)*. Ed. by Isil Dillig and Jens Palsberg. Vol. 10426. Lecture Notes in Computer Science. Springer, 2017, pp. 592–600. DOI: 10.1007/978-3-319-63390-9_31 (cit. on p. 10).

[Jen+19] Benjamin Jenett, Amira Abdel-Rahman, Kenneth Cheung, and Neil Gershenfeld. "Material–Robot System for Assembly of Discrete Cellular Structures". In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 4019–4026. DOI: 10.1109/LRA.2019.2930486 (cit. on pp. 27, 37, 77, 113).

[Tan+20] Ning Tan, Abdullah Aamir Hayat, Mohan Rajesh Elara, and Kristin L. Wood. "A Framework for Taxonomy and Evaluation of Self-Reconfigurable Robotic Systems". In: *IEEE Access* 8 (2020), pp. 13969–13986. DOI: 10.1109/ACCESS.2020.2965327 (cit. on p. 13).

[JS21] Andrew Jones and Jeremy Straub. "Simulation and Analysis of Self-Replicating Robot Decision-Making Systems". In: *Computers* 10.1 (2021), p. 9. DOI: 10.3390/computers10010009 (cit. on pp. 15, 16).

[Jon21] Andrew Burkhard Jones. "Decision-Making for Self-Replicating 3D Printed Robot Systems". Ph.D. Dissertation. North Dakota State University, 2021 (cit. on p. 14).

[Abd+22] Amira Abdel-Rahman, Christopher Cameron, Benjamin Jenett, Miana Smith, and Neil Gershenfeld. "Self-replicating hierarchical modular robotic swarms". In: *Communications Engineering* 1 (2022), p. 35. DOI: 10.1038/s44172-022-00034-3 (cit. on pp. 27, 28, 37).

[Chi22] Gregory S. Chirikjian. "Entropy, Symmetry, and the Difficulty of Self-Replication". In: *arXiv preprint arXiv:2202.02938* (2022) (cit. on pp. 13, 21).

[Sla+24] J. Tanner Slagel, Lauren M. White, Aaron Dutle, César A. Muñoz, and Nicolas Crespo. "A Formal Verification Framework for Runtime Assurance". In: *Proceedings of NASA Formal Methods (NFM 2024)*. Lecture Notes in Computer Science. Mountain View, CA, USA: Springer, June 2024, pp. 322–328. DOI: 10.1007/978-3-031-60698-4_19 (cit. on p. 12).

[Bab25] Dmitry Babaytsev. *Repository of a thesis*. Accessed: 2025-10-08. 2025. URL: https://github.com/Llao11/Model-checking-for-Self-Replicating-robotic-systems (cit. on pp. 77, 102).

[Rob25a] Open Robotics. *Gazebo Harmonic Documentation*. Accessed: 2025-10-08. 2025. URL: https://gazebosim.org/docs/harmonic/getstarted/ (cit. on p. 77).

[Rob25b] Open Robotics. *ROS 2 Documentation Jazzy Jalisco*. Accessed: 2025-10-08. 2025. URL: https://docs.ros.org/en/jazzy/index.html (cit. on p. 77).

A Code Listings

A.1 NuSMV models

A.1.1 robot_structure.smv

Trigonometric modules

```
MODULE sin(angle)
DEFINE
    val:= case
    angle = 0  :   0;   angle = 1  :  17;   angle = 2  :  34;
    angle = 3  : 50;  angle = 4  :  64;   angle = 5  :  77;
    angle = 6  :  87;   angle = 7  : 94;  angle = 8  :  98;
    angle = 9  : 100;   angle = 10 :  98;   angle = 11 : 94;
    angle = 12 :  87;   angle = 13 :  77;   angle = 14 :  64;
    angle = 15 : 50;  angle = 16 :  34;   angle = 17 :  17;
    angle = 18 :   0;   angle = 19 :-17;  angle = 20 :-34;
    angle = 21 :-50;  angle = 22 :-64;  angle = 23 :-77;
    angle = 24 :-87;  angle = 25 :-94;  angle = 26 :-98;
    angle = 27 :-100;   angle = 28 :-98;  angle = 29 :-94;
    angle = 30 :-87;  angle = 31 :-77;  angle = 32 :-64;
    angle = 33 :-50;  angle = 34 :-34;  angle = 35 :-17;
    TRUE : 0;
    esac;

MODULE cos(angle)
DEFINE
    val:= case
    angle = 0  : 100;   angle = 1  :  98;   angle = 2  :  94;
    angle = 3  : 87;  angle = 4  :  77;   angle = 5  :  64;
    angle = 6  :  50;   angle = 7  : 34;  angle = 8  :  17;
    angle = 9  :   0;   angle = 10 :-17;  angle = 11 :-34;
    angle = 12 :-50;  angle = 13 :-64;  angle = 14 :-77;
    angle = 15 :-87;  angle = 16 :-94;  angle = 17 :-98;
    angle = 18 :-100;   angle = 19 :-98;  angle = 20 :-94;
    angle = 21 :-87;  angle = 22 :-77;  angle = 23 :-64;
    angle = 24 :-50;  angle = 25 :-34;  angle = 26 :-17;
    angle = 27 :   0;   angle = 28 :  17;   angle = 29 :  34;
    angle = 30 :  50;   angle = 31 : 64;  angle = 32 :  77;
    angle = 33 :  87;   angle = 34 :  94;   angle = 35 : 98;
    TRUE : 0;
```

block_base module

```
MODULE block_base(px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz, L)
DEFINE
  xhx2 := xhx;
  xhy2 := xhx;
  xhz2 := xhz;

  yhx2 := yhx;
  yhy2 := yhx;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px;
  py2 := py;
  pz2 := pz;
```

block_yaw module

```
MODULE block_yaw(px, py, pz,
                 xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
                 L, initYaw)
VAR
  yaw : 0..35;
  cy : cos(yaw);
  sy : sin(yaw);

DEFINE
  SCALE := 100;

  cos_yaw := cy.val;    -- [-100..100]
  sin_yaw := sy.val;

  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
  xhz2 := xhz;

  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
  yhz2 := yhz;

  zhx2 := zhx;
  zhy2 := zhy;
  zhz2 := zhz;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;


ASSIGN
  init(yaw) := initYaw;
  next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

block_pitch module

```
MODULE block_pitch(px, py, pz,
                   xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
```

```
                    L, initPitch)
VAR
  pitch : 0..35;

  cp : cos(pitch);
  sp : sin(pitch);

DEFINE
  SCALE := 100;

  cos_p := cp.val;    -- [-100..100]
  sin_p := sp.val;

  xhx2 := (xhx * cos_p + zhx * sin_p) / SCALE;
  xhy2 := (xhy * cos_p + zhy * sin_p) / SCALE;
  xhz2 := (xhz * cos_p + zhz * sin_p) / SCALE;

  yhx2 := yhx;  yhy2 := yhy;  yhz2 := yhz;

  zhx2 := (-xhx * sin_p + zhx * cos_p) / SCALE;
  zhy2 := (-xhy * sin_p + zhy * cos_p) / SCALE;
  zhz2 := (-xhz * sin_p + zhz * cos_p) / SCALE;

  px2 := px + (zhx2 * L) / SCALE;
  py2 := py + (zhy2 * L) / SCALE;
  pz2 := pz + (zhz2 * L) / SCALE;

ASSIGN
  init(pitch) := initPitch;
  next(pitch) := { pitch, (pitch+1) mod 36, (pitch+35) mod 36 };
```

main module

```
MODULE main

FROZENVAR

    checkX : -10..10;
    checkY : -10..10;
    checkZ : 0..10;
VAR
    block0 : block_base(base_px, base_py, base_pz,
                        base_xhx, base_xhy, base_xhz,
                        base_yhx, base_yhy, base_yhz,
                        base_zhx, base_zhy, base_zhz,
                        L1);
    block1 : block_yaw(block0.px, block0.py, block0.pz,
                        block0.xhx, block0.xhy, block0.xhz,
                        block0.yhx, block0.yhy, block0.yhz,
                        block0.zhx, block0.zhy, block0.zhz,
                        L1, yaw1 );
    block2 : block_pitch(block1.px2, block1.py2, block1.pz2,
                       block1.xhx2, block1.xhy2, block1.xhz2,
                       block1.yhx2, block1.yhy2, block1.yhz2,
                       block1.zhx2, block1.zhy2, block1.zhz2,
                       L2, pitch2);
    block3 : block_pitch(block2.px2, block2.py2, block2.pz2,
                       block2.xhx2, block2.xhy2, block2.xhz2,
                       block2.yhx2, block2.yhy2, block2.yhz2,
                       block2.zhx2, block2.zhy2, block2.zhz2,
```

```
                      L3, pitch3);
    block4 : block_yaw(block3.px, block3.py, block3.pz,
                       block3.xhx, block3.xhy, block3.xhz,
                       block3.yhx, block3.yhy, block3.yhz,
                       block3.zhx, block3.zhy, block3.zhz,
                       L4, yaw4 );
    block5 : block_pitch(block4.px2, block4.py2, block4.pz2,
                      block4.xhx2, block4.xhy2, block4.xhz2,
                      block4.yhx2, block4.yhy2, block4.yhz2,
                      block4.zhx2, block4.zhy2, block4.zhz2,
                      L5, pitch5);
DEFINE
    L1 := 10;
    yaw1    := 0;
    L2 := 10;
    pitch2  := 0;
    L3 := 10;
    pitch3  := 0;
    L4 := 10;
    yaw4    := 0;
    L5 := 10;
    pitch5  := 0;

    base_px := 0;       base_py := 0;       base_pz := 0;

    base_xhx := 100;    base_xhy := 0;      base_xhz := 0;
    base_yhx := 0;      base_yhy := 100;    base_yhz := 0;
    base_zhx := 0;      base_zhy := 0;      base_zhz := 100;

    endX := block3.px2;
    endY := block3.py2;
    endZ := block3.pz2;

    error := 1; -- +-1 (error=2) endX and endY coordinates error while checking
        reachability
    endX_inside_limits := (endX <= (checkX + error) & endX >= (checkX - error));
    endY_inside_limits := (endY <= (checkY + error) & endY >= (checkY - error));
    endZ_inside_limits := (endZ <= (checkZ + error) & endZ >= (checkZ - error));

CTLSPEC
      EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits ) --&
          blocks_above_serface)
```

A.1.2 Assemble_initial_template.smv

```
MODULE main
  DEFINE
    field_max_index := {field_max_index};
    robot_size := {robot_size};
    initial_grid := {initial_grid}
  VAR
    grid : array 0..field_max_index of array 0..field_max_index of {object_types};
    checked_cells : array 0..field_max_index of array 0..field_max_index of {
        visit_cells_types};
    x : 0..field_max_index; -- horizontal axis
    y : 0..field_max_index; -- vertical axis
    robot_state : {robot_states};
    partX : 0..field_max_index;
    partY : 0..field_max_index;
    assembly_index : 0..robot_size;
```

```
15    sequence : array 0..robot_size of {robot_part_types};
16  ASSIGN
17    init(robot_state) := MOVING;      -- start in MOVING mode
18    init(x) := 0; init(y) := 0;          -- start at position (0,0) (assembly site for
          example)
19    init(assembly_index) := 0;           -- no parts FINISHED initially
20    init(grid) := initial_grid;
21    {init_robot_sequence}
22    next(robot_state) := case
23      (robot_state = MOVING &
24      (assembly_index <= robot_size) &
25      ((x = 0                  | grid[x - 1][y] != NONE ) &
26        (x = field_max_index   | grid[x + 1][y] != NONE) &
27        (y = 0                 | grid[x][y - 1] != NONE ) &
28        (y = field_max_index   | grid[x][y + 1] != NONE ))
29      ) : FAILED;
30      (robot_state = MOVING &
31       ((x > 0                & grid[x - 1][y] != NONE) |
32        (x < field_max_index  & grid[x + 1][y] != NONE) |
33        (y > 0                & grid[x][y - 1] != NONE) |
34        (y < field_max_index  & grid[x][y + 1] != NONE))
35      ) : CHECKING;
36      (robot_state = CHECKING &
37      assembly_index <= robot_size
38      ) : case
39            grid[partX][partY]  = sequence[assembly_index] : ASSEMBLING; -- if part is
                needed type, go to ASSEMBLING (to pick it up, transport to assemble
                site and assemble)
40            grid[partX][partY] != sequence[assembly_index] : MOVING; -- if part is not
                needed, go back to MOVING (continue looking)
41          esac;
42      (robot_state = ASSEMBLING
43      ) : case
44            assembly_index < robot_size : MOVING;
45            assembly_index = robot_size : FINISHED;
46          esac;
47      robot_state = FINISHED  : FINISHED;
48      robot_state = FAILED    : FAILED;
49      TRUE : robot_state;  -- default: no change (should not happen under normal
          conditions)
50    esac;
51    next(assembly_index) := case
52      (robot_state = ASSEMBLING &
53      (assembly_index < robot_size)
54      ): assembly_index + 1;   -- delivered a part, count it
55      TRUE : assembly_index;
56    esac;
57    next(partX) := case
58      (
59        robot_state = MOVING & next(robot_state) = CHECKING &
60        x > 0 & grid[x - 1][y] = sequence[assembly_index]
61      ) : x - 1;  -- found in left neighbour cell
62      (
63        robot_state = MOVING & next(robot_state) = CHECKING &
64        x < field_max_index & grid[x + 1][y] = sequence[assembly_index]
65      ) : x + 1;  -- found in right neighbour cell
66      TRUE : x;  -- otherwise, retain previous (or irrelevant outside checking context)
67    esac;
68    next(partY) := case
69      (
70        robot_state = MOVING & next(robot_state) = CHECKING &
71        y > 0 & grid[x][y - 1] = sequence[assembly_index]
```

```
72        ) : y - 1;  -- found in upper neighbour cell
73        (
74          robot_state = MOVING & next(robot_state) = CHECKING &
75          y < field_max_index & grid[x][y + 1] = sequence[assembly_index]
76        ) : y + 1;  -- found in down neighbour cell
77        TRUE : y;
78      esac;
79    TRANS
80      (next(robot_state) = MOVING ) -> (
81                                        (
82                                          next(x) = x - 1 & x > 0
83                                          & next(y) = y
84                                          & grid[x - 1][y] = NONE
85                                          & checked_cells[x - 1][y] != VISITED
86                                        ) |
87                                       (
88                                         next(x) = x + 1 & x < field_max_index
89                                         & next(y) = y
90                                         & grid[x + 1][y] = NONE
91                                         & checked_cells[x + 1][y] != VISITED
92                                       )|
93                                       (next(y) = y - 1 & y > 0
94                                         & next(x) = x
95                                         & grid[x][y - 1] = NONE
96                                         & checked_cells[x][y - 1] != VISITED
97                                       ) |
98                                       (
99                                         next(y) = y + 1 & y < field_max_index
100                                        & next(x) = x
101                                        & grid[x][y + 1] = NONE
102                                        & checked_cells[x][y + 1] != VISITED
103                                      )
104                                      );
105   TRANS
106   (next(robot_state) != MOVING) -> ((next(x) = x) & (next(y) = y));
107   TRANS   (next(grid) = grid);
108   TRANS   (next(sequence) = sequence);
109   TRANS   (robot_state != ASSEMBLING) -> (next(assembly_index) = assembly_index);
110
111 LTLSPEC
112   G F (robot_state != FINISHED);
```

A.2 Python scripts

A.2.1 Counterexample.py class

```python
import subprocess
import matplotlib.pyplot as plt


class Counterexample:
    """Class to parse nusmv counterexample, retreive trace states,
    filter variables and represent traces as graphs"""

    def __init__(self, nusmv_result: str):
        self.nusmv_result = nusmv_result
        self.robot_len = 5

    def plot_lines(self, ax, points):
        """
        Plot a list of 3D points connected with lines and save the figure.
        """
        xs, ys, zs = zip(*points)
        ax.plot(xs, ys, zs, marker="o", linestyle="-")
        for i, (x, y, z) in enumerate(points):
            ax.text(x, y, z, f"{i}", fontsize=9, color="black")

    def plot_trace(self, robot_step=0, save_path="3d_plot.png"):
        """Plot a end-effector trace from counterexample."""
        end_coordinates = ["endX", "endY", "endZ"]
        trace = self.filter_variables(end_coordinates)
        fig = plt.figure()
        ax = fig.add_subplot(111, projection="3d")
        ax.set_xlim(-20, 20)  # X axis range
        ax.set_ylim(-20, 20)  # Y axis range
        ax.set_zlim(0, 40)  # Z axis range
        ax.set_box_aspect([1, 1, 1])
        self.plot_lines(ax, trace)
        robot_pos0 = self.get_robot_blocks(self.robot_len)[robot_step]
        self.plot_lines(ax, robot_pos0)
        ax.set_xlabel("X")
        ax.set_ylabel("Y")
        ax.set_zlabel("Z")
        plt.savefig(save_path, dpi=300)
        plt.show()
        plt.close(fig)  # Close the figure to free memory
        print(f"3D plot saved to {save_path}")

    def filter_variables(self, vars: list[str]) -> list[list[int]] | None:
        """returns the list of steps, each step is a list of states with defined
            variable"""
        all_steps = [vars]
        states_list = self.nusmv_result.split("-> State")
        states_list = states_list[1:]
        previous_step_values = vars[:]
        for state in states_list:
            step_values = previous_step_values
            for line in state.split(sep="\n"):
                for i, var in enumerate(vars):
                    if var in line:
                        value = line.split("=")[1]
```

```
                        step_values[i] = value
            all_steps.append(step_values)
            previous_step_values = step_values[:]
        try:
            all_steps = [[int(variable) for variable in step] for step in all_steps
                [1:]]
            return all_steps
        except ValueError:
            print(
                "Error while str -> int transformation, check if all elements are int:"
            )
            return None

    def get_robot_blocks(self, robot_size):
        """Return robot blocks coordinates for each step."""
        blocks = []
        steps = []
        # get changin by blocks by steps [[changing block0], [changing block1] ...]
        for i in range(robot_size):
            blocks_coordinates = [
                f"block{i}.px2",
                f"block{i}.py2",
                f"block{i}.pz2",
            ]
            block_steps = self.filter_variables(blocks_coordinates)
            blocks.append(block_steps)
        # transpose the matrix (blocks to steps)
        for i in range(len(blocks[0])):
            step = []
            for j in range(len(blocks)):
                step.append(blocks[j][i])
            steps.append(step)
        print("Robot blocks by steps:")
        self.print_steps(steps)
        return steps

    def get_angles(
        self, angle_vars=[".yaw", ".pitch"], angle_descritization=10
    ) -> list[list[int]] | None:
        """returns the list of states with angles in degrees for each step"""
        all_steps = self.filter_variables(angle_vars)
        all_steps = [
            [angle * angle_descritization for angle in step] for step in all_steps
        ]
        return all_steps

    def print_steps(self, all_steps):
        """Print list"""
        for i in all_steps:
            print(i)

    def get_target_coordinates(
        self, target_coordinates=["checkX", "checkY", "checkZ"]
    ) -> list[list[int]] | None:
        """returns target coordinates values"""
        return self.filter_variables(target_coordinates)

    def get_end_coordinates(
        self, end_coordinates=["endX", "endY", "endZ"]
    ) -> list[list[int]] | None:
        """list of end-effector coordinates by steps"""
        return self.filter_variables(end_coordinates)
```

```
116
117     @staticmethod
118     def run_nusmv_file(smv_file="./smv/robot_structure.smv"):
119         """runs smv file and if there is a counterexample store the result in file"""
120         nusmv_path = ".././NuSMV-2.7.0-linux64/bin/NuSMV"
121         file = smv_file
122         output_file = "result.txt"
123         result = subprocess.run(
124             [
125                 nusmv_path,
126                 "-dynamic",
127                 file,
128             ],
129             capture_output=True,
130             text=True,
131         )
132         stdout = result.stdout
133         if "is true" in result.stdout:
134             print("The LTL property holds.")
135         else:
136             with open(output_file, "w") as result_file:
137                 print("Generating counterexample..")
138                 result_file.write(stdout)
```

A.3 ROS2 nodes

A.3.1 RobotController class

```python
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray, Empty
from sensor_msgs.msg import JointState
from typing import List
import threading
import math


class RobotController(Node):
    def __init__(self, sensorNode):
        super().__init__("robot_controller")
        self.sensorNode = sensorNode
        self.create_publishers()
        self.create_subscribers()
        self.fixed_end = 1
        self.joint_diff_threshold = 2  # [degrees]
        self.joint_angles_current = [0, 0, 0, 0, 0]

    def create_publishers(self):
        """
        publishers for attach and detach topics: robot to base, voxel to robot ends
        """
        self.attach_publisher1 = self.create_publisher(Empty, "/attach_end1", 10)
        self.detach_publisher1 = self.create_publisher(Empty, "/detach_end1", 10)
        self.attach_publisher2 = self.create_publisher(Empty, "/attach_end2", 10)
        self.detach_publisher2 = self.create_publisher(Empty, "/detach_end2", 10)
        self.command_publisher1 = self.create_publisher(
            Float64MultiArray, "/position_controller1/commands", 10
        )
        self.command_publisher2 = self.create_publisher(
            Float64MultiArray, "/position_controller2/commands", 10
        )
        self.command_publisher3 = self.create_publisher(
            Float64MultiArray, "/position_controller3/commands", 10
        )
        self.command_publisher4 = self.create_publisher(
            Float64MultiArray, "/position_controller4/commands", 10
        )
        self.command_publisher5 = self.create_publisher(
            Float64MultiArray, "/position_controller5/commands", 10
        )
        self.command_publishers = [
            self.command_publisher1,
            self.command_publisher2,
            self.command_publisher3,
            self.command_publisher4,
            self.command_publisher5,
        ]

    def create_subscribers(self):
        self.joints_angles_subscriber = self.create_subscription(
            JointState, "/joint_states", self.joint_state_changed, 10
        )

    def goto_XYZ(self, x, y, z, step_size=3):
```

```
        """
        High level function to go closer and move free end to the target coordinates x,
            y,z
        """
        if x == "\n":
            x = 0
        if y == "\n":
            y = 0
        if z == "\n":
            z = 0
        x = float(x)
        y = float(y)
        z = float(z)
        if abs(x) > step_size or abs(y) > step_size:
            # going to a far located point
            if x > step_size:
                stepX = step_size
                x = x - step_size
            elif x < -step_size:
                stepX = -step_size
                x = x + step_size
            else:
                stepX = 0
            if y > step_size:
                stepY = step_size
                y = y - step_size
            elif y < -step_size:
                stepY = -step_size
                y = y + step_size
            else:
                stepY = 0
            self.get_logger().info(f"\nX: {x}\n Y:{y}\n Z:{z}")
            # Step algorithm:
            self.goto_XYZ(stepX, stepY, 1)
            self.goto_XYZ(stepX, stepY, 0)
            self.swap_fix_block()  # change base
            self.goto_XYZ(x, y, z, step_size)
        # near pose calculation
        else:
            command_sequences = self.calculate_joint_angles(x, y, z)
            self.rotate_joints(command_sequences)

    def calculate_joint_angles(self, x, y, z) -> List[float]:
        command_sequences = []
        alpha, beta_0, gamma = self.relative_angles(x, y, z)
        if z < 4:
            joint4 = alpha + gamma
        elif z >= 4 and z <= 6:
            joint4 = alpha + gamma - 90
        else:
            raise Exception("z is more than 6")
        joint1 = beta_0
        joint2 = alpha - gamma
        joint3 = 180 - 2 * alpha
        joint5 = beta_0
        if self.fixed_end == 1:
            joint1 = joint1
            joint5 = joint5
            command_sequences = [joint1, joint2, joint3, joint4, joint5]
        elif self.fixed_end == 2:
            joint1 = joint1 + 180
            joint5 = joint5 + 180
```

```
            command_sequences = [joint5, joint4, joint3, joint2, joint1]
        else:
            self.get_logger().info("ERROR: End blocks not fixed")
        return command_sequences

    def relative_angles(self, x, y, z):
        """Calculate angles:
        alpha - pitch/2 of block2 without gamma
        beta - yaw of block1
        gamma - correction of alpha
        """
        beta = math.degrees(math.atan2(y, x))
        if z < 4:
            r = math.sqrt(x * x + y * y)
            r_0 = math.sqrt(abs(x * x + y * y - 1))
        elif z >= 4 and z <= 6:
            z = z - 2
            r = math.sqrt(x * x + y * y)
            r_0 = math.sqrt(abs(x * x + y * y - 1)) - 2

        self.get_logger().info(f"{r=}\n{r_0=}")
        if r != 0:
            beta_0 = beta - math.degrees(math.asin(1 / r))
        else:
            beta_0 = 0
        alpha = math.degrees(math.asin((math.sqrt(r_0 * r_0 + z * z)) / 4))
        gamma = math.degrees(math.atan2(z, abs(r_0)))
        return alpha, beta_0, gamma

    def rotate_joints(self, command_sequences):
        for joint_index in range(len(command_sequences)):
            self.rotate_joint(joint_index, float(command_sequences[joint_index]))
        self.wait_movement_finish(command_sequences)

    def rotate_joint(self, joint_index, angle):
        command = Float64MultiArray()
        # Degrees to Radians
        try:
            command.data = [float(angle) * math.pi / 180.0]
            self.command_publishers[joint_index].publish(command)
        except:
            self.get_logger().info(
                f"Error: No data for joint {joint_index}: {command.data}"
            )

    def wait_movement_finish(self, joints_angles_target: list[float]):
        """Wait until movement finished joint target angles in degrees"""
        self.current_target_angles = joints_angles_target
        move_finished = threading.Event()
        timer_rate = 0.1
        max_waiting_time = 5.0

        def isAchieved() -> bool:
            joints_angles_target_deg = [float(a) for a in self.current_target_angles]
            joint_angles_current_deg = [
                float(angle) * 180.0 / math.pi for angle in self.joint_angles_current
            ]
            diff = [
                abs(a - b)
                for a, b in zip(joints_angles_target_deg, joint_angles_current_deg)
            ]
            if max(diff) < self.joint_diff_threshold:
```

```
                move_finished.set()
                timer.cancel()  # or: self.destroy_timer(timer)
                return True
            else:
                return False

        timer = self.create_timer(timer_rate, isAchieved)
        move_finished.wait(timeout=max_waiting_time)
        if isAchieved():
            self.get_logger().info(f"Target achived")
        else:
            self.get_logger().info(f"ERROR: target point not achived \n")
        timer.cancel()  # or: self.destroy_timer(timer)

    def joint_state_changed(self, msg):
        joint_angles_current_dict = dict(zip(msg.name, msg.position))
        sorted_names = ["rev0_1", "rev2_3", "rev5_6", "rev8_9", "rev9_10"]
        self.joint_angles_current = [joint_angles_current_dict[i] for i in sorted_names
            ]

    def get_joint_angle(self, joint_num):
        joint = float(self.joint_angles_current[joint_num]) * 180.0 / math.pi
        return joint

    def get_fixed_end(self):
        return self.fixed_end

    def swap_fix_block(self):
        if self.get_fixed_end() == 1:
            self.fix_end2_base()
        elif self.get_fixed_end() == 2:
            self.fix_end1_base()

    def fix_end1_base(self):
        msg = Empty()
        self.attach_publisher1.publish(msg)
        self.get_logger().info("Published attach1 message.")
        self.fixed_end = 1
        self.free_end2_base()
        self.get_logger().info(f"fixed_end=1")

    def fix_end2_base(self):
        msg = Empty()
        self.attach_publisher2.publish(msg)
        self.get_logger().info("Published attach2 message.")
        self.fixed_end = 2
        self.free_end1_base()
        self.get_logger().info(f"fixed_end=2")

    def free_end1_base(self):
        msg = Empty()
        self.detach_publisher1.publish(msg)
        self.get_logger().info("Published detach1 message.")

    def free_end2_base(self):
        msg = Empty()
        self.detach_publisher2.publish(msg)
        self.get_logger().info("Published detach2 message.")
```

A.3.2 PartController class

```python
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray, Empty

import subprocess
import signal


class PartController(Node):
    def __init__(self, number) -> None:
        super().__init__("part_controller")
        self.part_number = number
        self.create_publishers()
        self.create_bridge()
        self.empty_msg = Empty()
        self.detach_part_end1()  # send detach end-effectors from parts
        self.detach_part_end2()  # send detach end-effectors from parts

    def create_publishers(self):
        """
        Create publishers to send commands to the part
        """
        num = self.part_number
        # attach/detach PART to  END-EFFECTORS 1 and 2
        self.end1_attach_publisher = self.create_publisher(
            Empty, f"/attach_end1_part{num}", 10
        )
        self.end2_attach_publisher = self.create_publisher(
            Empty, f"/attach_end2_part{num}", 10
        )
        self.end1_detach_publisher = self.create_publisher(
            Empty, f"/detach_end1_part{num}", 10
        )
        self.end2_detach_publisher = self.create_publisher(
            Empty, f"/detach_end2_part{num}", 10
        )
        # attach/detach PART to BASE
        self.base_attach_publisher = self.create_publisher(
            Empty, f"/attach_part{num}_base", 10
        )
        self.base_detach_publisher = self.create_publisher(
            Empty, f"/detach_part{num}_base", 10
        )
        # joint controller publisher
        self.command_publisher = self.create_publisher(
            Float64MultiArray, f"/position_controller{num}/commands", 10
        )

    def attach_part_end1(self):
        """send command to a part to attach to end1"""
        self.end1_attach_publisher.publish(self.empty_msg)

    def detach_part_end1(self):
        """send command to a part to detach from end1"""
        self.end1_detach_publisher.publish(self.empty_msg)

    def attach_part_end2(self):
        """send command to a part to attach to end 2"""
        self.end2_attach_publisher.publish(self.empty_msg)

```

```
60      def detach_part_end2(self):
61          """send command to a part to detach from end2"""
62          self.end2_detach_publisher.publish(self.empty_msg)
63
64      def attach_part_base(self):
65          """send command to a part to attach to end 1"""
66          self.base_attach_publisher.publish(self.empty_msg)
67
68      def detach_part_base(self):
69          """send command to a part to attach to end 2"""
70          self.base_detach_publisher.publish(self.empty_msg)
71
72      def create_bridge(self):
73          """Create ROS-Gazebo bridges to send commands"""
74          num = self.part_number
75          BRIDGE_RULES = [
76              f"/attach_end1_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
77              f"/detach_end1_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
78              f"/attach_end2_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
79              f"/detach_end2_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
80              f"/attach_part{num}_base@std_msgs/msg/Empty@gz.msgs.Empty",
81              f"/detach_part{num}_base@std_msgs/msg/Empty@gz.msgs.Empty",
82              f"/model/part{num}/pose@geometry_msgs/msg/Pose@gz.msgs.Pose",
83          ]
84          argv = ["ros2", "run", "ros_gz_bridge", "parameter_bridge"]
85          argv.extend(BRIDGE_RULES)
86
87          # Optionally remap namespaces or add ROS args:
88          # argv += ["--ros-args", "-r", "__ns:=/sim"]
89
90          self.get_logger().info(f"Starting ros_gz_bridge: {' '.join(argv)}")
91          # Start detached but  manage its lifetime
92          self.proc = subprocess.Popen(
93              argv, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, bufsize
                    =1
94          )
95
96      def destroy_node(self):
97          """Destroy this Node with closing bridges"""
98          try:
99              if self.proc and self.proc.poll() is None:
100                 self.get_logger().info("Stopping ros_gz_bridge...")
101                 self.proc.send_signal(signal.SIGINT)
102                 try:
103                     self.proc.wait(timeout=5)
104                 except subprocess.TimeoutExpired:
105                     self.proc.kill()
106         finally:
107             super().destroy_node()
```

A.4 Gazebo sdf,urdf and xacro code

A.4.1 base.sdf

```
<?xml version="1.0"?>
<sdf version="1.10">
  <model name="base">
    <static>true</static>
      <link name="base_link">
      <pose>0 0 0 0 0 0</pose>
        <visual name="body_visual">
          <geometry>
            <mesh>
              <uri>/srrs_sim/meshes/base10x10.stl</uri>
              <scale> 0.001 0.001 0.001 </scale>
            </mesh>
          </geometry>
          <material>
            <ambient>0.8 0.8 0.8 1</ambient>
            <diffuse>0.8 0.8 0.8 1</diffuse>
          </material>
        </visual>
        <collision name="body_collision">
          <geometry>
            <box>
              <size>2 2 0.265 </size>
            </box>
          </geometry>
        </collision>
      </link>
      <joint name="fixed_to_world" type="fixed">
        <parent>world</parent>
        <child>base_link</child>
        <pose>0 0 0 0 0 0</pose>
      </joint>
    </model>
</sdf>
```

A.4.2 robot.xacro

```
<?xml version="1.0" ?>
<robot name="rel_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">
    <xacro:property name="green" value="0 0.8 0 1" />
    <xacro:property name="red" value="0.8 0 0 1" />
    <xacro:property name="blue" value="0 0 0.8 1" />
    <xacro:property name="yellow" value="1 1 0 1" />
    <xacro:property name="white" value="1 1 1 1" />
    <xacro:property name="cell_step" value="0.134"/>
    <xacro:property name="grey" value="0.2 0.2 0.2 1" />
    <xacro:property name="black" value="0 0 0 1" />

    <!-- MACROS -->

    <xacro:macro name="voxel" params="name origin color" >
        <link name="${name}">
            <visual>
                <geometry>
```

```
18                      <mesh filename="/home/lao/Documents/Masterarbeit/git/
                            SRRS_gazebo_sim/ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="
                            0.001 0.001 0.001"/>
19                  </geometry>
20                  <origin xyz="${origin}"/>
21              </visual>
22              <collision name="${name}_collision">
23                  <geometry>
24                      <mesh filename="/home/lao/Documents/Masterarbeit/git/
                            SRRS_gazebo_sim/ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="
                            0.001 0.001 0.001"/>
25                  </geometry>
26              </collision>
27              <inertial>
28                  <mass value="0.1"/>
29                  <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0" izz="1.0"/>
30              </inertial>
31          </link>
32          <gazebo reference="${name}">
33              <visual>
34                  <material>
35                      <ambient>${color}</ambient>
36                      <diffuse>${color}</diffuse>
37                  </material>
38              </visual>
39          </gazebo>
40      </xacro:macro>
41
42      <xacro:macro name="joint_fix" params="name parent child" >
43          <joint name="${name}" type="fixed">
44              <origin xyz="0 0 0"/>
45              <parent link="${parent}"/>
46              <child link="${child}"/>
47          </joint>
48      </xacro:macro>
49
50      <xacro:macro name="joint_end" params="name parent child origin axis rpy" >
51          <joint name="${name}" type="revolute">
52              <axis xyz="${axis}"/>
53              <origin xyz="${origin}"/>
54              <parent link="${parent}"/>
55              <child link="${child}"/>
56              <limit lower="0" upper="0" effort="1000" velocity="0.0"/>
57              <dynamics damping="0.1" friction="0.1"/>
58          </joint>
59      </xacro:macro>
60
61      <xacro:macro name="joint_revolution" params="name parent child origin axis rpy" >
62          <joint name="${name}" type="revolute">
63              <axis xyz="${axis}"/>
64              <origin xyz="${origin}"/>
65              <parent link="${parent}"/>
66              <child link="${child}"/>
67              <limit effort="50.0" lower="-6.29" upper="6.29" velocity="1"/>
68              <dynamics damping="0.1" friction="0.1"/>
69          </joint>
70          <link name="${name}visual">
71              <visual>
72                  <geometry>
73                      <cylinder radius="0.01" length="0.1"/>
74                  </geometry>
75                  <origin rpy="${rpy}" xyz="${origin}"/>
```

```
76              </visual>
77          </link>
78          <xacro:joint_fix name="${name}visualfixed" parent="${parent}" child="${name}
                visual"/>
79      </xacro:macro>
80
81      <!-- BLOCK MACKROS -->
82
83      <xacro:macro name="block_end" params="block_number parent_number x y z " >
84          <xacro:voxel name="block${block_number}" origin="${x*cell_step} ${y*cell_step}
                ${z*cell_step}" color="${red}"/>
85          <xacro:joint_end name="base${parent_number}_${block_number}" parent="block${
                parent_number}" child="block${block_number}" axis="0 0 1" origin="${x*
                cell_step} ${y*cell_step} ${(z+1)*cell_step}" rpy="0 0 0"/>
86      </xacro:macro>
87
88      <xacro:macro name="block_fix" params="block_number parent_number x y z " >
89          <xacro:voxel name="block${block_number}" origin="${x*cell_step} ${y*cell_step}
                ${z*cell_step}" color="${grey}"/>
90          <xacro:joint_fix name="fixed${parent_number}_${block_number}" parent="block${
                parent_number}" child="block${block_number}"/>
91      </xacro:macro>
92
93      <xacro:macro name="block_ver" params="block_number parent_number x y z " >
94          <xacro:voxel name="block${block_number}" origin="${0} ${0} ${0}" color="${green
                }"/>
95          <xacro:joint_revolution name="rev${parent_number}_${block_number}" parent="
                block${parent_number}" child="block${block_number}" axis="0 0 1" origin="
                ${x*cell_step} ${y*cell_step} ${(z+1)*cell_step}" rpy="0 0 0"/>
96      </xacro:macro>
97
98      <xacro:macro name="block_hor" params="block_number parent_number x y z " >
99          <xacro:joint_revolution name="rev${parent_number}_${block_number}" parent="
                block${parent_number}" child="block${block_number}" axis="0 1 0" origin="
                ${x*cell_step} ${y*cell_step} ${(z+0.5)*cell_step}" rpy="1.57 0 0"/>
100         <xacro:voxel name="block${block_number}" origin="${0} ${0} ${-0.5*cell_step}"
                color="${blue}"/>
101     </xacro:macro>
102
103
104     <!-- SENSORS MACROS -->
105
106     <xacro:macro name="contact_plate" params="name parent origin" >
107         <link name="${name}">
108             <visual>
109                 <geometry>
110                     <cylinder radius="0.05" length="0.002"/>
111                 </geometry>
112                 <!-- <origin xyz="${origin}"/> -->
113             </visual>
114             <collision name="${name}_collision">
115                 <geometry>
116                     <cylinder radius="0.05" length="0.002"/>
117                 </geometry>
118             </collision>
119         </link>
120
121         <joint name="${name}_joint" type="fixed">
122             <origin xyz="${origin}"/>
123             <parent link="${parent}"/>
124             <child link="${name}"/>
125         </joint>
```

```
126
127         <gazebo reference="${name}">
128             <sensor name="${name}_sensor" type="contact">
129                 <contact>
130                     <!-- If doesnt work - check contact collision name in SDF, transfer
                             xacro to urdf to sdf -->
131                     <collision>${parent}_fixed_joint_lump__${name}
                            _collision_collision_1</collision>
132                     <topic>/robot/${name}</topic>
133                 </contact>
134                 <always_on>true</always_on>
135                 <update_rate>50</update_rate>
136             </sensor>
137         </gazebo>
138
139     </xacro:macro>
140
141
142     <xacro:macro name="camera_sensor" params="name parent origin_xyz origin_rpy" >
143         <link name="${name}">
144             <visual name="${name}_visual">
145                 <geometry>
146                     <box size="0.01 0.01 0.01"/>
147                 </geometry>
148             </visual>
149         </link>
150
151         <joint name="${name}_joint" type="fixed">
152             <parent link="${parent}"/>
153             <child link="${name}"/>
154             <origin xyz="${origin_xyz}" rpy="${origin_rpy}"/>
155         </joint>
156
157         <link name="${name}_optical"></link>
158
159         <joint name="${name}_optical_joint" type="fixed">
160             <parent link="${name}"/>
161             <child link="${name}_optical"/>
162             <origin xyz="0 0 0" rpy="${-pi/2} 0 ${-pi/2}"/>
163         </joint>
164
165         <gazebo reference="${name}">
166             <sensor name="${name}_sensor" type="camera">
167                 <visualize>true</visualize>
168                 <camera>
169                     <horizontal_fov>1.3962634</horizontal_fov>
170                     <image>
171                         <width>640</width>
172                         <height>480</height>
173                         <format>R8G8B8</format>
174                     </image>
175                     <clip>
176                         <near>0.1</near>
177                         <far>15</far>
178                     </clip>
179                     <optical_frame_id>${name}_optical</optical_frame_id>
180                     <camera_info_topic>/${name}/camera_info</camera_info_topic>
181                 </camera>
182                 <always_on>1</always_on>
183                 <update_rate>20</update_rate>
184                 <visualize>true</visualize>
185                 <topic>/${name}/image</topic>
```

```
186             </sensor>
187         </gazebo>
188     </xacro:macro>
189 
190     <xacro:macro name="imu_sensor" params="name parent">
191         <gazebo reference="${parent}">
192             <sensor name="imu_sensor" type="imu">
193                 <always_on>1</always_on>
194                 <update_rate>50</update_rate>
195                 <visualize>true</visualize>
196                 <topic>/robot/${name}</topic>
197             </sensor>
198         </gazebo>
199     </xacro:macro>
200 
201     <!-- MAIN STRUCTURE -->
202     <!-- For Revolute joints parent_coord_system->joint_coordinate_system->
            child_coordinate_system -->
203     <!-- joint CS is relative to parent CS -->
204     <!-- child CS is relative to joint -->
205     <!-- for fixed joint all CS relative to world -->
206 
207     <xacro:voxel name="block0" origin="0 0 0" color="${red}"/>
208 
209     <xacro:block_ver block_number="1"  parent_number="0"    x="0"   y="0"    z="0" />
210 
211     <xacro:block_fix block_number="2"  parent_number="1"    x="0"   y="0"    z="1" />
212 
213     <xacro:block_hor block_number="3"  parent_number="2"    x="0"   y="1"    z="1" />
214 
215     <xacro:block_fix block_number="4"  parent_number="3"    x="0"   y="0"    z="0.5" />
216 
217     <xacro:block_fix block_number="5"  parent_number="4"    x="0"   y="0"    z="1.5" />
218 
219     <xacro:block_hor block_number="6"  parent_number="5"    x="0"   y="-1"   z="1.5" />
220 
221     <xacro:block_fix block_number="7"  parent_number="6"    x="0"   y="0"    z="0.5" />
222 
223     <xacro:block_fix block_number="8"  parent_number="7"    x="0"   y="0"    z="1.5" />
224 
225     <xacro:block_hor block_number="9"  parent_number="8"    x="0"   y="1"    z="1.5" />
226 
227     <xacro:block_ver block_number="10" parent_number="9"    x="0"   y="0"    z="-0.5" /
            >
228 
229     <xacro:block_end block_number="11" parent_number="10"   x="0"   y="0"    z="0" />
230 
231 
232     <!-- CAMERA PART -->
233     <xacro:camera_sensor name="camera1" parent="block0" origin_xyz="0 0 0" origin_rpy="
            0 ${pi/2} 0"/>
234     <xacro:camera_sensor name="camera2" parent="block11" origin_xyz="0 0 ${1*cell_step}
            " origin_rpy="0 ${-pi/2} 0"/>
235 
236     <!--  CONTACT sensors -->
237     <xacro:contact_plate name="contact1" parent="block0" origin="0 0 0"/>
238     <xacro:contact_plate name="contact2" parent="block11" origin="0 0 ${1*cell_step}"/>
239 
240     <!-- IMU sensors -->
241     <xacro:imu_sensor name="imu1" parent="block0"/>
242     <xacro:imu_sensor name="imu2" parent="block11"/>
243 
```

```
244
245     <!-- JOINTS CONTROL -->
246     <ros2_control name="GazeboSimSystem" type="system">
247         <hardware>
248             <plugin>gz_ros2_control/GazeboSimSystem</plugin>
249         </hardware>
250         <joint name="rev0_1">
251             <command_interface name="position"/>
252             <state_interface name="position"/>
253         </joint>
254         <joint name="rev2_3">
255             <command_interface name="position"/>
256             <state_interface name="position"/>
257         </joint>
258         <joint name="rev5_6">
259             <command_interface name="position"/>
260             <state_interface name="position"/>
261         </joint>
262         <joint name="rev8_9">
263             <command_interface name="position"/>
264             <state_interface name="position"/>
265         </joint>
266         <joint name="rev9_10">
267             <command_interface name="position"/>
268             <state_interface name="position"/>
269         </joint>
270     </ros2_control>
271
272
273
274     <gazebo>
275         <plugin filename="gz_ros2_control-system" name="
                gz_ros2_control::GazeboSimROS2ControlPlugin">
276             <!-- <parameters>$(find srrs_sim)/config/two_blocks_controller.yaml</
                    parameters> -->
277             <parameters>$(find srrs_sim)/config/robot_controller.yaml</parameters>
278         </plugin>
279
280         <plugin
281             filename="libgz-sim8-detachable-joint-system.so"
282             name="gz::sim::systems::DetachableJoint">
283             <parent_link>block0</parent_link>
284             <child_model>base</child_model>
285             <child_link>base_link</child_link>
286             <attach_topic>attach_end1</attach_topic>
287             <detach_topic>detach_end1</detach_topic>
288             <output_topic>detachable_link_state1</output_topic>
289         </plugin>
290
291         <plugin
292             filename="libgz-sim8-detachable-joint-system.so"
293             name="gz::sim::systems::DetachableJoint">
294             <parent_link>block11</parent_link>
295             <child_model>base</child_model>
296             <child_link>base_link</child_link>
297             <attach_topic>attach_end2</attach_topic>
298             <detach_topic>detach_end2</detach_topic>
299             <output_topic>detachable_link_state2</output_topic>
300         </plugin>
301
302         <plugin
303             filename="gz-sim-contact-system"
```

```
            name="gz::sim::systems::Contact">
        </plugin>
    </gazebo>

</robot>
```

A.4.3 part.xacro

```
<?xml version="1.0" ?>
<robot name="part" xmlns:xacro="http://www.ros.org/wiki/xacro">
    <xacro:arg name="part_num" default="0"/>
    <!-- <xacro:arg name="parent_model" default="0"/> -->
    <!-- <xacro:arg name="parent_link" default="0"/> -->

    <xacro:property name="green" value="0 0.8 0 1" />
    <xacro:property name="red" value="0.8 0 0 1" />
    <xacro:property name="orange" value="1 0.65 0 1" />
    <xacro:property name="blue" value="0 0 0.8 1" />
    <xacro:property name="yellow" value="1 1 0 1" />
    <xacro:property name="white" value="1 1 1 1" />
    <xacro:property name="grey" value="0.2 0.2 0.2 1" />
    <xacro:property name="black" value="0 0 0 1" />

    <link name="part">
        <visual>
            <geometry>
                <mesh filename="/home/lao/Documents/Masterarbeit/git/SRRS_gazebo_sim/
                    ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="0.001 0.001 0.001"/>
                <!-- <box size="0.1 0.1 0.1"/> -->
            </geometry>
        </visual>
        <collision name="part_collision">
            <geometry>
                <mesh filename="/home/lao/Documents/Masterarbeit/git/SRRS_gazebo_sim/
                    ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="0.001 0.001 0.001"/>
                <!-- <box size="0.1 0.1 0.1"/> -->
            </geometry>
        </collision>
        <inertial>
            <mass value="0.1"/>
            <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0" izz="1.0"/>
        </inertial>
    </link>

    <gazebo reference="part">
        <visual>
            <material>
                <ambient>${black}</ambient>
                <diffuse>${black}</diffuse>
            </material>
        </visual>

    </gazebo>
    <gazebo>
        <!-- <plugin filename="gz_ros2_control-system" name="
            gz_ros2_control::GazeboSimROS2ControlPlugin">
      <parameters>$(find srrs_sim)/config/robot_controller.yaml</parameters>
    </plugin> -->

        <plugin
```

```
50              filename="libgz-sim8-detachable-joint-system.so"
51              name="gz::sim::systems::DetachableJoint">
52              <parent_link>part</parent_link>
53              <child_model>robot</child_model>
54              <child_link>block0</child_link>
55              <attach_topic>attach_end1_part${xacro.arg('part_num')}</attach_topic>
56              <detach_topic>detach_end1_part${xacro.arg('part_num')}</detach_topic>
57              <output_topic>detachable_obj_end1_state${xacro.arg('part_num')}</
                    output_topic>
58              <suppress_child_warning>true</suppress_child_warning>
59          </plugin>
60
61          <plugin
62              filename="libgz-sim8-detachable-joint-system.so"
63              name="gz::sim::systems::DetachableJoint">
64              <parent_link>part</parent_link>
65              <child_model>robot</child_model>
66              <child_link>block11</child_link>
67              <attach_topic>attach_end2_part${xacro.arg('part_num')}</attach_topic>
68              <detach_topic>detach_end2_part${xacro.arg('part_num')}</detach_topic>
69              <output_topic>detachable_obj_end2_state${xacro.arg('part_num')}</
                    output_topic>
70              <suppress_child_warning>true</suppress_child_warning>
71          </plugin>
72
73          <!-- TODO: correct plugin to connect to previous block -->
74          <plugin
75              filename="libgz-sim8-detachable-joint-system.so"
76              name="gz::sim::systems::DetachableJoint">
77              <parent_link>part</parent_link>
78              <child_model>base</child_model>
79              <child_link>base_link</child_link>
80
81              <!-- <child_model>${xacro.arg('parent_model')}</child_model> -->
82              <!-- <child_link>${xacro.arg('parent_link')}</child_link> -->
83
84              <attach_topic>attach_part${xacro.arg('part_num')}_base</attach_topic>
85              <detach_topic>detach_part${xacro.arg('part_num')}_base</detach_topic>
86              <!---->
87              <!-- <output_topic>detachable_obj_state${xacro.arg('parent_model')}</
                    output_topic> -->
88
89              <suppress_child_warning>true</suppress_child_warning>
90          </plugin>
91
92          <plugin filename="gz-sim-pose-publisher-system" name="
                gz::sim::systems::PosePublisher">
93              <!-- <publish_model_pose>true</publish_model_pose> -->
94              <use_pose_vector_msg>false</use_pose_vector_msg>
95              <update_frequency>30</update_frequency>
96              <publish_nested_model_pose>true</publish_nested_model_pose>
97          </plugin>
98      </gazebo>
99
100 </robot>
```

B AI reference index

B.1 Use of AI for publications search

System: Perplexity

URL: https://www.perplexity.ai/

Date of use: June-August 2025

Purpose: Search for related scientific papers during the preparation of Background chapter.

Description: During the preparation process, AI was used to create a list of relevant scientific articles, working groups and published results regarding Self-Replication systems and implementation of Formal Methods to Robotics.

Result: The use of AI has accelerated the search process and helped to reflect the entire spectrum of the current state of research in a field of SRRS and application of Formal Methods. Around 20% of scientific sources were found by this AI tool. The results and found publications were used in Chapter 2.

B.2 Use of AI for code debugging and improvement

System: Claude

URL: https://claude.ai

Date of use: June-September 2025

Purpose: Code debugging and improvement

Description: During the code development process, AI was used to debug and discover solutions for the ROS2-Gazebo simulation issues and to implement best practices for Python3 software development.

Result: The use of AI has accelerated the development of Gazebo simulation and ROS2 control Nodes in Chapter 6 and create a more comprehensible and clean code for Chapters 5 and 7. All the software solutions described in these chapters are my own, from concept to implementation. AI was used only as an auxiliary system for finding errors and applying best design practices.

B.3 Use of AI for LaTeX support

System: ChatGPT 5

URL: https://chatgpt.com/

Date of use: July-September 2025

Purpose: Suggestions for LaTeX syntax and layout

Description: During the Thesis writing process, AI was used to clarify certain commands and techniques in LaTeX (e.g., figure references, bibliography formatting).

Result: The use of AI has slightly accelerated the understanding of LaTeX syntax and reduced the number of errors in formatting the document.

B.4 Use of AI for rephrasing, grammar correction and synonym search

System: ChatGPT-5

URL: https://chatgpt.com/

Date of use: August - October 2025

Description: AI was used to select synonyms and alternative formulations for my own sentences, and to correct grammar. The thoughts and arguments remained mine; AI only helped to find a clearer or less repetitive way of expressing them. This made the text more smooth and reduced the number of stylistic repetitions.

Result: The main meaning was preserved, but the wording became clearer and more varied, which improved readability and the overall quality of the work.